

LECTURE 18
Speculative Parallelism

Xuhao Chen
April 16, 2025



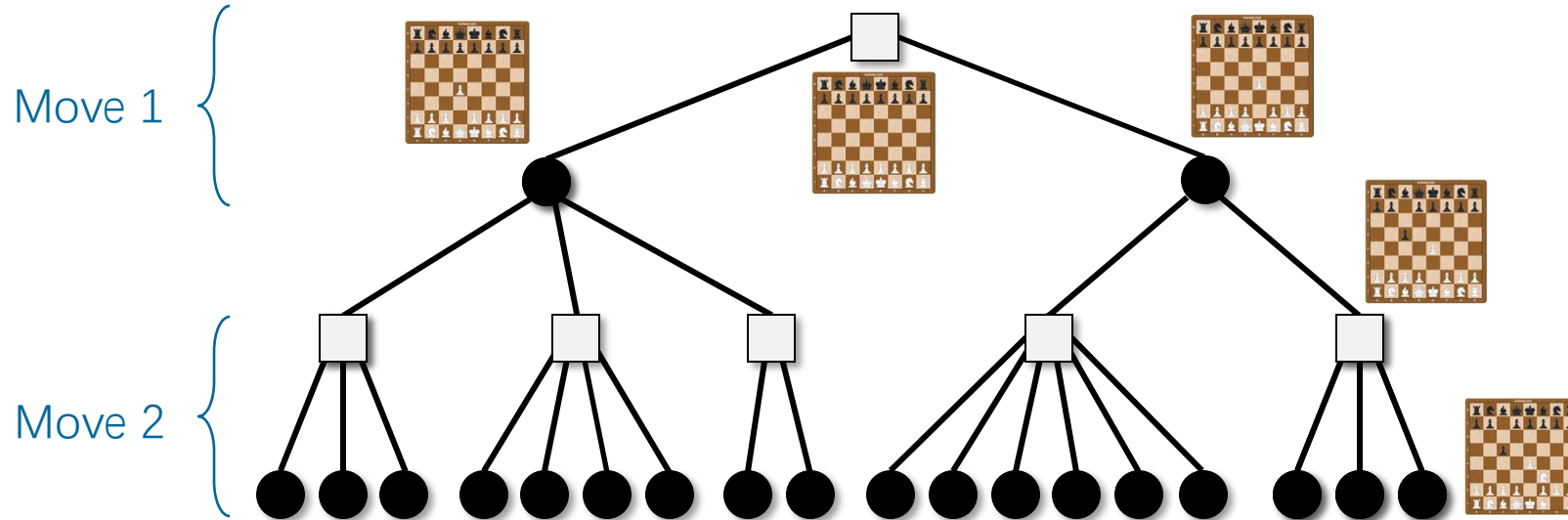
Outline

- Computer Chess
- Alpha-Beta Search
 - ▣ The Idea
 - ▣ The Code
 - ▣ A worked example
 - ▣ Analysis
- Speculative Parallelism
- Parallel Alpha-Beta Search

COMPUTER CHESS



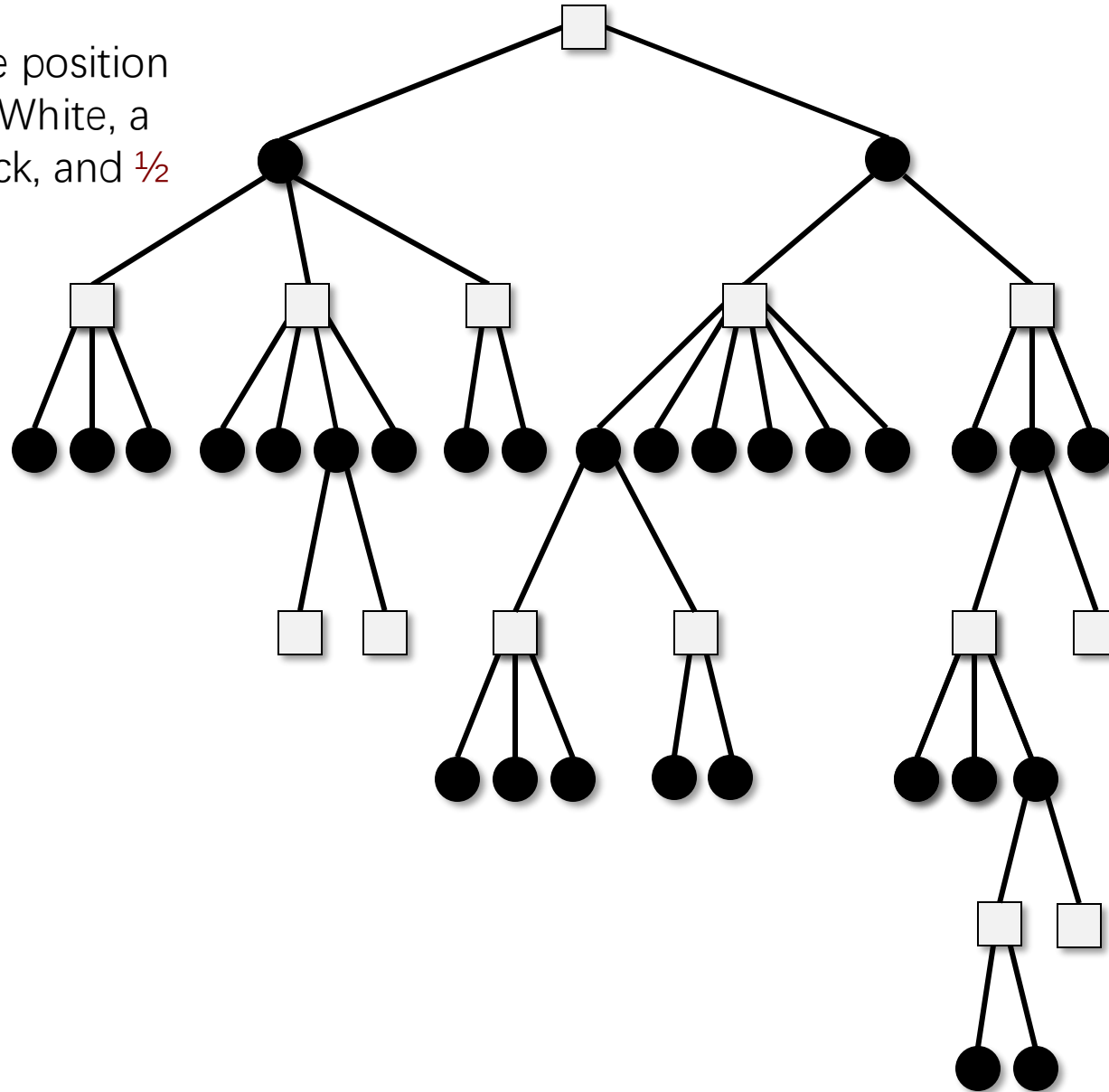
Game Trees



- Two players: White  and Black .
- Each node in the tree corresponds to a game position.
- A ply is one level in the game tree. An even ply and the next deeper odd ply form a move.
- On even plies, it's White's turn to move. On odd plies, it's Black's turn.
- An edge from node p to node c means that there is a move in the game from p that yields c .
- The value of a leaf is 1 if the position is a win for White, a 0 if it is a win for Black, and $\frac{1}{2}$ if it is a draw.
- On their turns, each player tries to find a move that leads to a win.

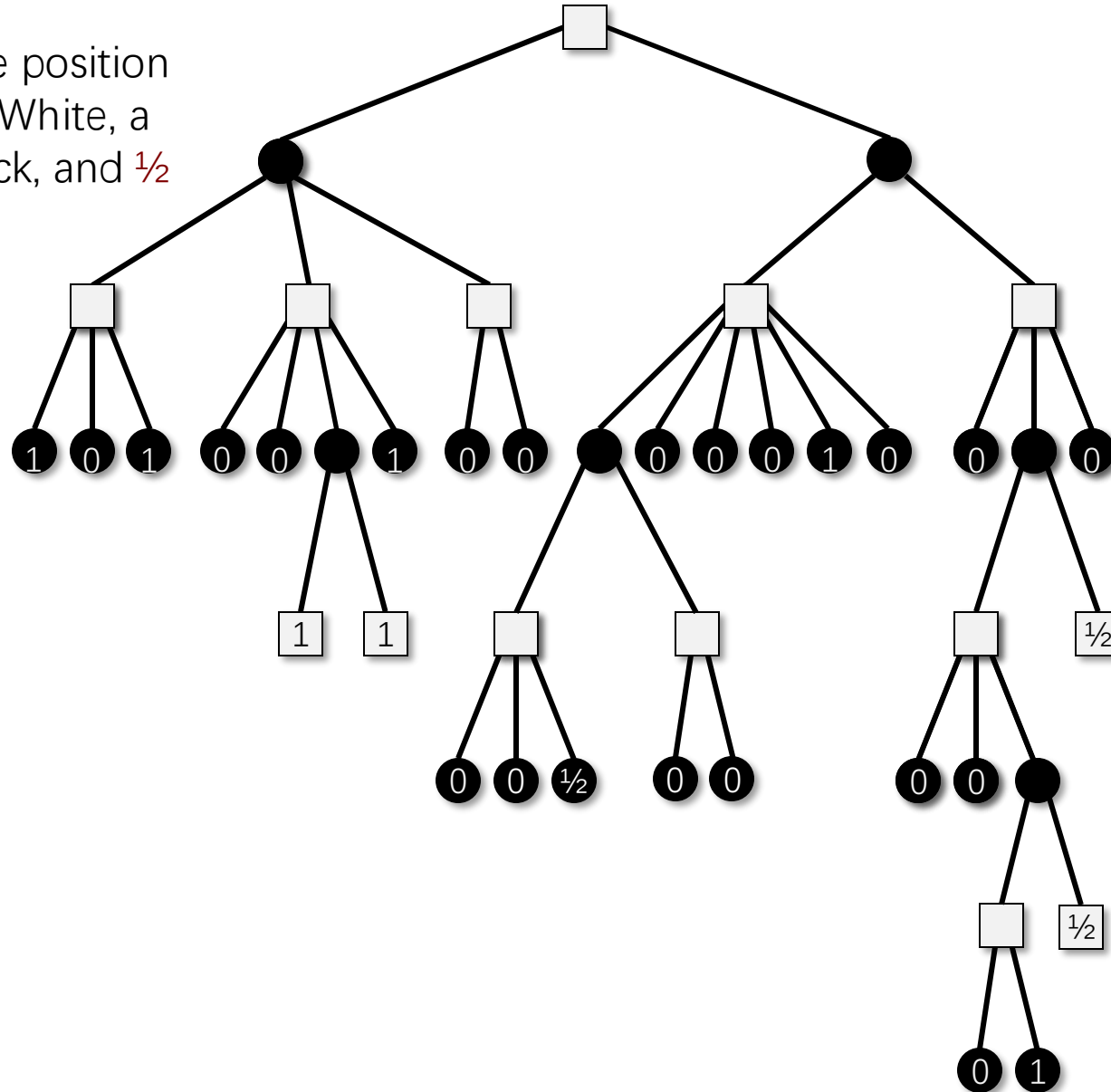
Min-Max Game Trees

The **value** of a leaf is **1** if the position represents a checkmate by White, a **0** if it is a checkmate by Black, and **$\frac{1}{2}$** if it is a draw.



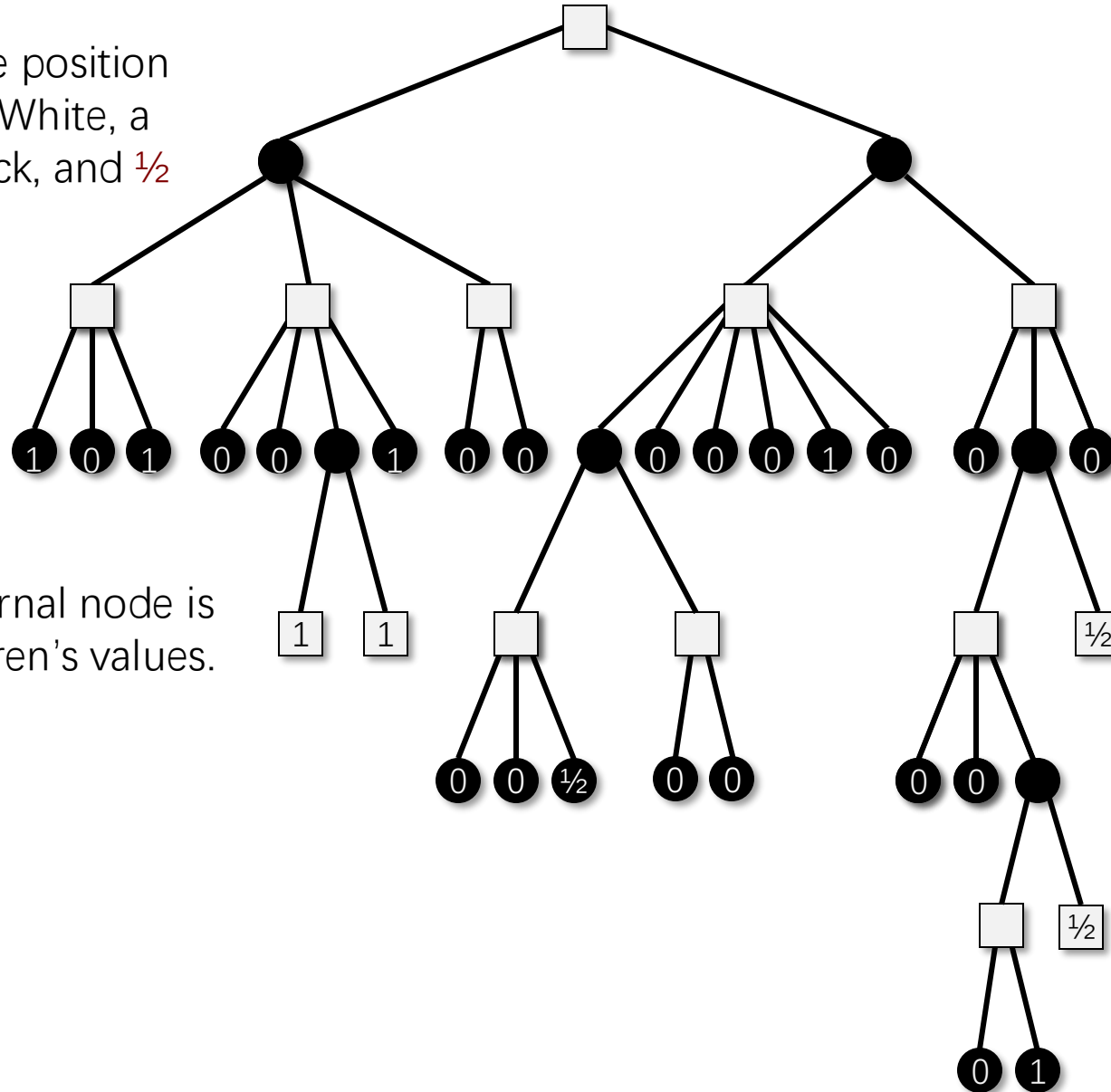
Game Trees

The **value** of a leaf is **1** if the position represents a checkmate by White, a **0** if it is a checkmate by Black, and $\frac{1}{2}$ if it is a draw.



Game Trees

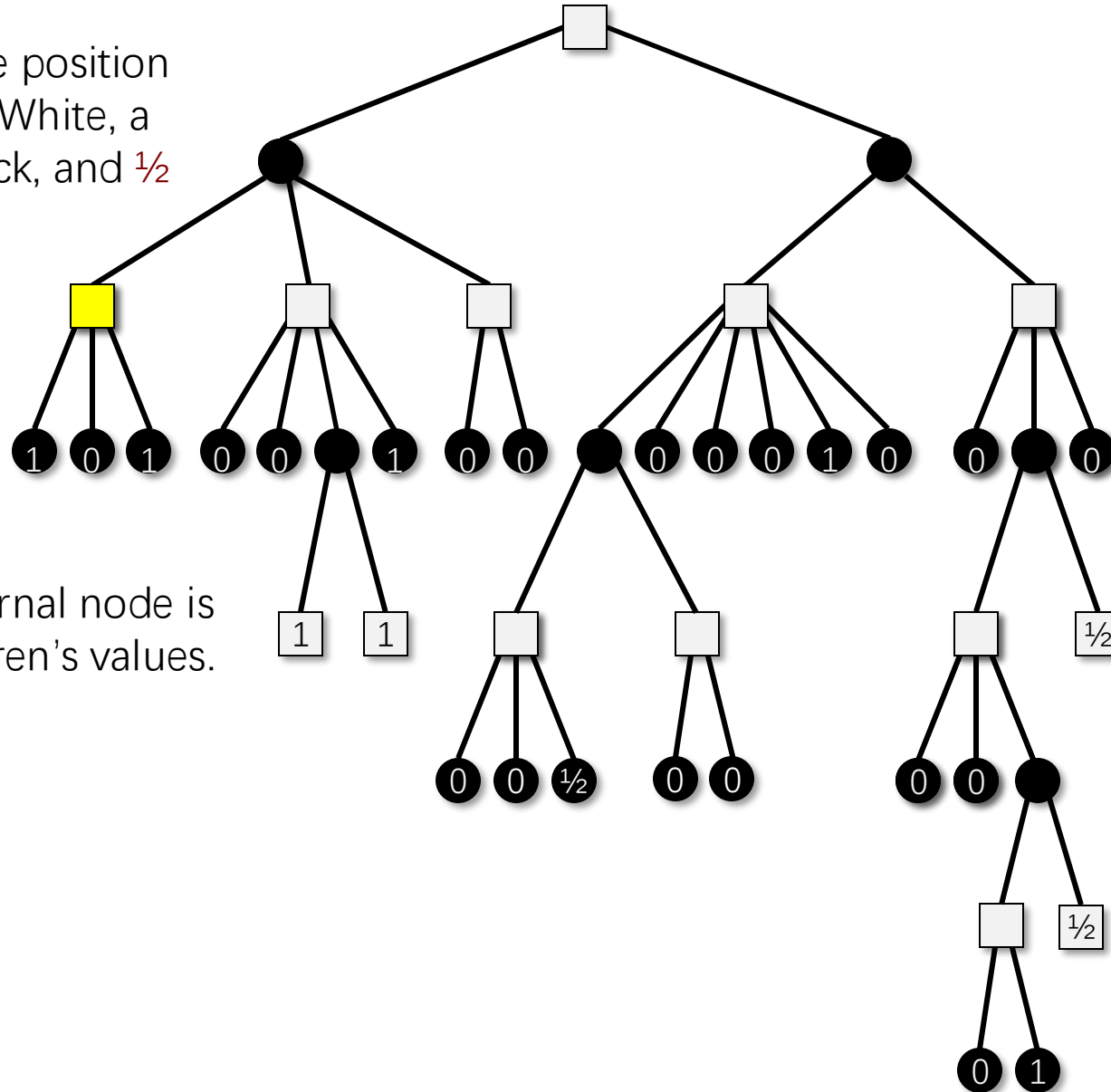
The **value** of a leaf is **1** if the position represents a checkmate by White, a **0** if it is a checkmate by Black, and **$\frac{1}{2}$** if it is a draw.



- The value of a White internal node is the maximum of its children's values.

Game Trees

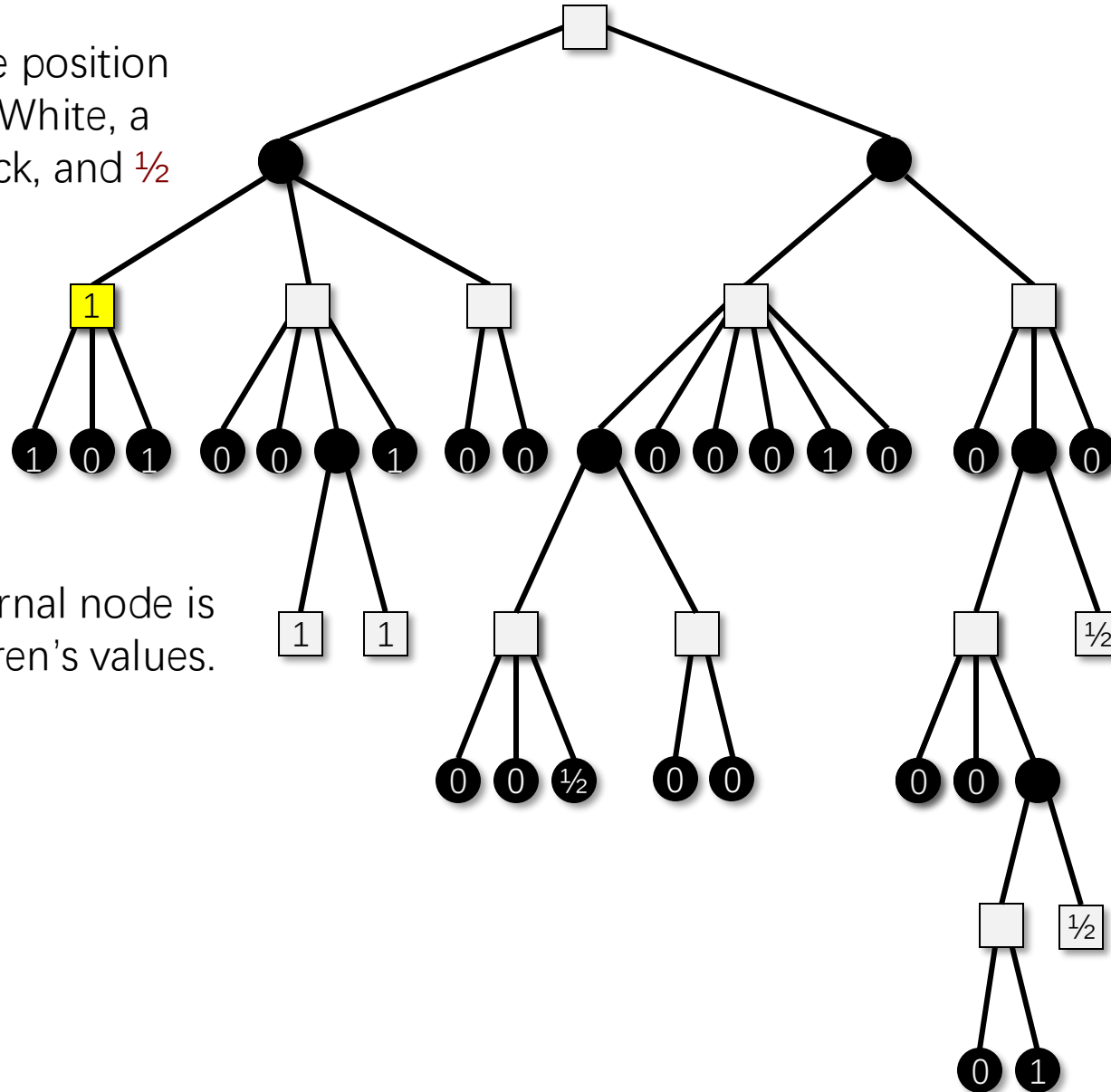
The **value** of a leaf is **1** if the position represents a checkmate by White, a **0** if it is a checkmate by Black, and $\frac{1}{2}$ if it is a draw.



- The value of a White internal node is the maximum of its children's values.

Game Trees

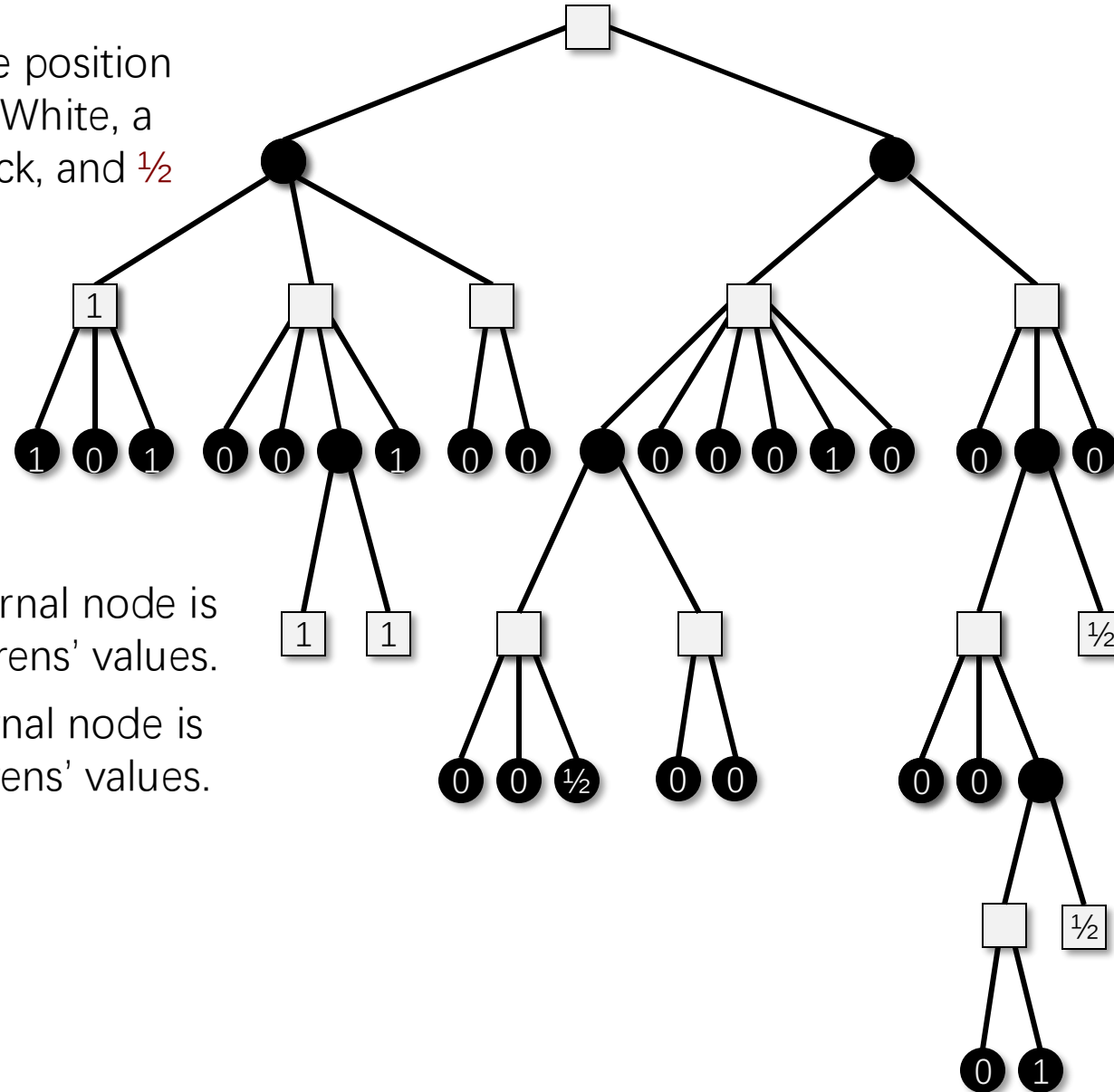
The **value** of a leaf is **1** if the position represents a checkmate by White, a **0** if it is a checkmate by Black, and $\frac{1}{2}$ if it is a draw.



- The value of a White internal node is the maximum of its children's values.

Game Trees

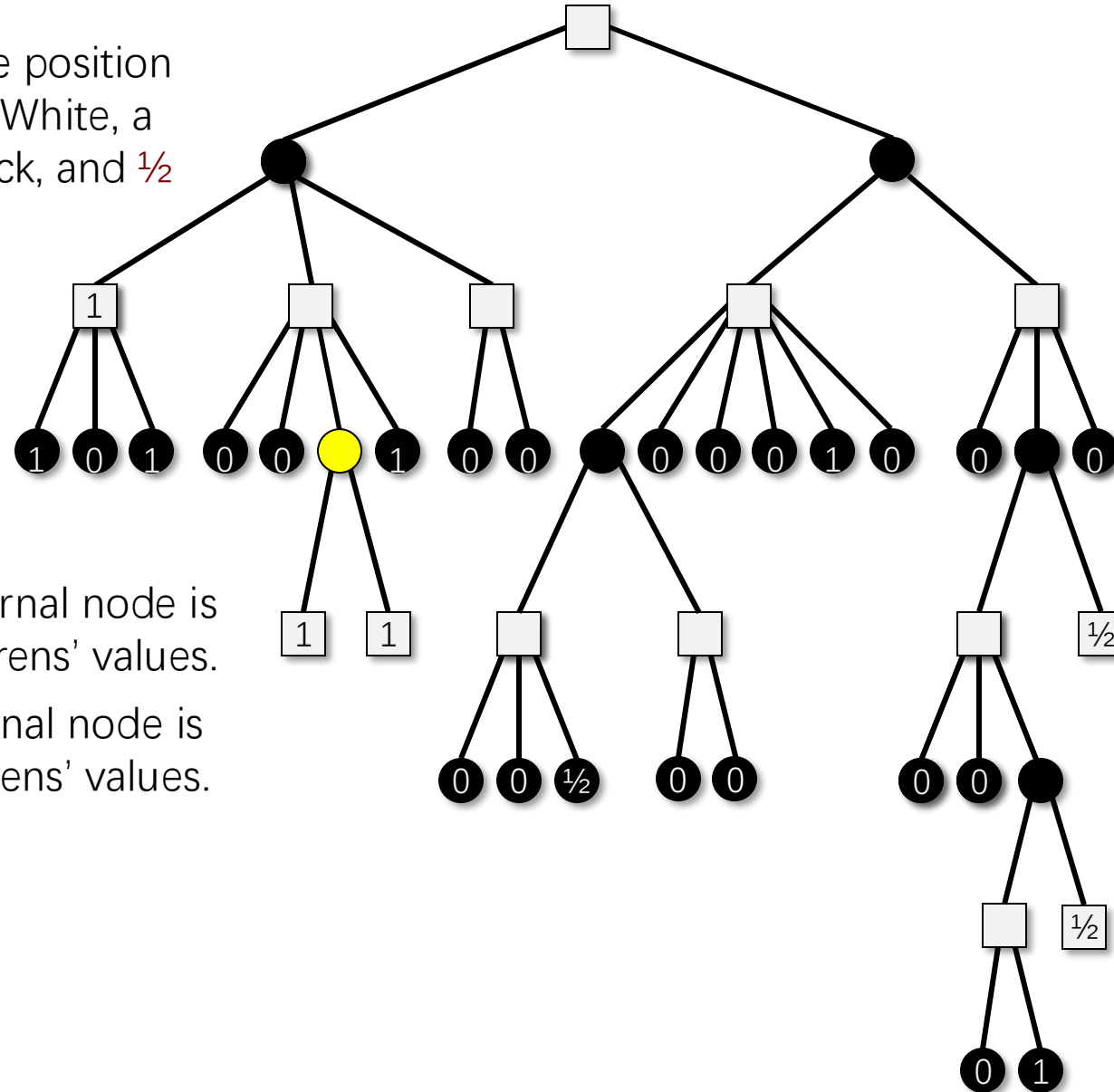
The **value** of a leaf is **1** if the position represents a checkmate by White, a **0** if it is a checkmate by Black, and **$\frac{1}{2}$** if it is a draw.



- The value of a White internal node is the maximum of its childrens' values.
- The value of a Black internal node is the minimum of its childrens' values.

Game Trees

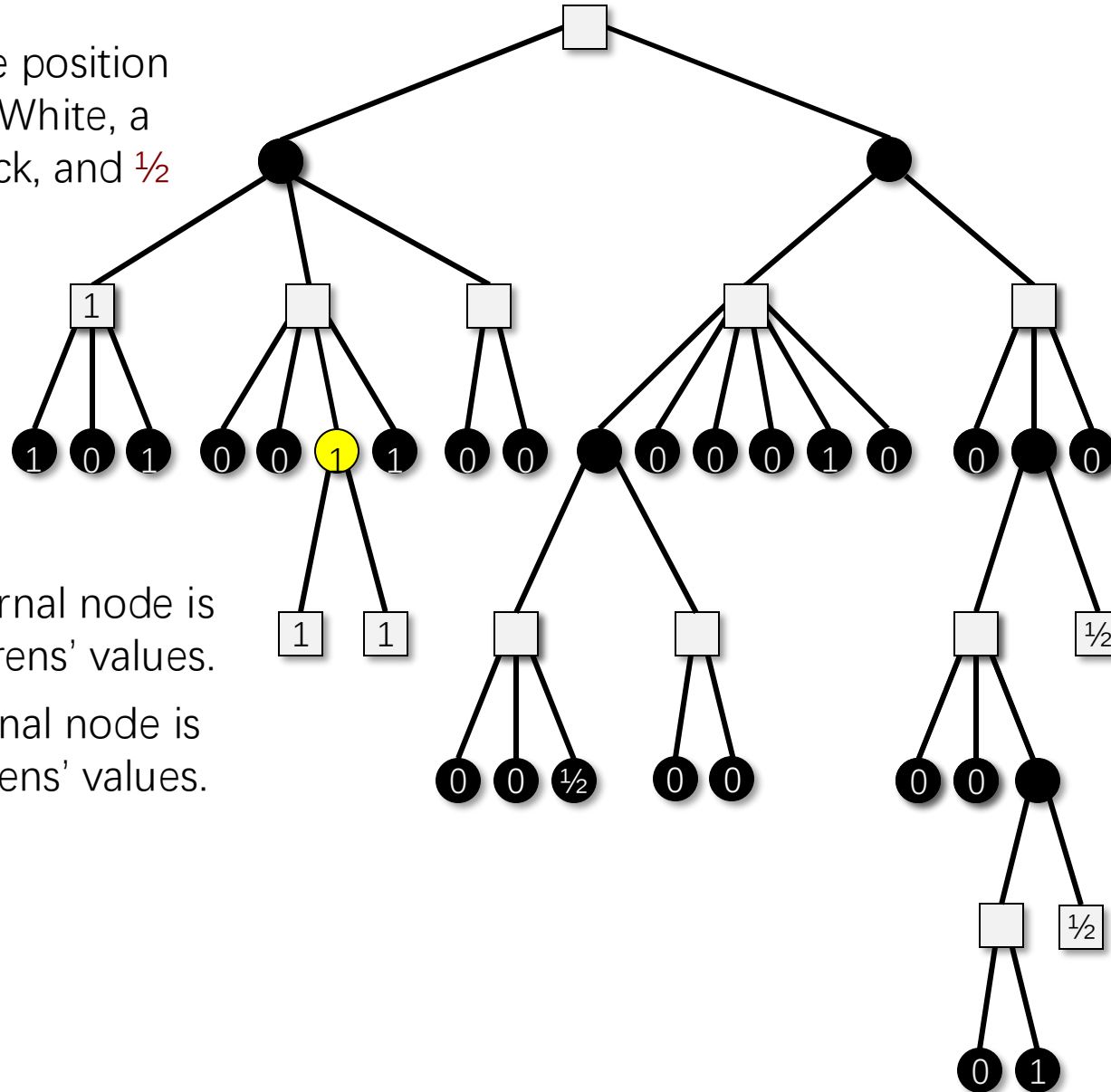
The **value** of a leaf is **1** if the position represents a checkmate by White, a **0** if it is a checkmate by Black, and **$\frac{1}{2}$** if it is a draw.



- The value of a White internal node is the maximum of its childrens' values.
- The value of a Black internal node is the minimum of its childrens' values.

Game Trees

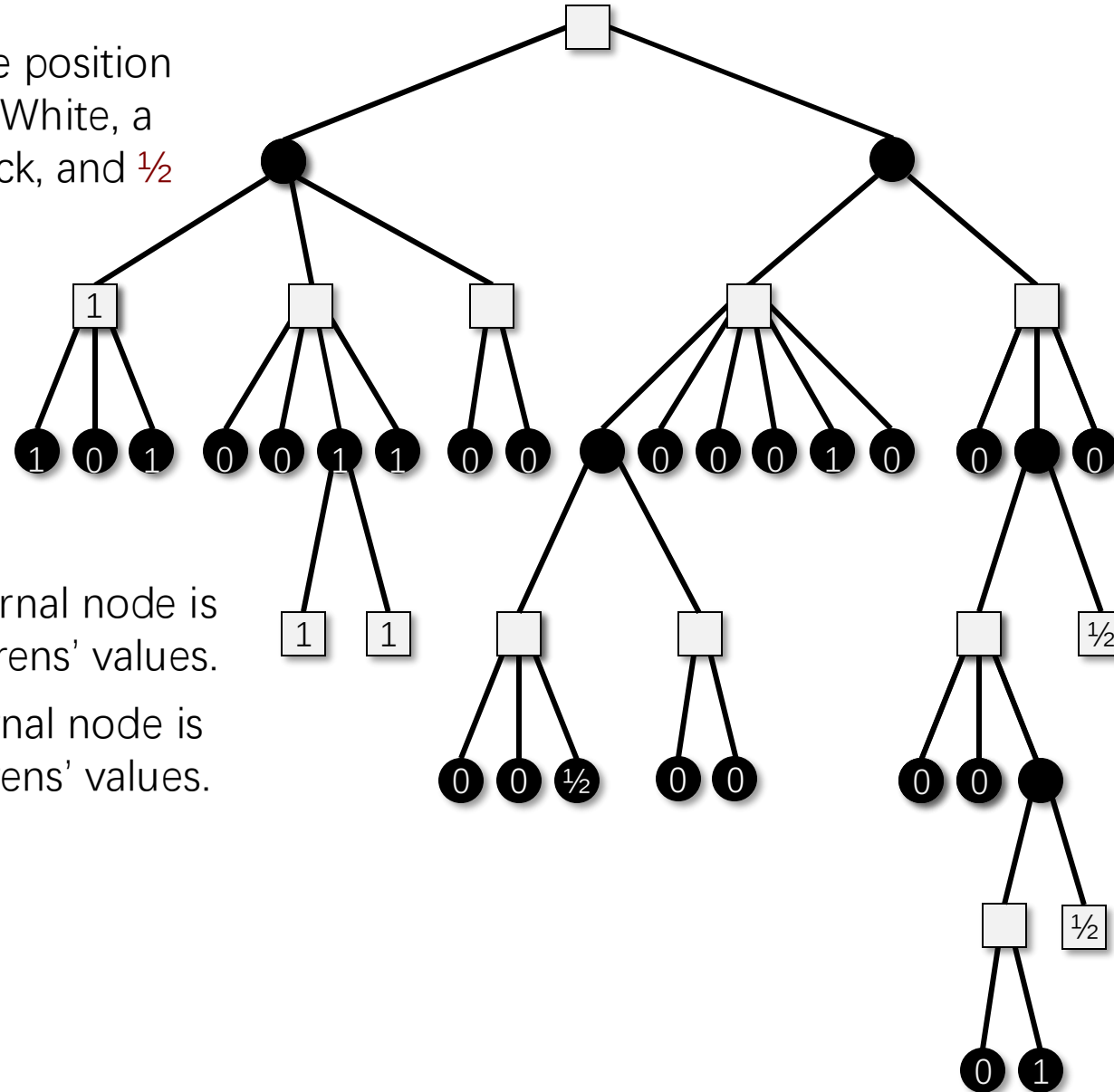
The **value** of a leaf is **1** if the position represents a checkmate by White, a **0** if it is a checkmate by Black, and **$\frac{1}{2}$** if it is a draw.



- The value of a White internal node is the maximum of its childrens' values.
- The value of a Black internal node is the minimum of its childrens' values.

Game Trees

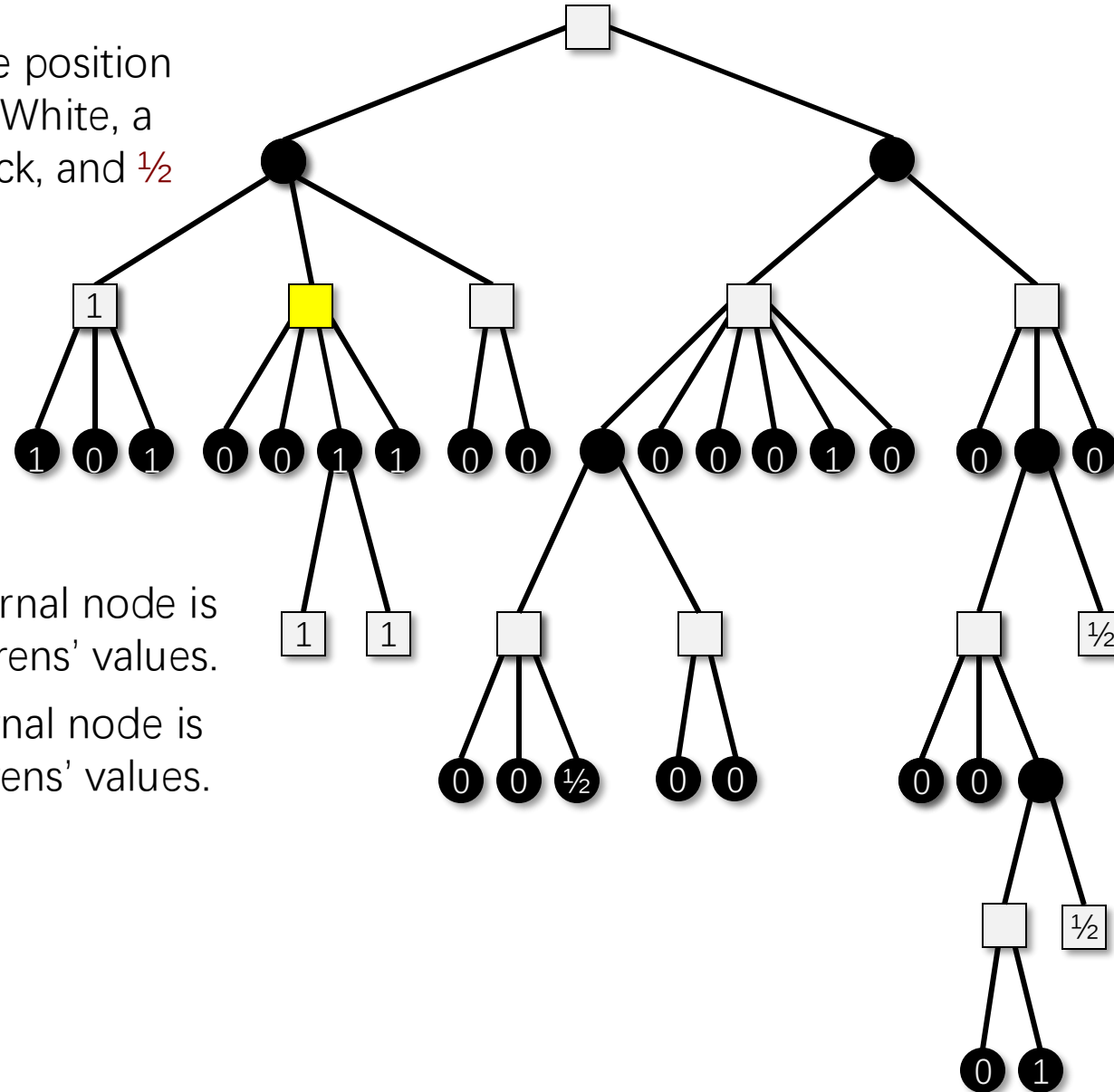
The **value** of a leaf is **1** if the position represents a checkmate by White, a **0** if it is a checkmate by Black, and **$\frac{1}{2}$** if it is a draw.



- The value of a White internal node is the maximum of its children's values.
- The value of a Black internal node is the minimum of its children's values.

Game Trees

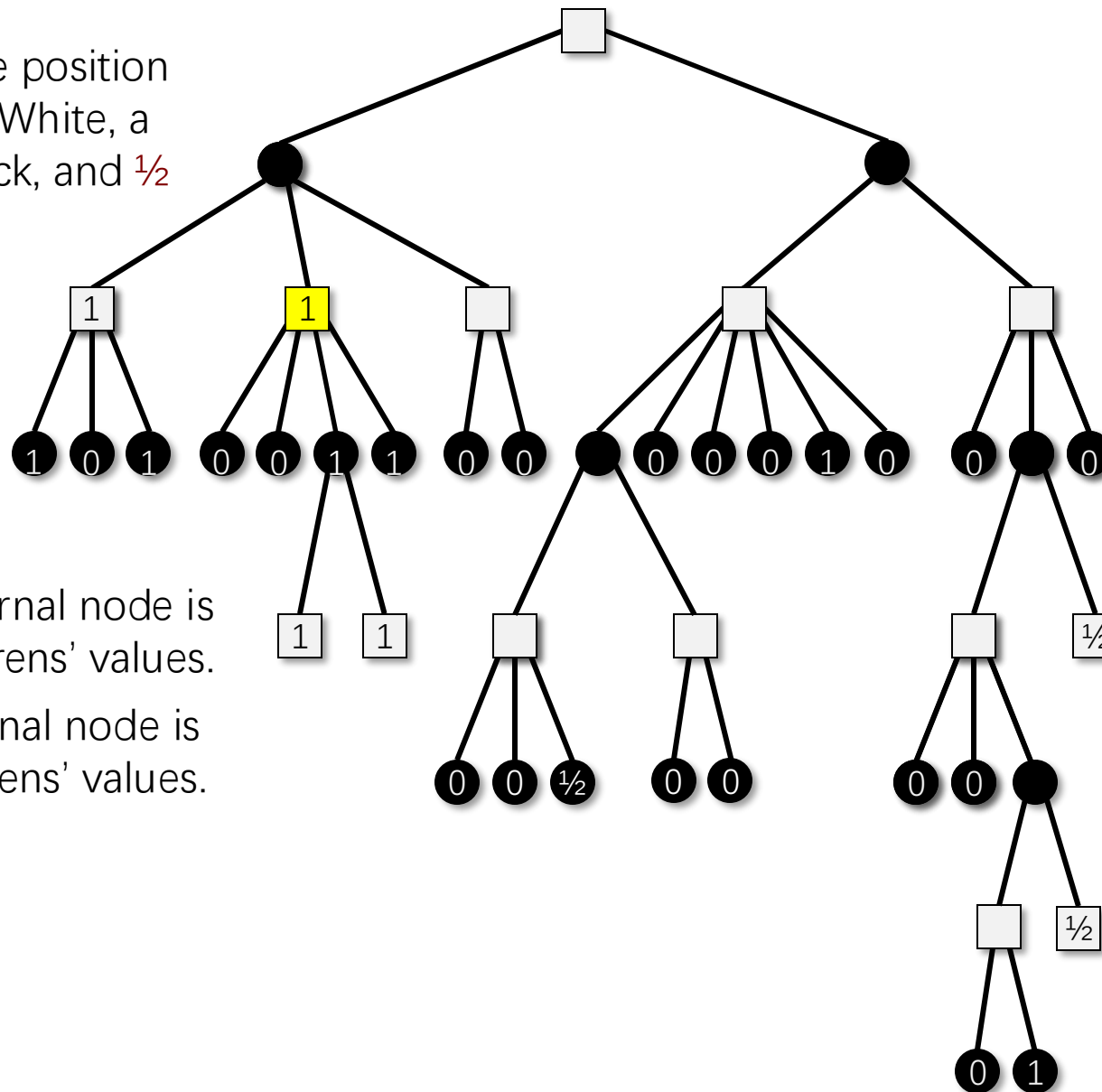
The **value** of a leaf is **1** if the position represents a checkmate by White, a **0** if it is a checkmate by Black, and **$\frac{1}{2}$** if it is a draw.



- The value of a White internal node is the maximum of its children's values.
- The value of a Black internal node is the minimum of its children's values.

Game Trees

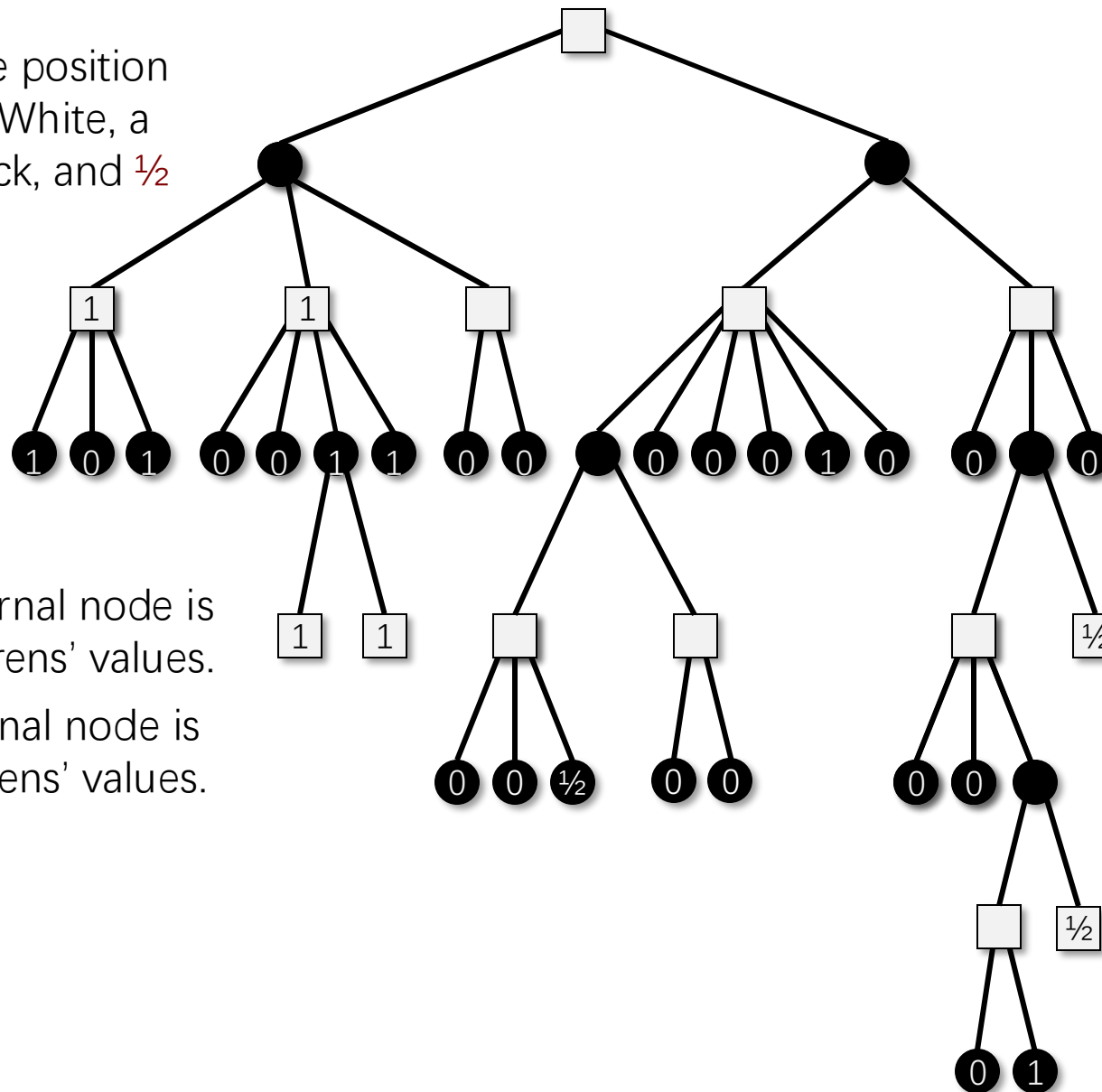
The **value** of a leaf is **1** if the position represents a checkmate by White, a **0** if it is a checkmate by Black, and $\frac{1}{2}$ if it is a draw.



- The value of a White internal node is the maximum of its childrens' values.
- The value of a Black internal node is the minimum of its childrens' values.

Game Trees

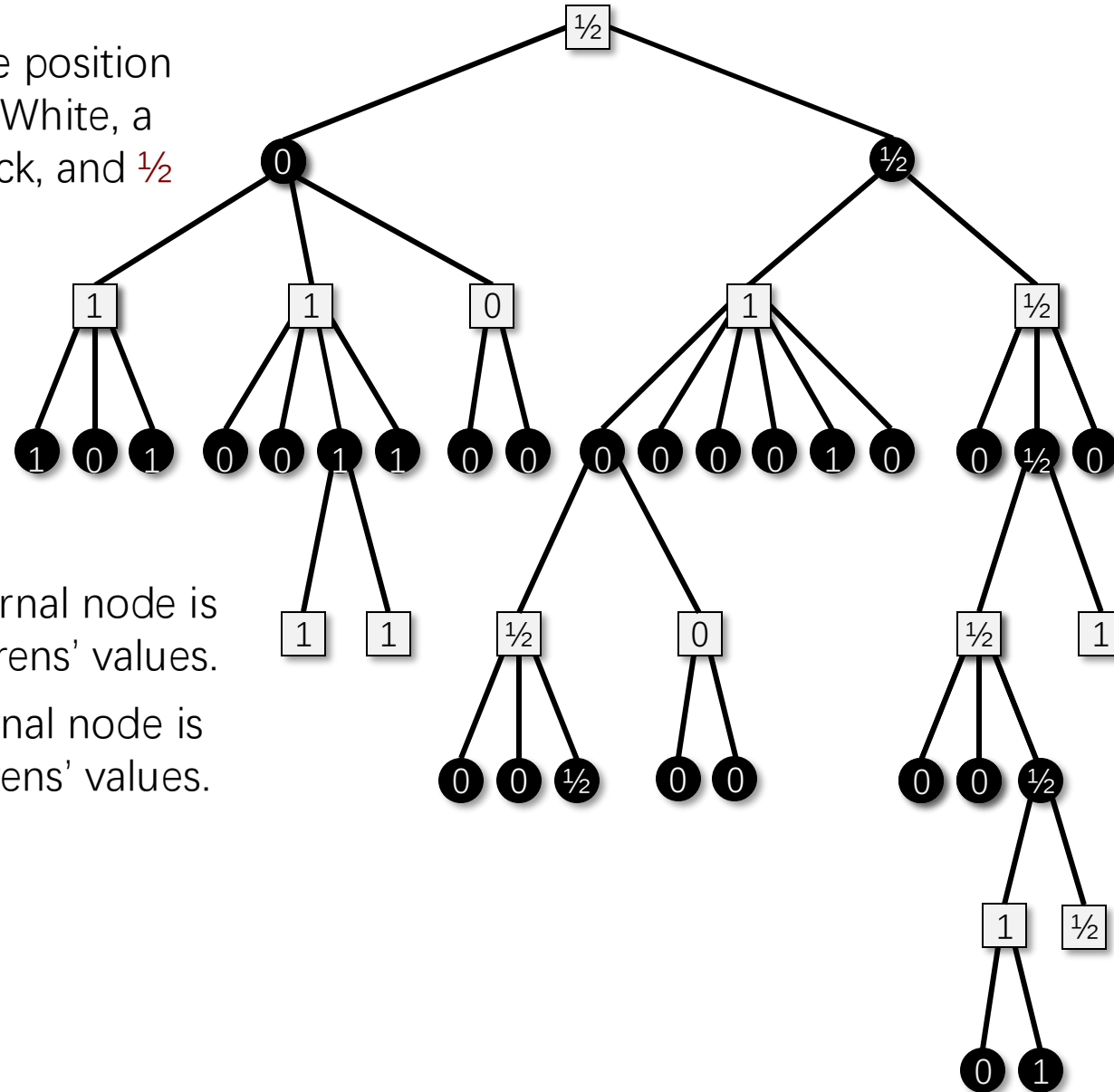
The **value** of a leaf is **1** if the position represents a checkmate by White, a **0** if it is a checkmate by Black, and $\frac{1}{2}$ if it is a draw.



- The value of a White internal node is the maximum of its childrens' values.
- The value of a Black internal node is the minimum of its childrens' values.

Game Trees

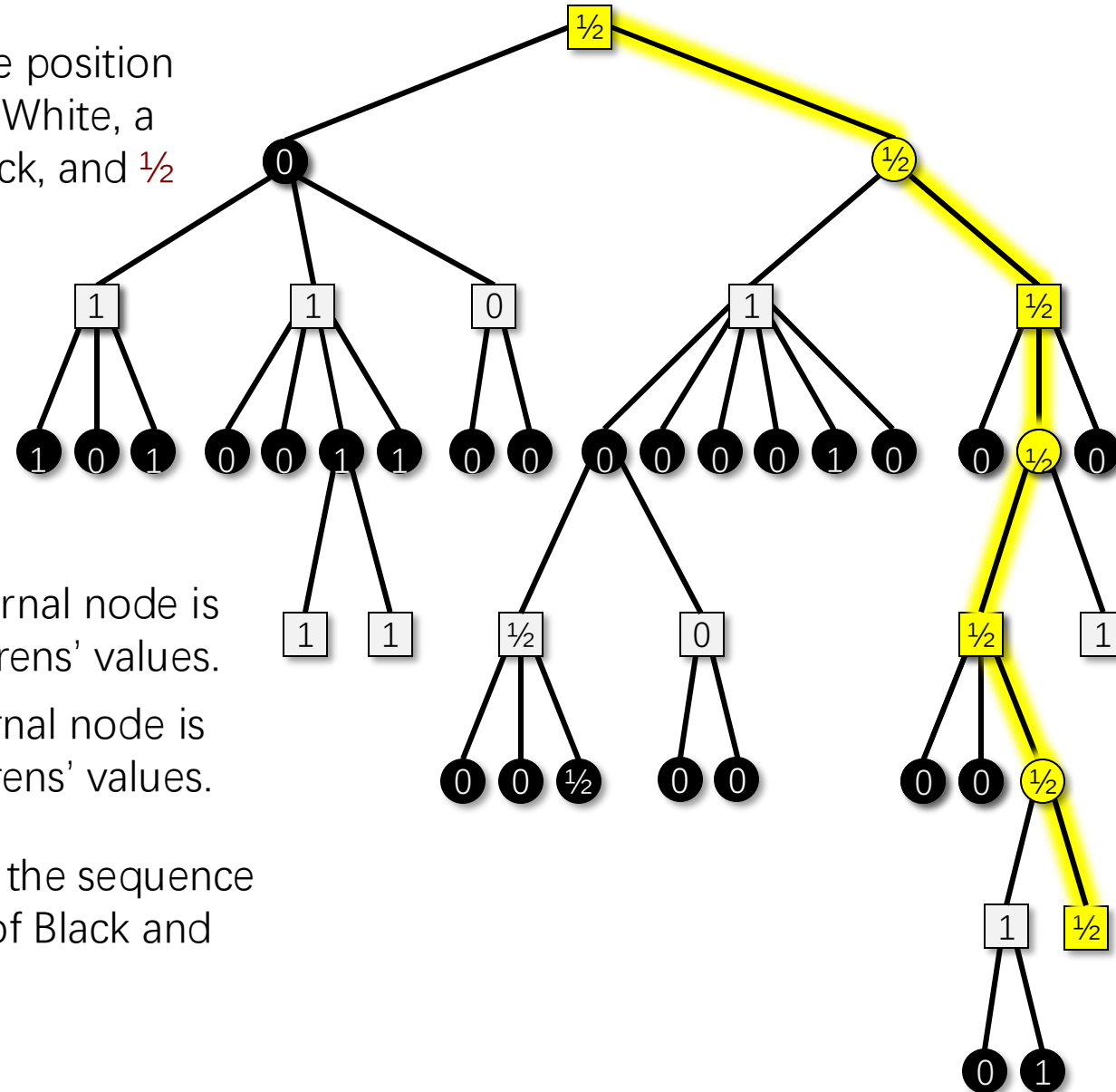
The **value** of a leaf is **1** if the position represents a checkmate by White, a **0** if it is a checkmate by Black, and $\frac{1}{2}$ if it is a draw.



- The value of a White internal node is the maximum of its childrens' values.
- The value of a Black internal node is the minimum of its childrens' values.

Game Trees

The **value** of a leaf is **1** if the position represents a checkmate by White, a **0** if it is a checkmate by Black, and $\frac{1}{2}$ if it is a draw.



- The value of a White internal node is the **maximum** of its childrens' values.
- The value of a Black internal node is the **minimum** of its childrens' values.
- The **principal variation** is the sequence of moves in which each of Black and White play optimally.

Impossibility

- Computing the game tree for chess is impossible
- Claude Shannon & Alan Turing made the first modern progress on chess-playing programs — estimated 10^{120} as the number of reachable chess positions
- A recent estimate by Shirish Chinchalkar puts an upper bound at “only” around $10^{46.25}$, far more than one can reasonably compute
 - ▣ If each chess position took only 1 nanosecond to compute, it would still take over 10^{27} centuries to enumerate them



Claude Shannon

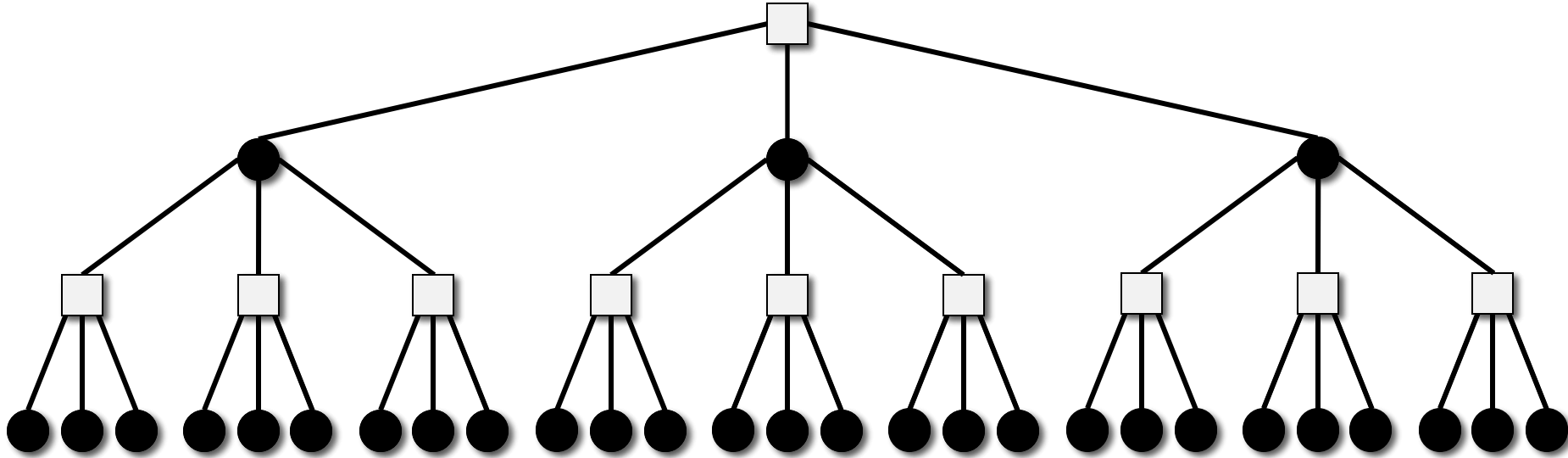


Shirish Chinchalkar



Alan Turing

Compromise Heuristic



Basic strategy

- Search to some fixed depth **d**.
- Use a **static evaluation** at the leaves to estimate the expected value of the leaf (no longer just 0, $\frac{1}{2}$, and 1).
- Back up the values, using min and max as before, to obtain the value of the root.

Bug in the Heuristic

- The static evaluation is intended to provide an estimate of the strategic value of a position
- It does not model tactics, which are best discovered by the search
- Consequently, if the board position at depth **d** is unstable, the static evaluation may return inaccurate results
 - ▣ **Example:** Suppose that White's Queen captures a Knight at the end of the depth-**d** search. Then the static evaluation will think that White is up a Knight. But if Black can respond by capturing the White Queen with a Pawn, it's really Black who is up far more
 - ▣ **Solution: Quiescence.** Before static evaluation, play off all captures to quiet the position. (A player can "pass" if a capture would hurt.) Now, the static evaluation can return the strategic value of the position without tactical "noise."

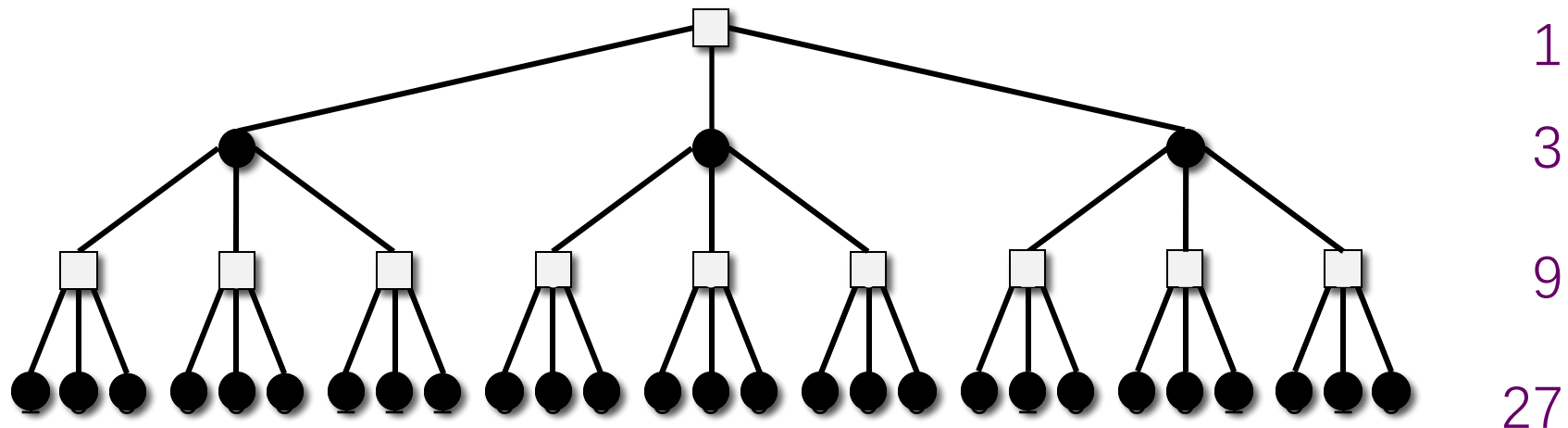
Combinatorial Explosion

A depth-**d** search can produce an exponentially large search tree.

- The number of nodes in a game tree with branching factor $b > 1$ and depth $d \geq 0$ is

$$1 + b + b^2 + \dots + b^d = (b^{d+1} - 1) / (b - 1) \\ \approx b^d.$$

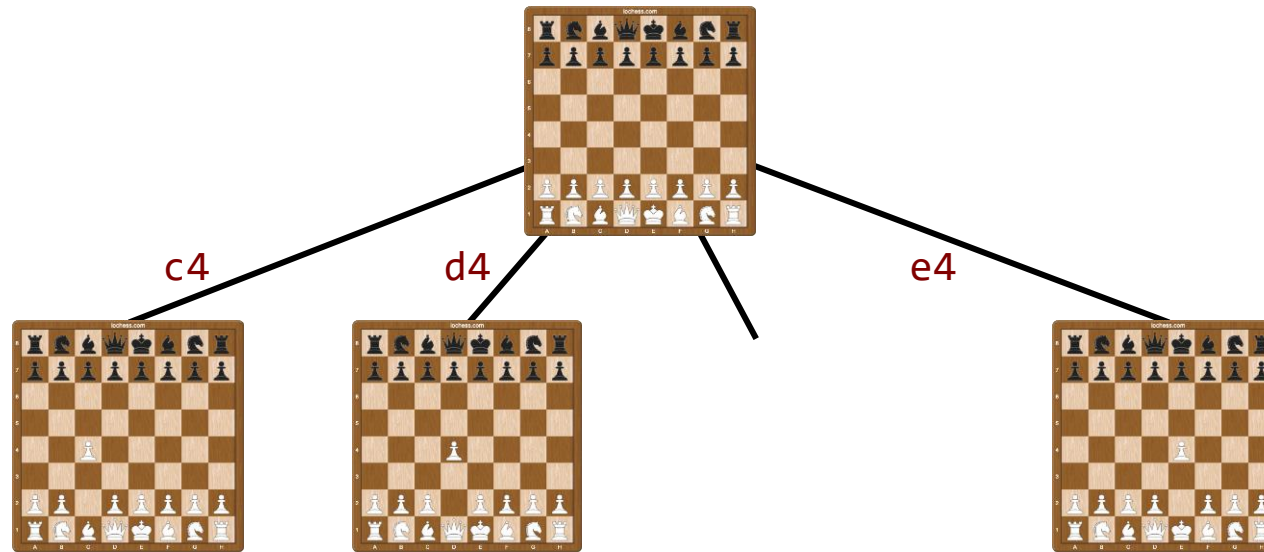
Example: $b = 3$



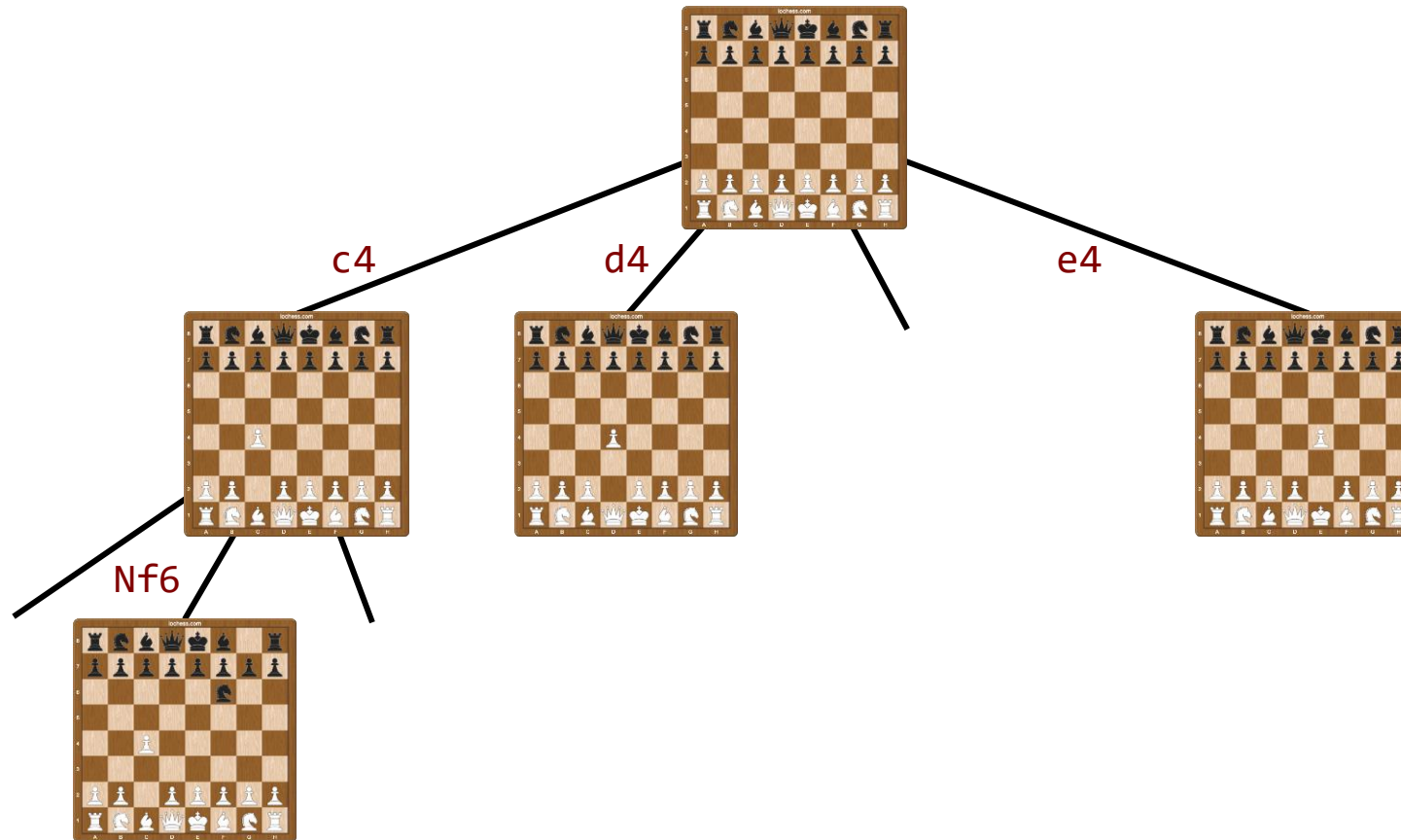
The Game Tree Is Really a Dag



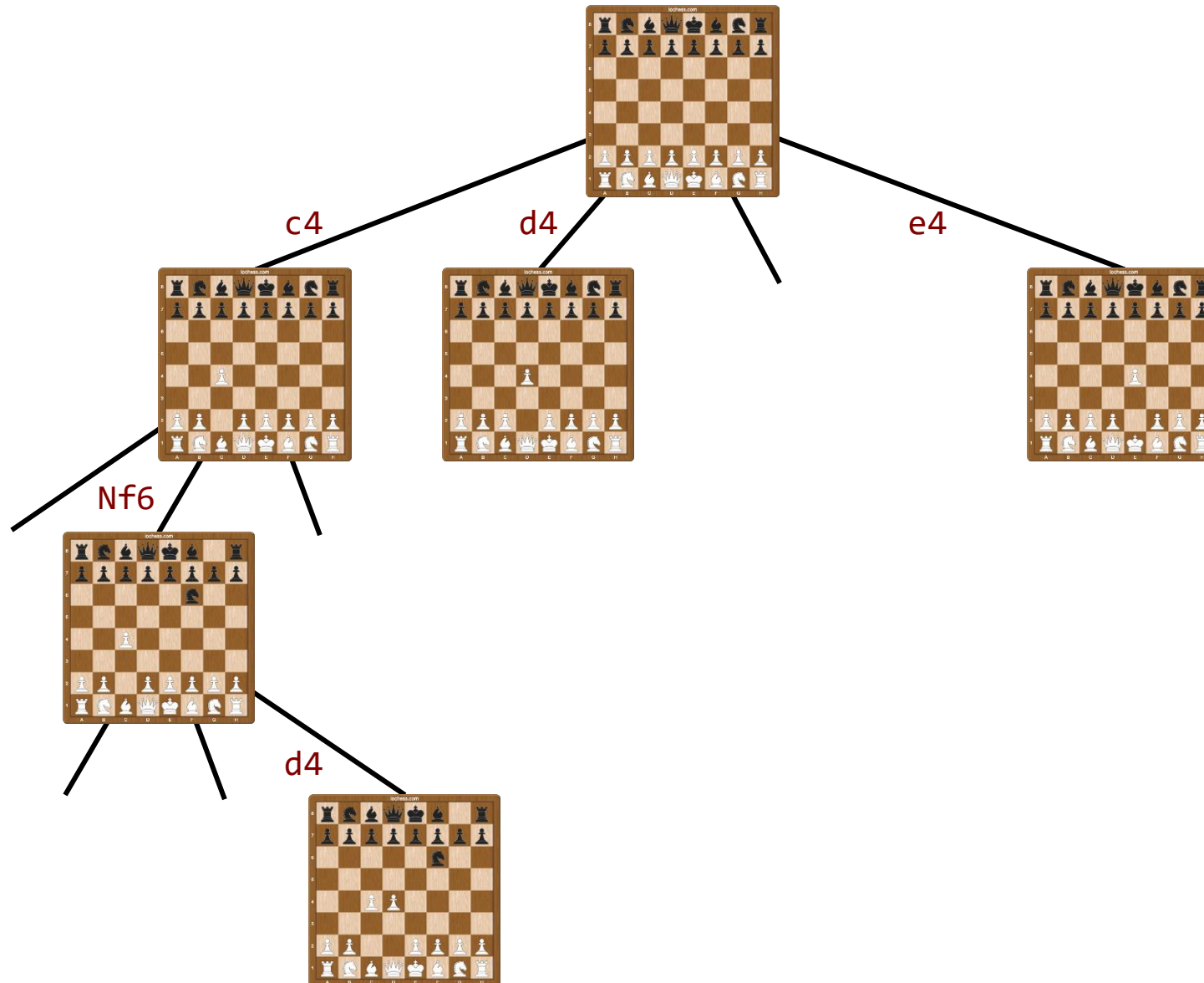
The Game Tree Is Really a Dag



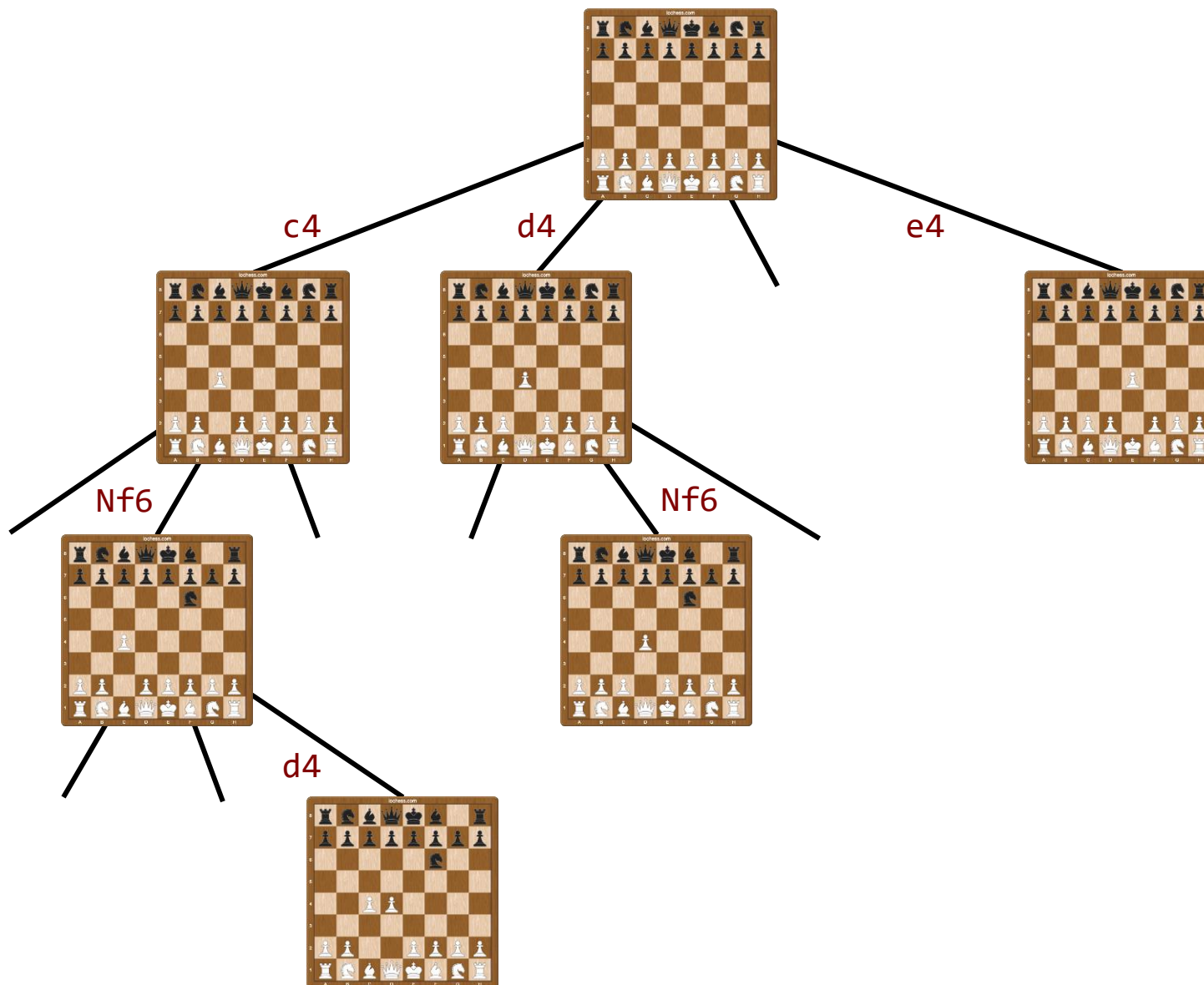
The Game Tree Is Really a Dag



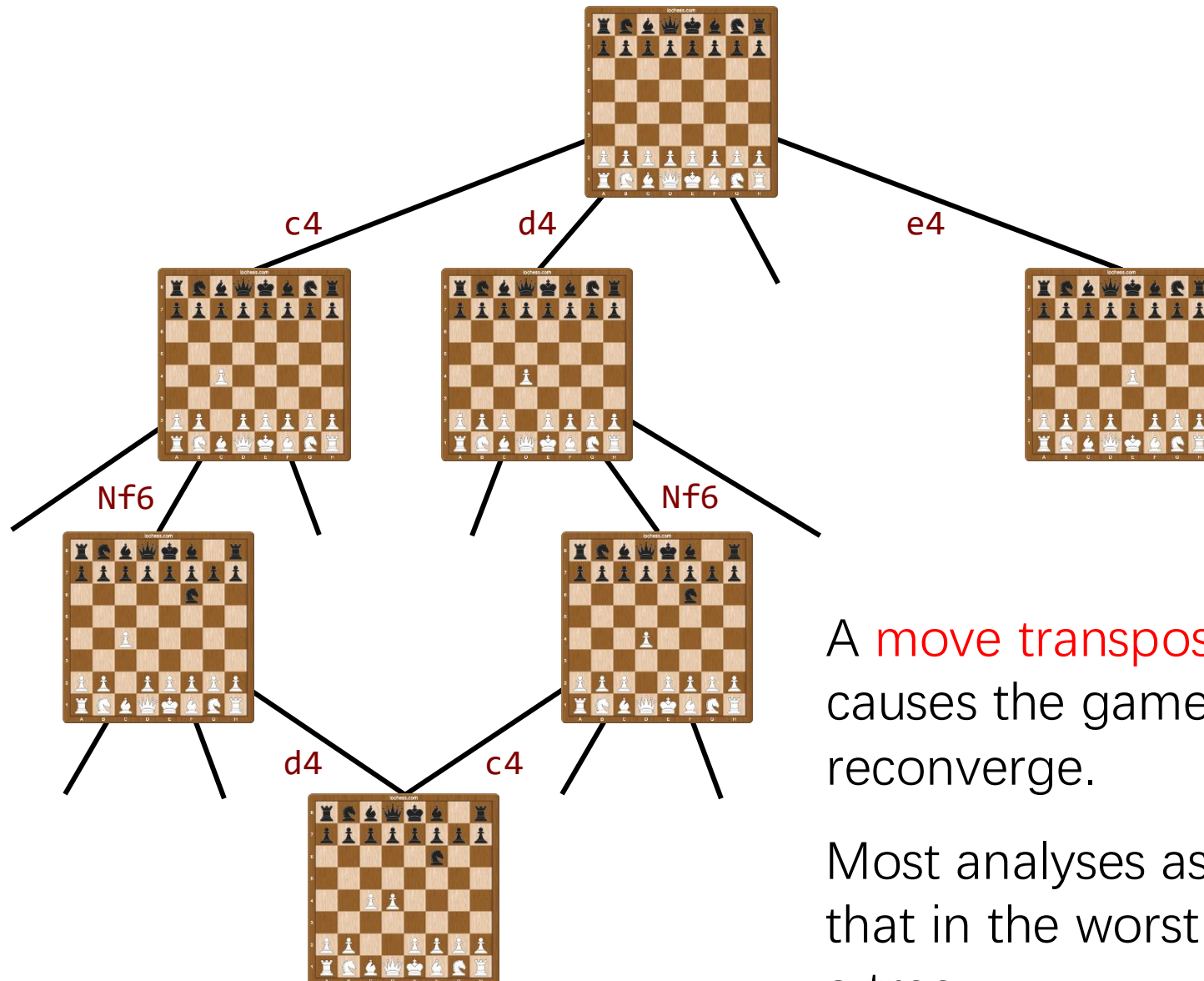
The Game Tree Is Really a Dag



The Game Tree Is Really a Dag



The Game Tree Is Really a Dag



A **move transposition** causes the game tree to reconverge.

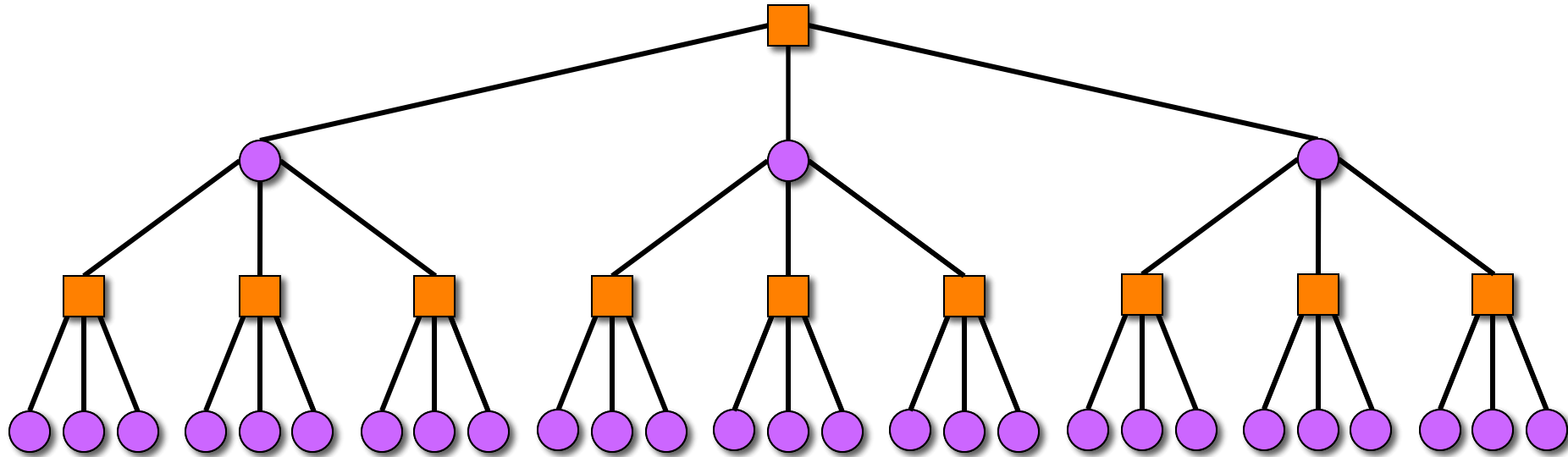
Most analyses assume that in the worst case, it's a tree.

ALPHA-BETA SEARCH

THE IDEA

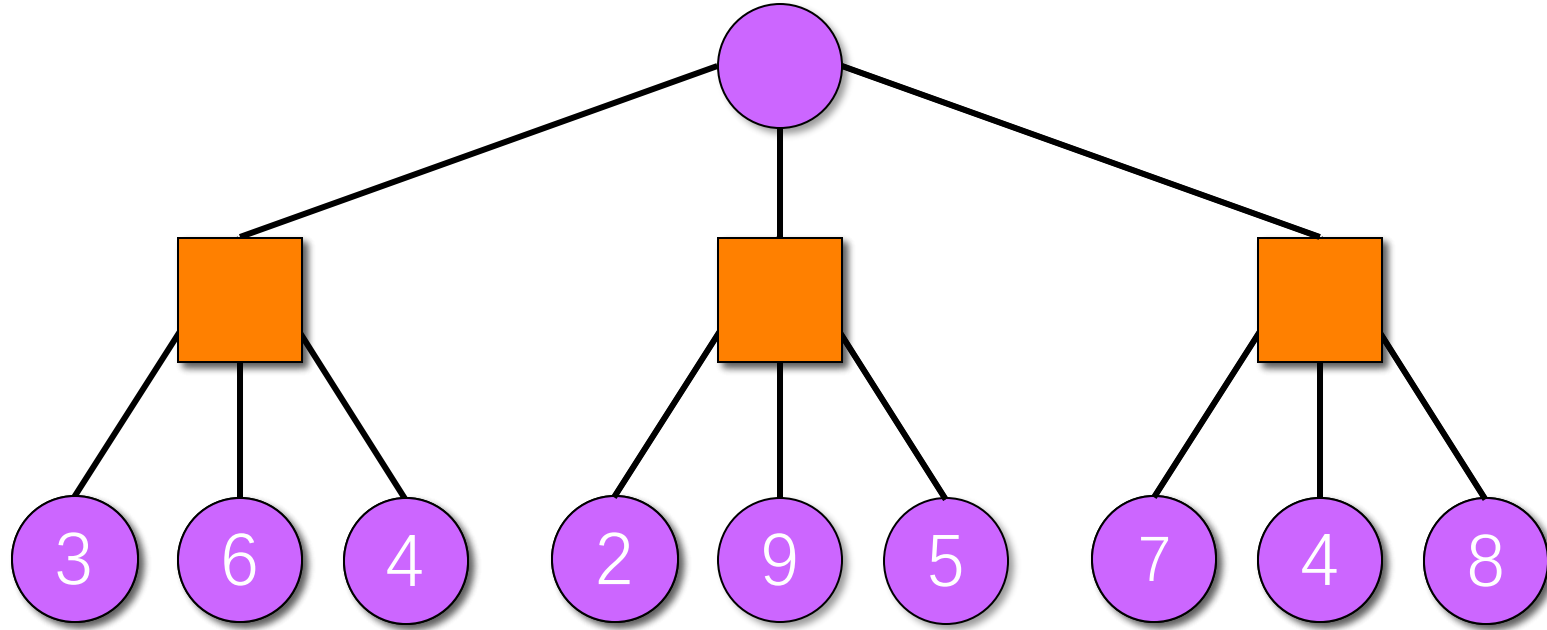


Min-Max Search



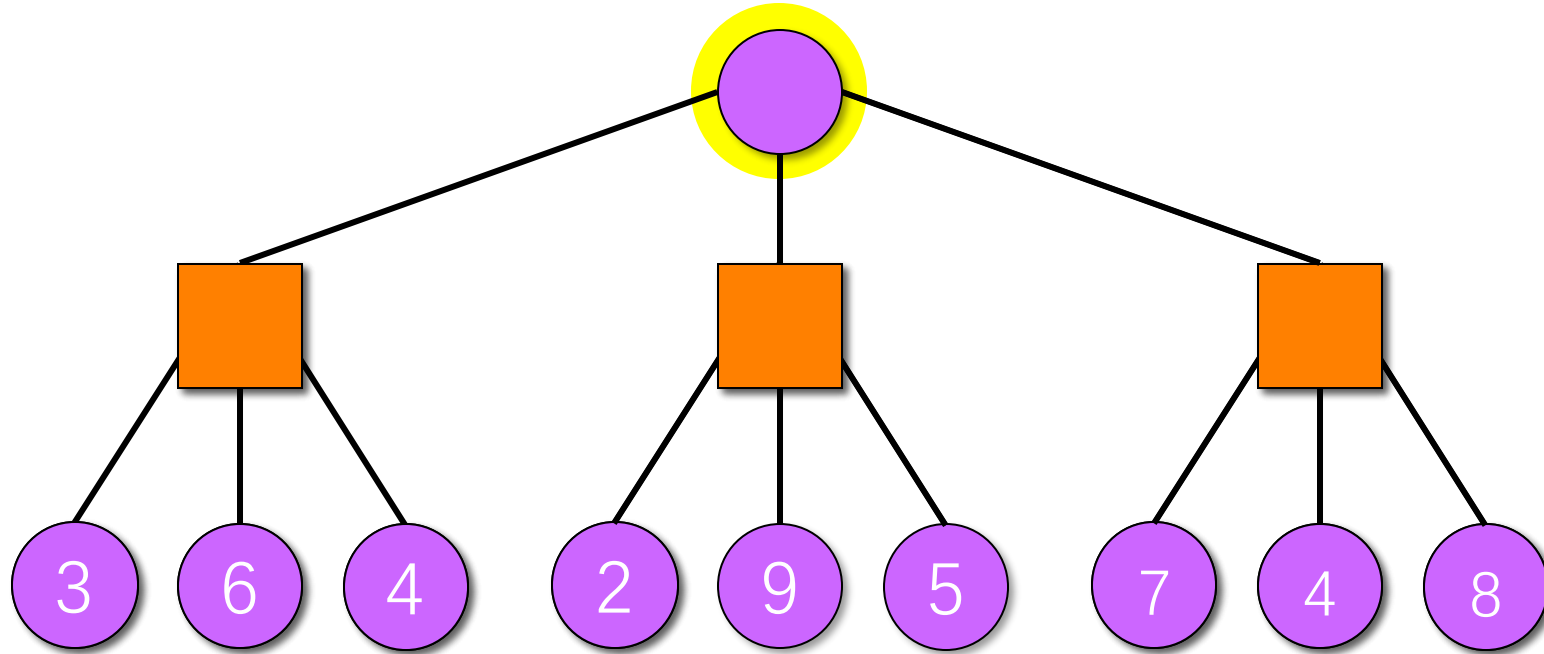
- Two players: MAX ■ and MIN ●.
- The **game tree** represents all moves from the current position within a given search **ply** (depth).
- At leaves, apply a **static evaluation function**.
- MAX chooses the maximum score among its children.
- MIN chooses the minimum score among its children.

Alpha-Beta Pruning



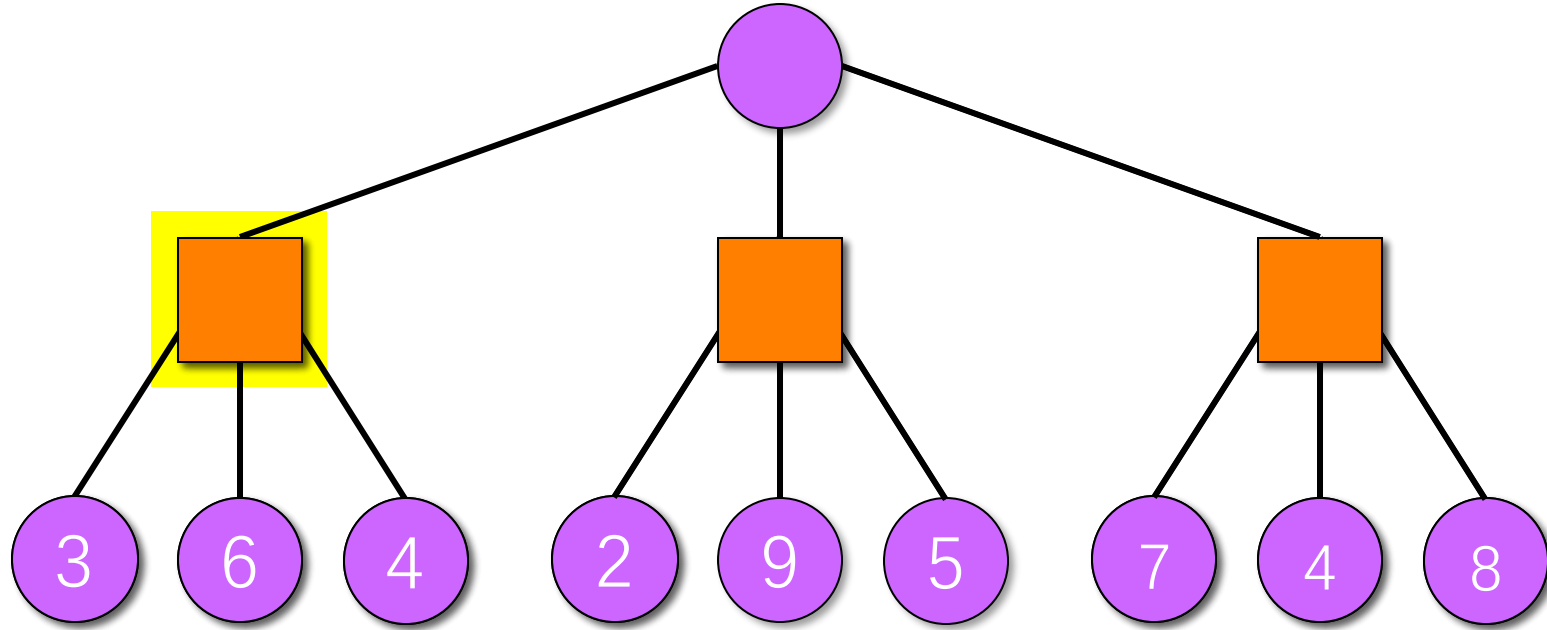
IDEA: If MAX discovers a move so good that MIN would never allow that position, MAX's other children need not be searched — beta cutoff.

Alpha-Beta Pruning



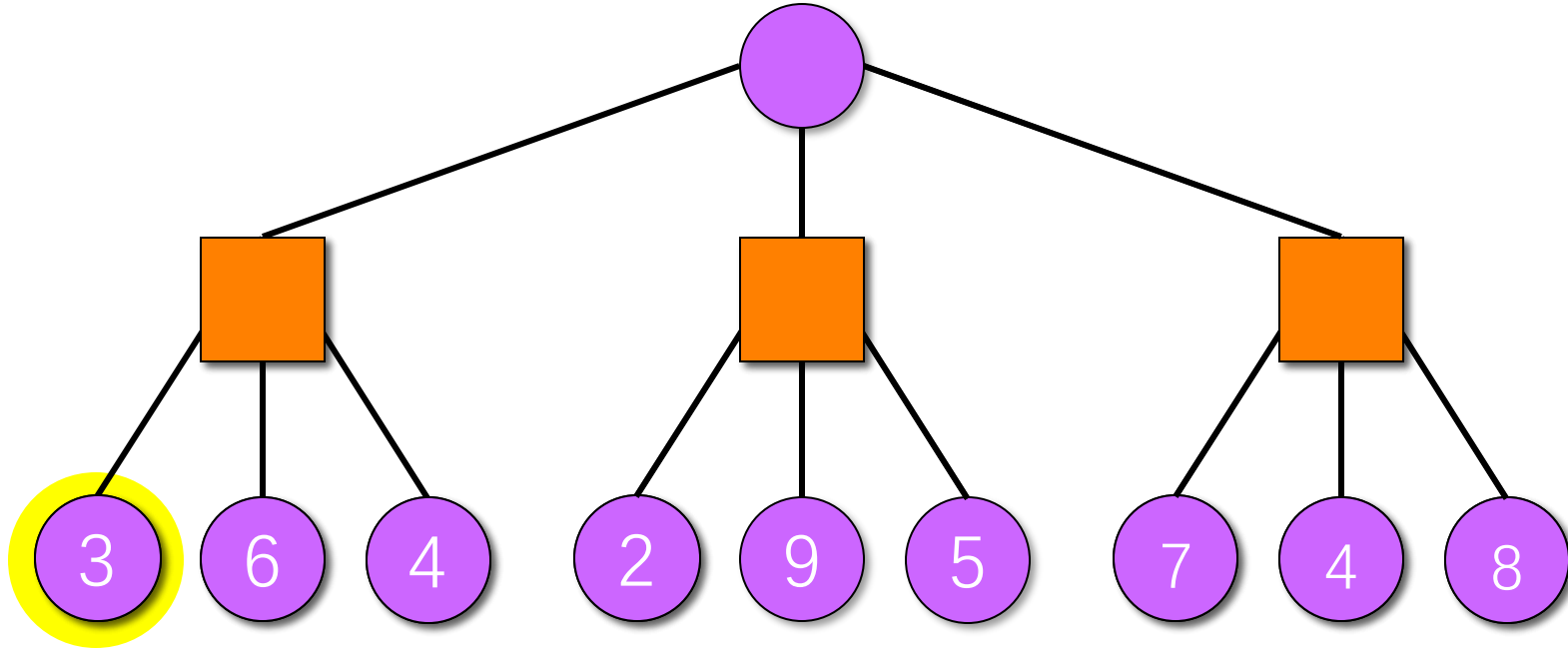
IDEA: If MAX discovers a move so good that MIN would never allow that position, MAX's other children need not be searched — beta cutoff.

Alpha-Beta Pruning



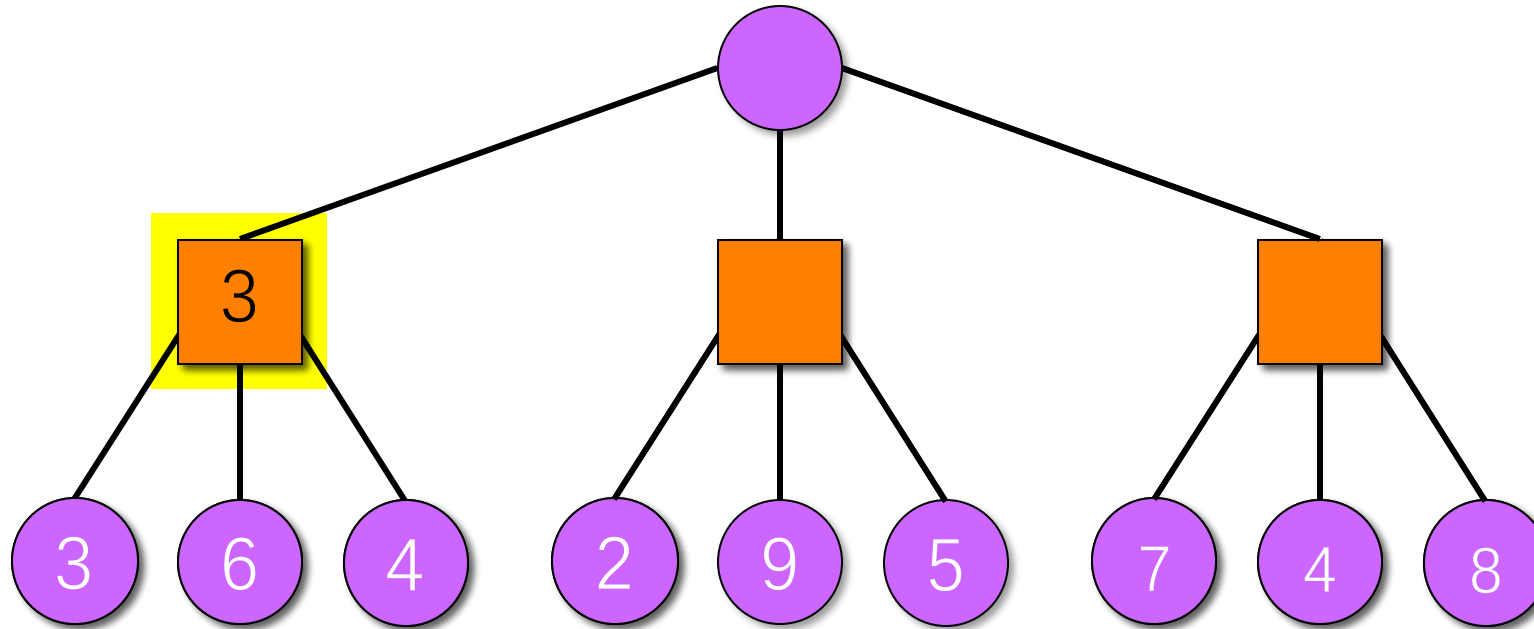
IDEA: If **MAX** discovers a move so good that **MIN** would never allow that position, **MAX**'s other children need not be searched — **beta cutoff**.

Alpha-Beta Pruning



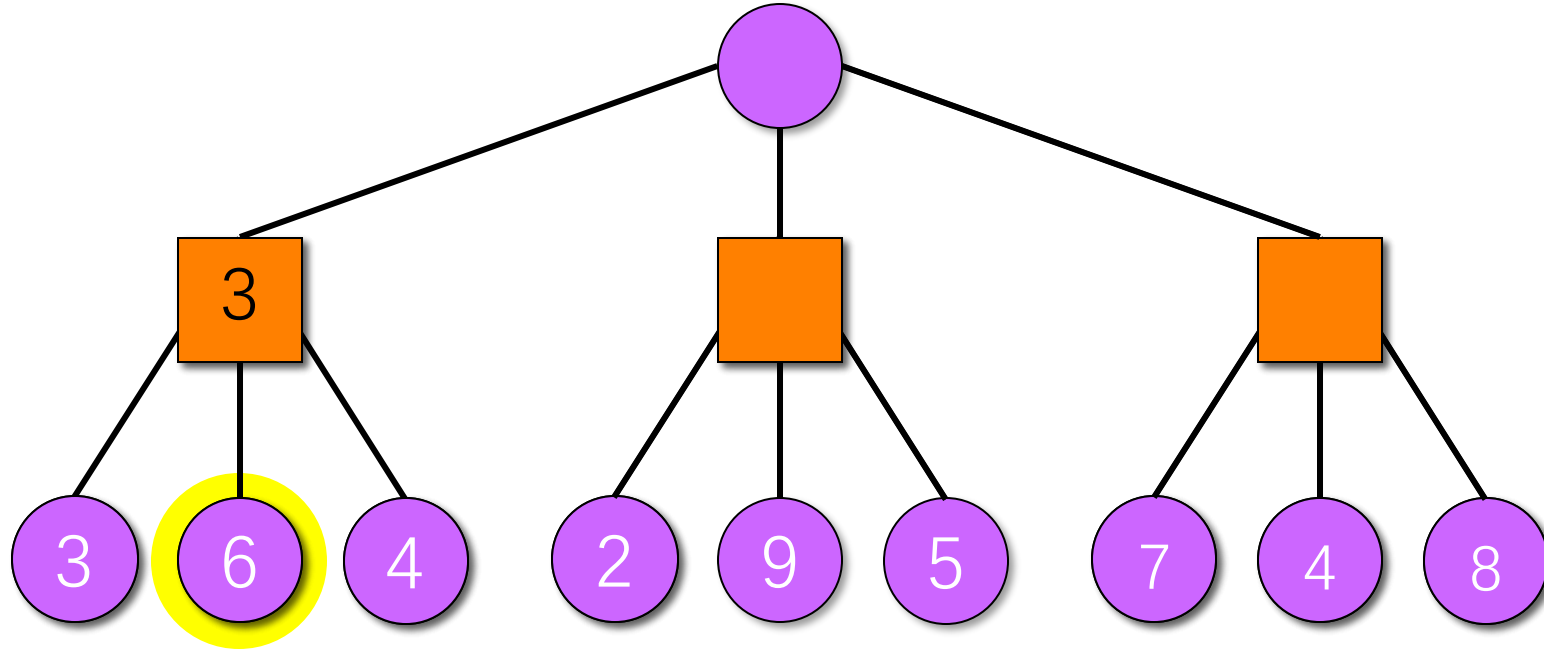
IDEA: If MAX discovers a move so good that MIN would never allow that position, MAX's other children need not be searched — beta cutoff.

Alpha-Beta Pruning



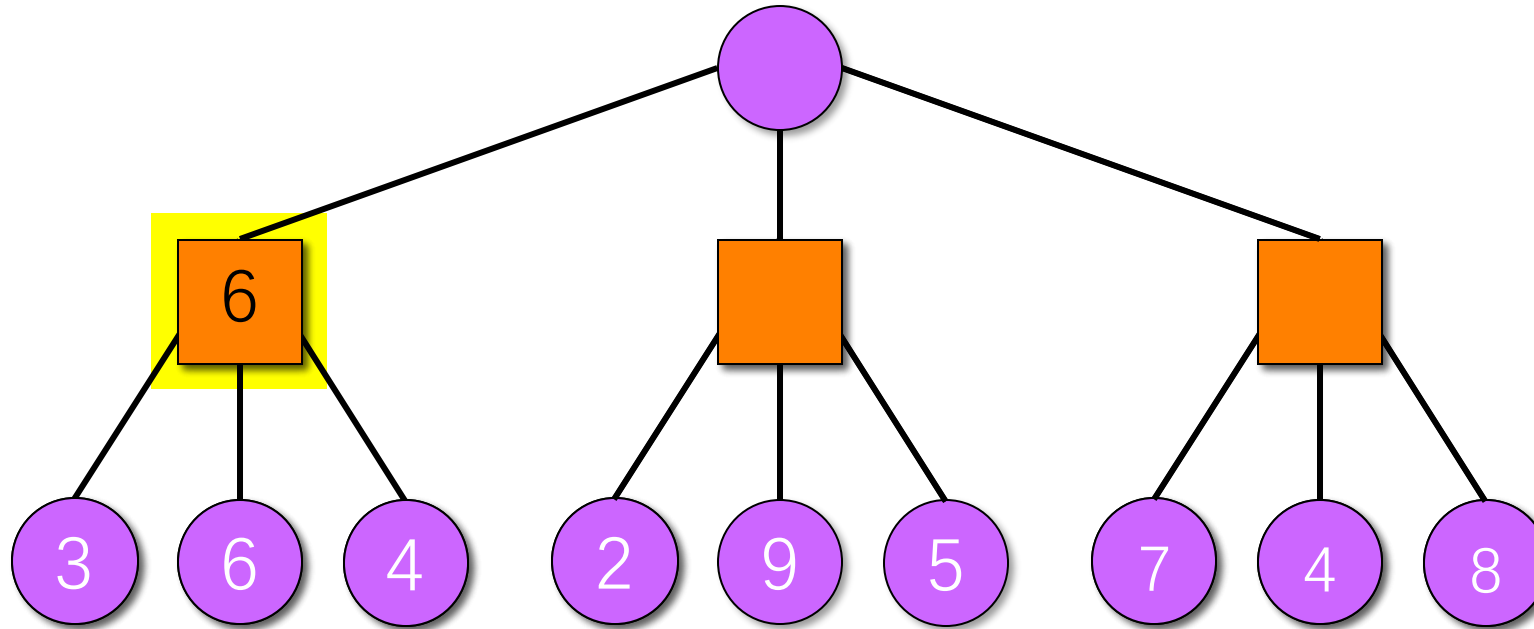
IDEA: If **MAX** discovers a move so good that **MIN** would never allow that position, **MAX**'s other children need not be searched — **beta cutoff**.

Alpha-Beta Pruning



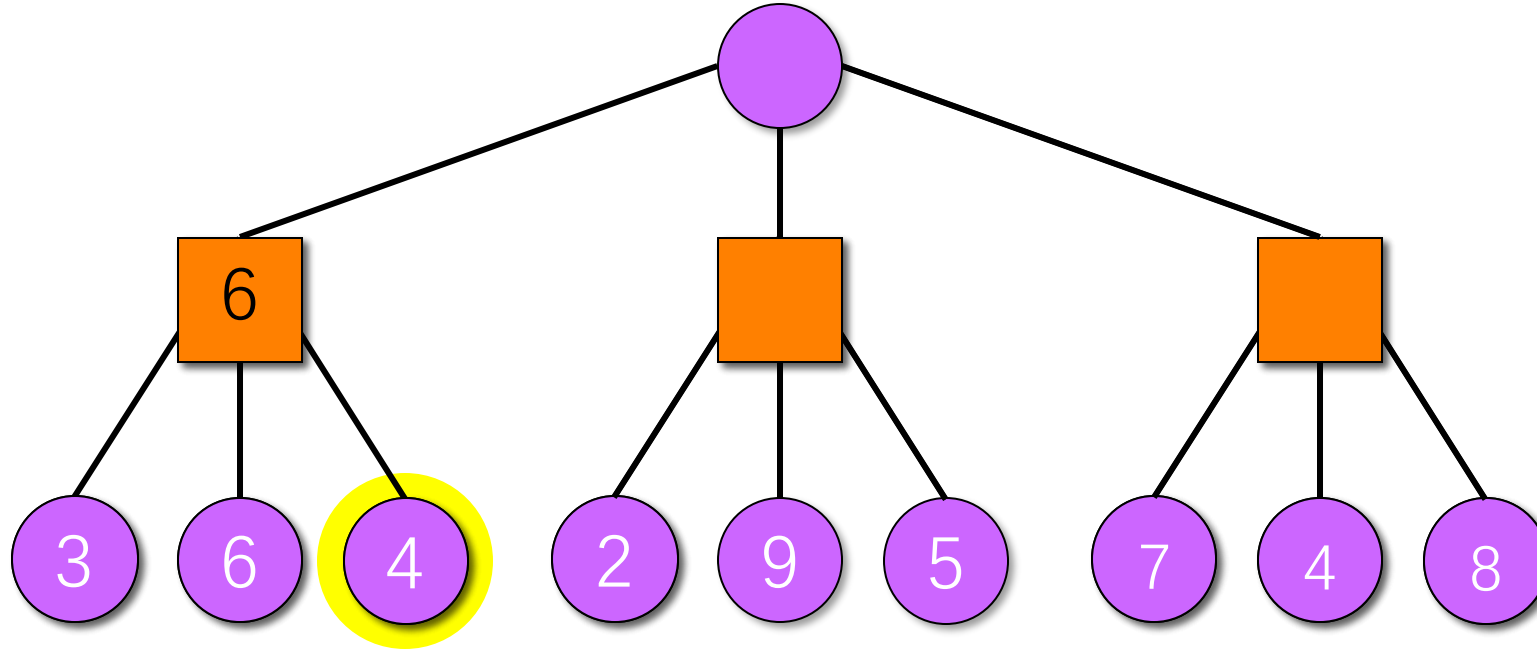
IDEA: If MAX discovers a move so good that MIN would never allow that position, MAX's other children need not be searched — beta cutoff.

Alpha-Beta Pruning



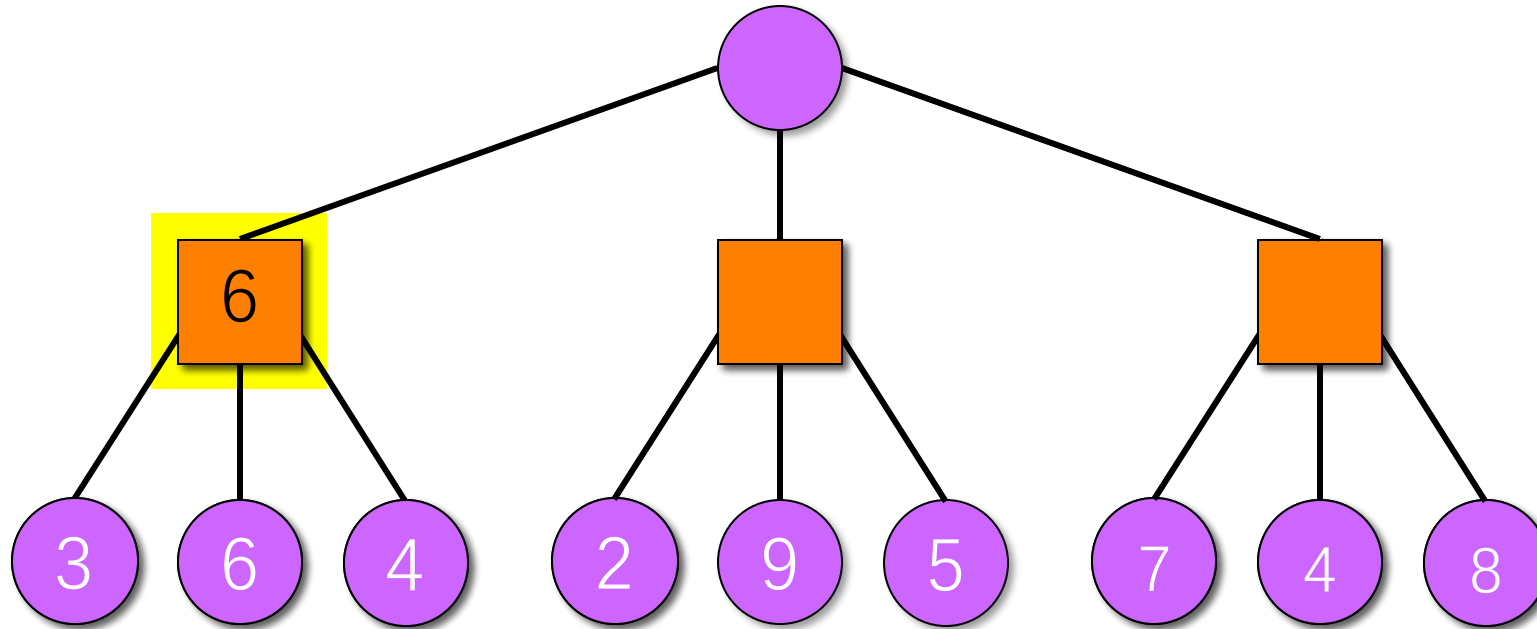
IDEA: If **MAX** discovers a move so good that **MIN** would never allow that position, **MAX**'s other children need not be searched — **beta cutoff**.

Alpha-Beta Pruning



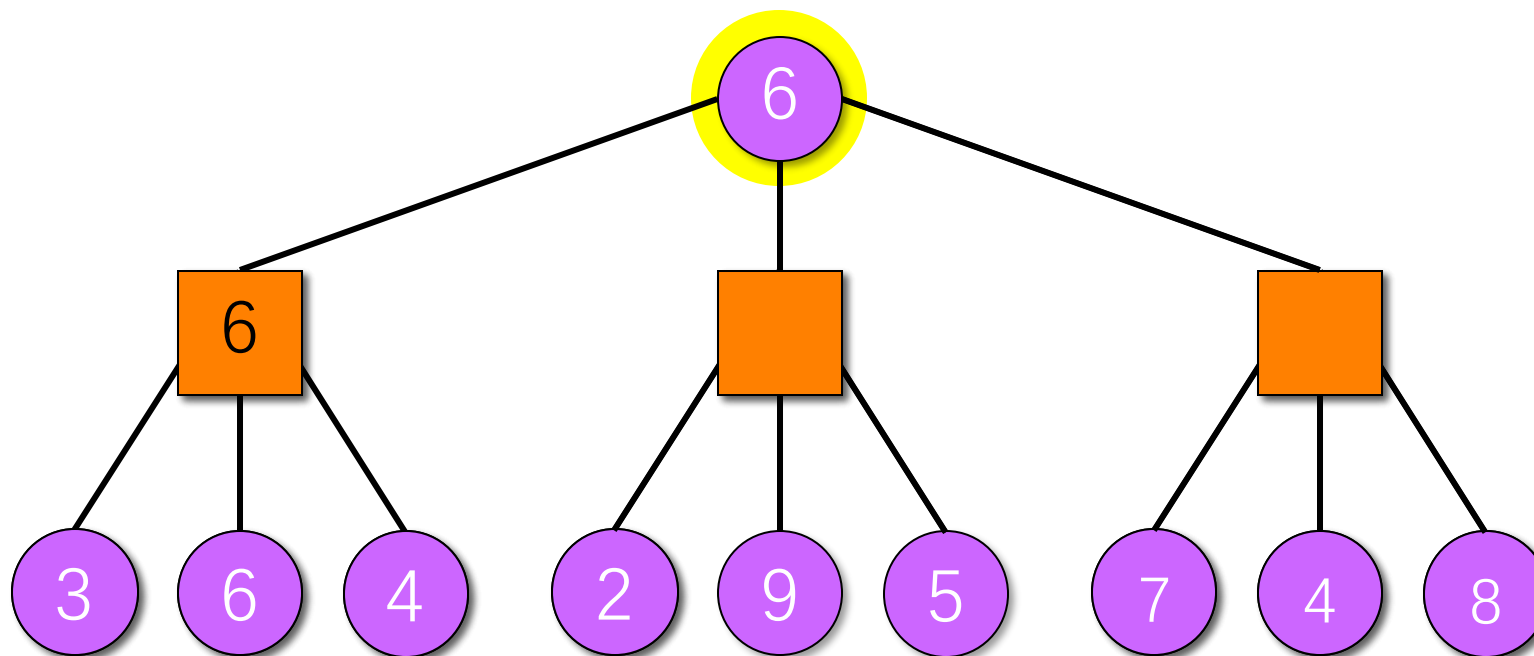
IDEA: If MAX discovers a move so good that MIN would never allow that position, MAX's other children need not be searched — beta cutoff.

Alpha-Beta Pruning



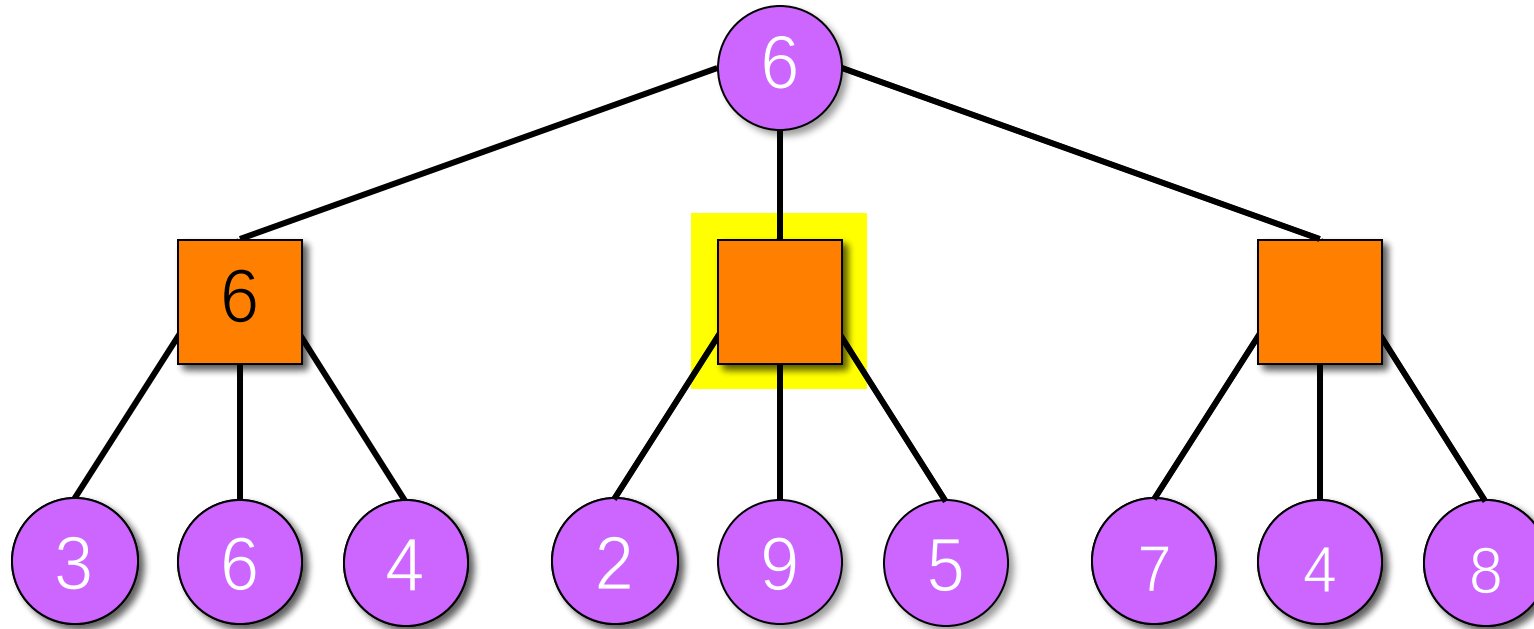
IDEA: If MAX discovers a move so good that MIN would never allow that position, MAX's other children need not be searched — beta cutoff.

Alpha-Beta Pruning



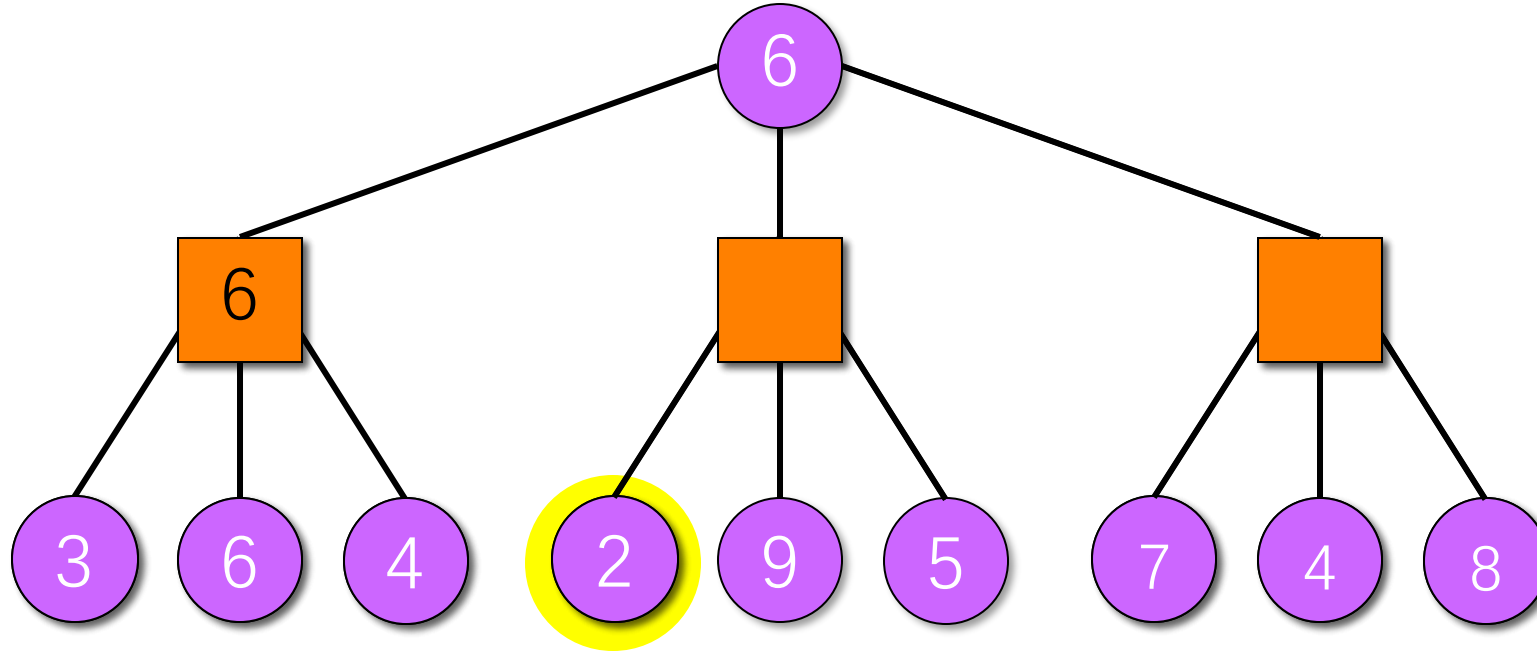
IDEA: If MAX discovers a move so good that MIN would never allow that position, MAX's other children need not be searched — beta cutoff.

Alpha-Beta Pruning



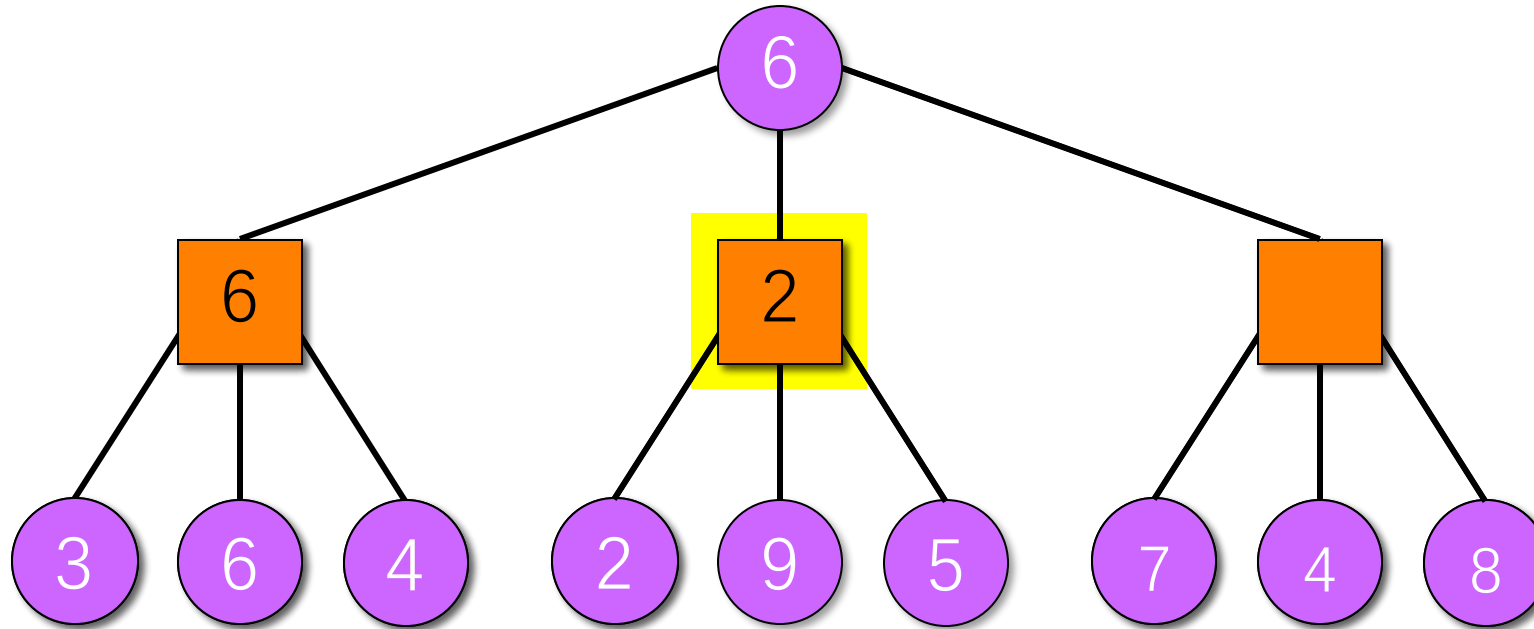
IDEA: If MAX discovers a move so good that MIN would never allow that position, MAX's other children need not be searched — beta cutoff.

Alpha-Beta Pruning



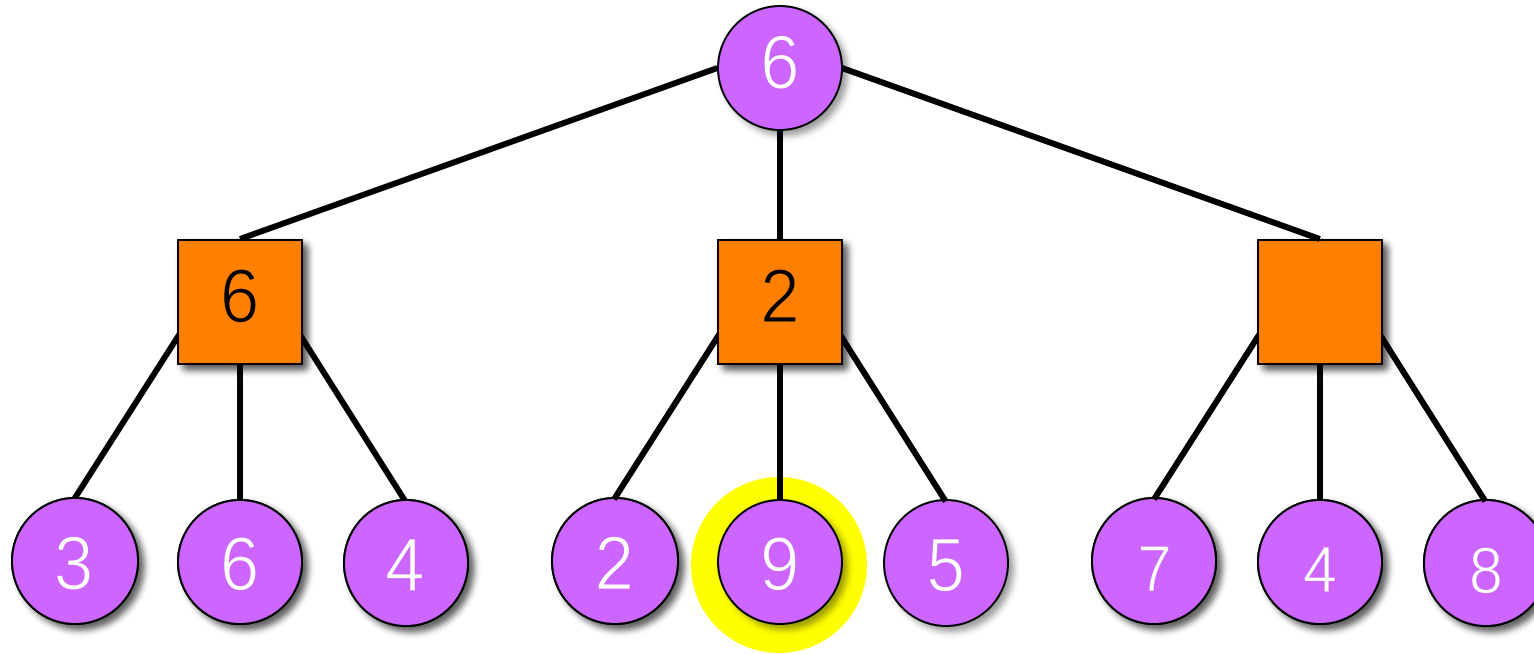
IDEA: If MAX discovers a move so good that MIN would never allow that position, MAX's other children need not be searched — beta cutoff.

Alpha-Beta Pruning



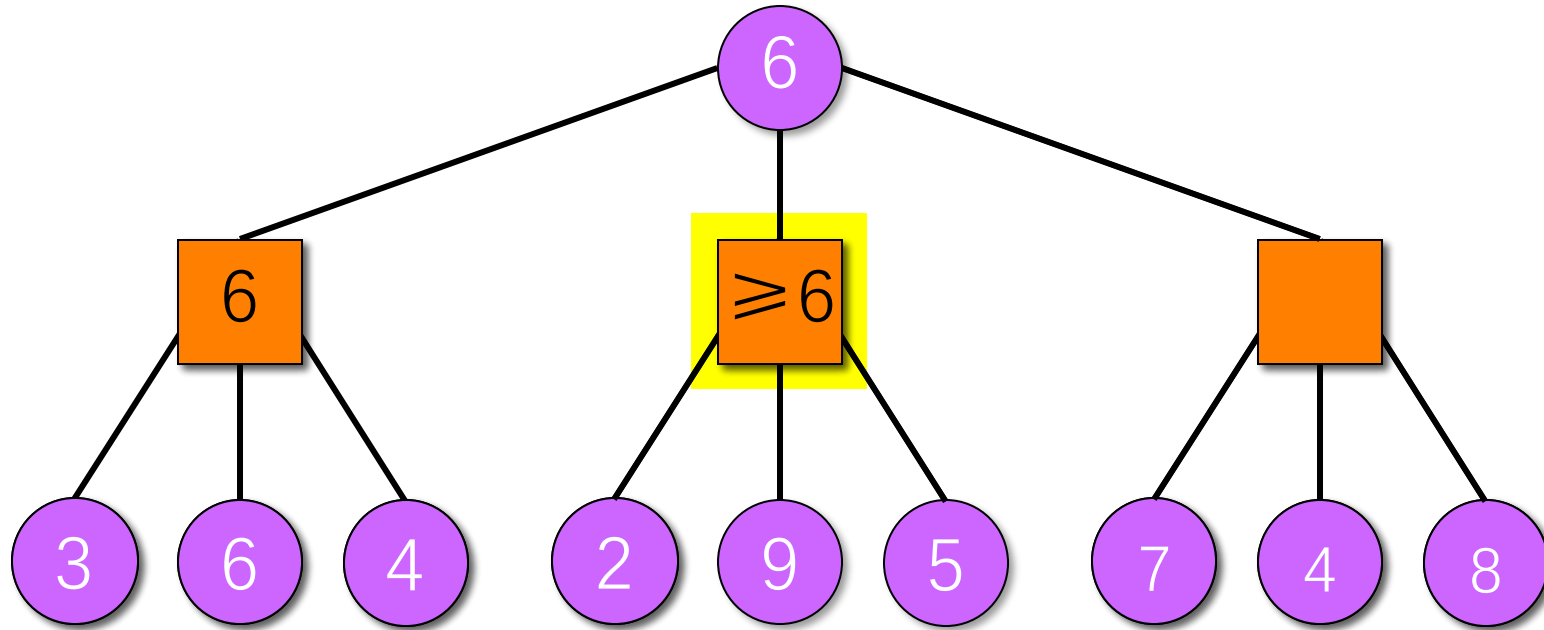
IDEA: If MAX discovers a move so good that MIN would never allow that position, MAX's other children need not be searched — beta cutoff.

Alpha-Beta Pruning



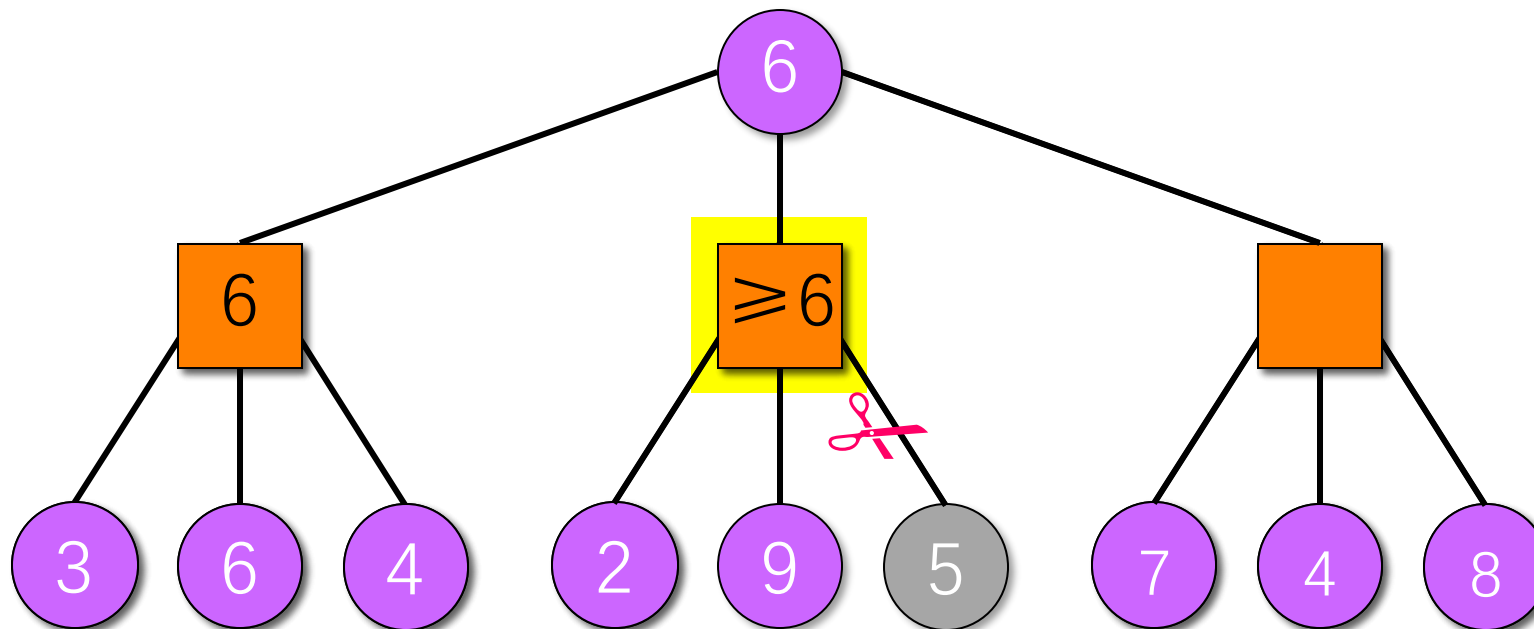
IDEA: If MAX discovers a move so good that MIN would never allow that position, MAX's other children need not be searched — beta cutoff.

Alpha-Beta Pruning



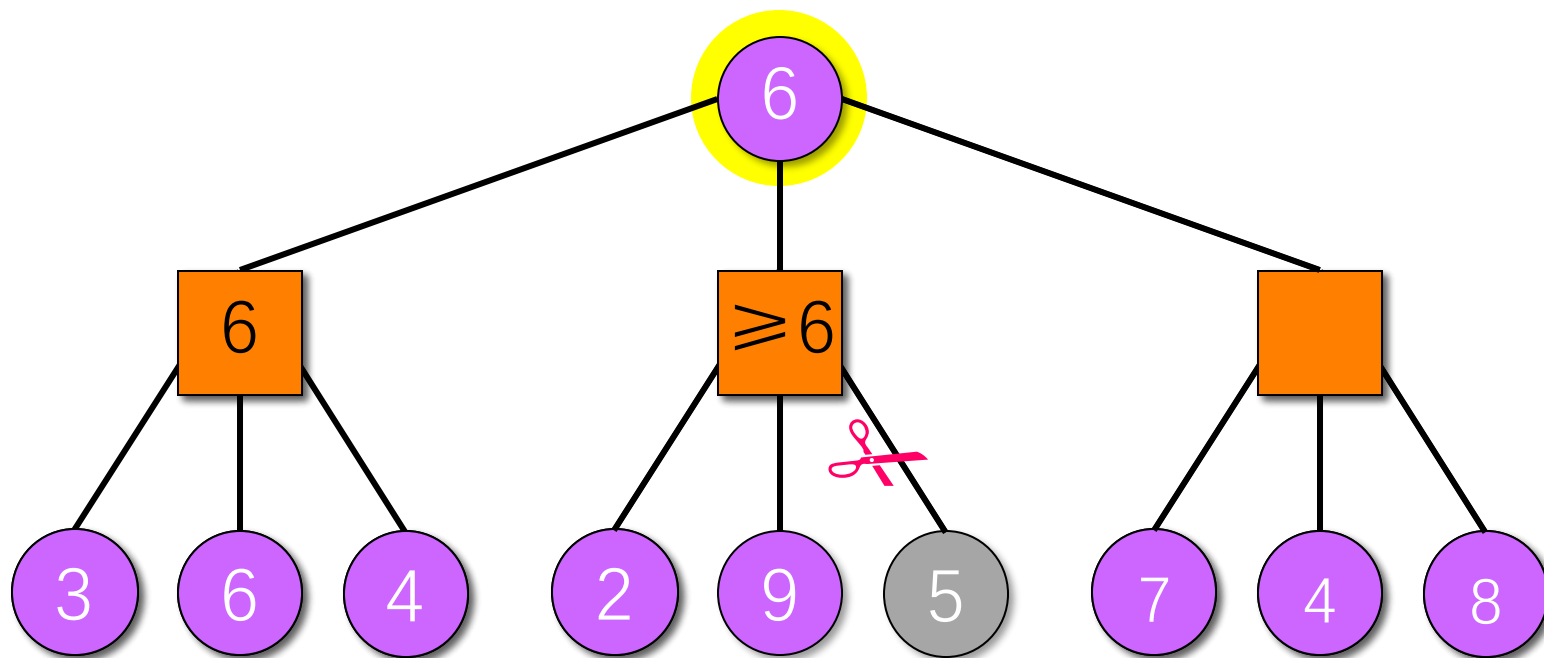
IDEA: If MAX discovers a move so good that MIN would never allow that position, MAX's other children need not be searched — beta cutoff.

Alpha-Beta Pruning



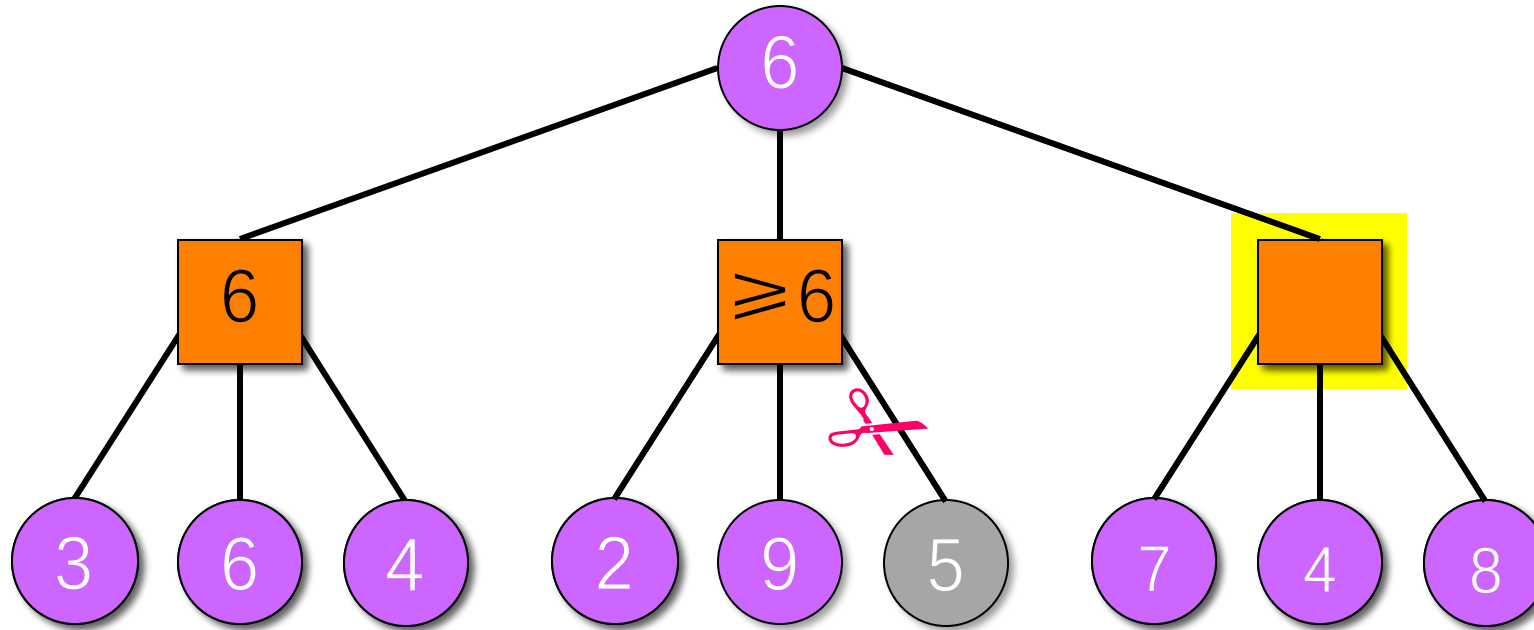
IDEA: If MAX discovers a move so good that MIN would never allow that position, MAX's other children need not be searched — beta cutoff.

Alpha-Beta Pruning



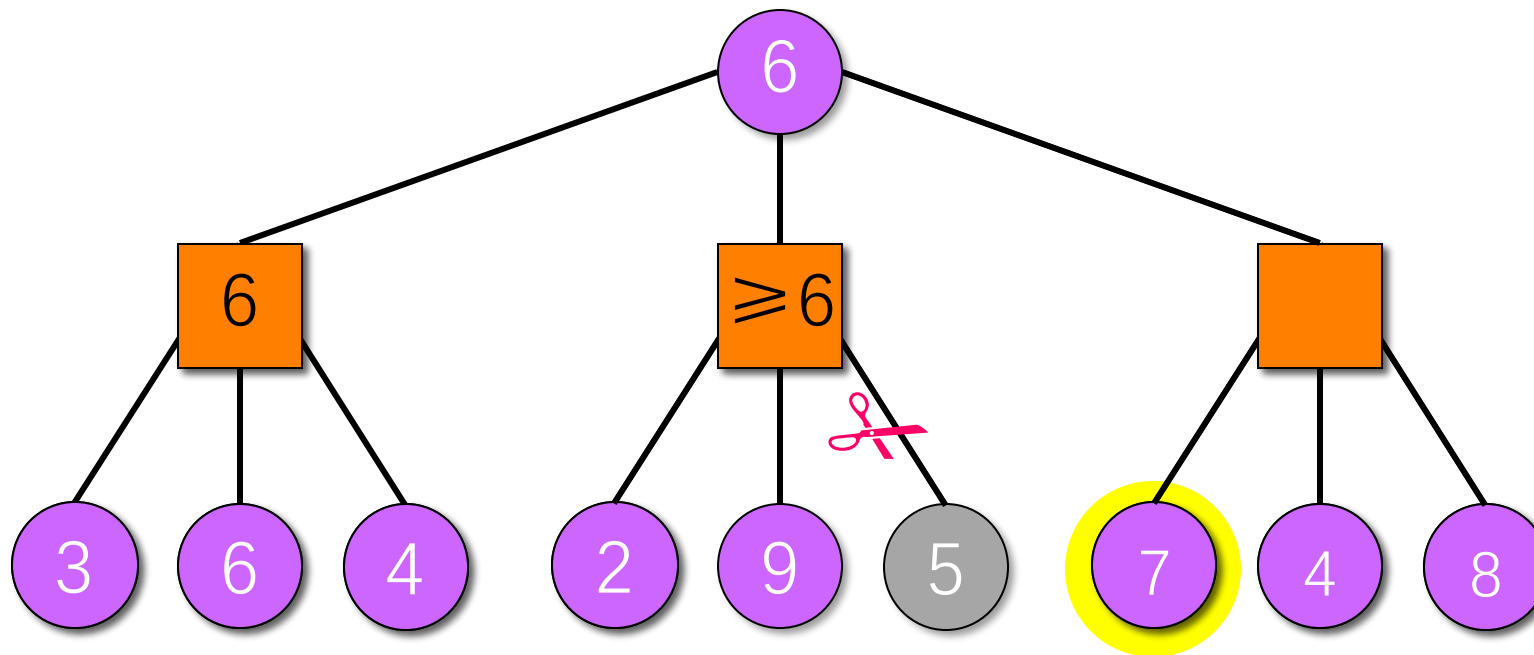
IDEA: If MAX discovers a move so good that MIN would never allow that position, MAX's other children need not be searched — beta cutoff.

Alpha-Beta Pruning



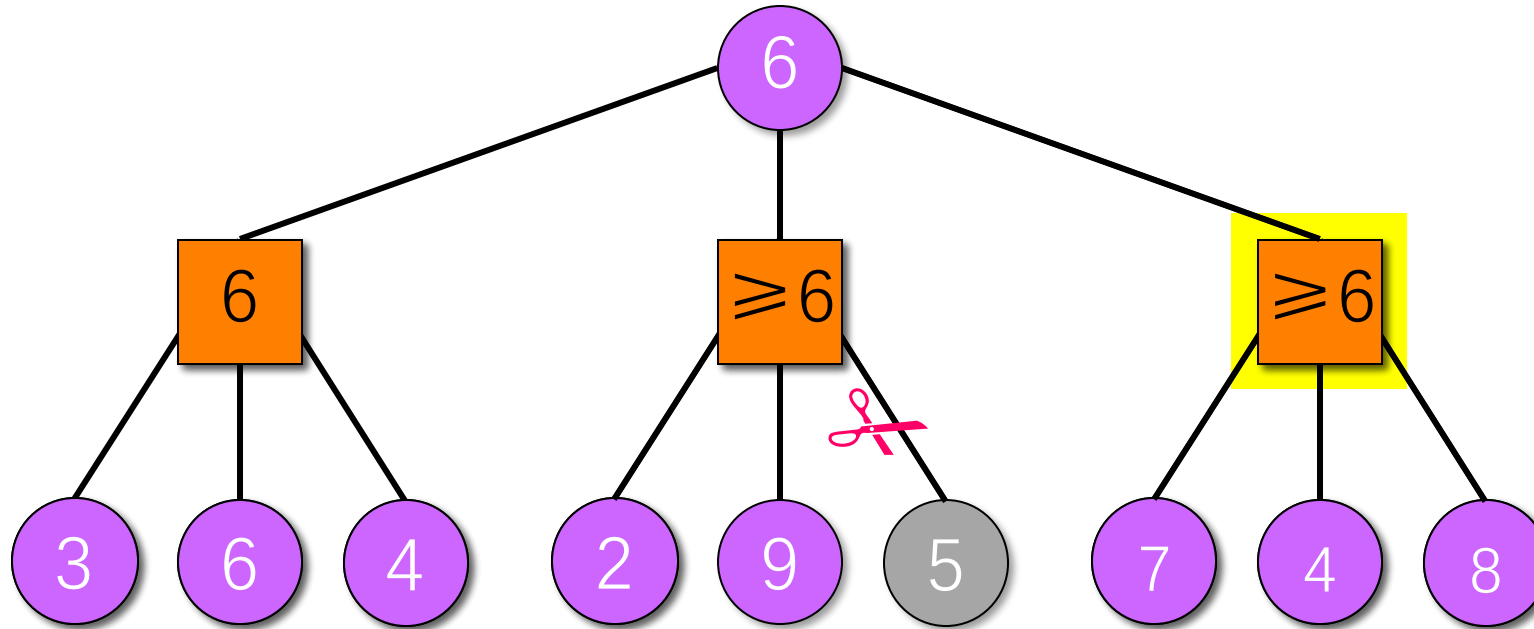
IDEA: If MAX discovers a move so good that MIN would never allow that position, MAX's other children need not be searched — **beta cutoff**.

Alpha-Beta Pruning



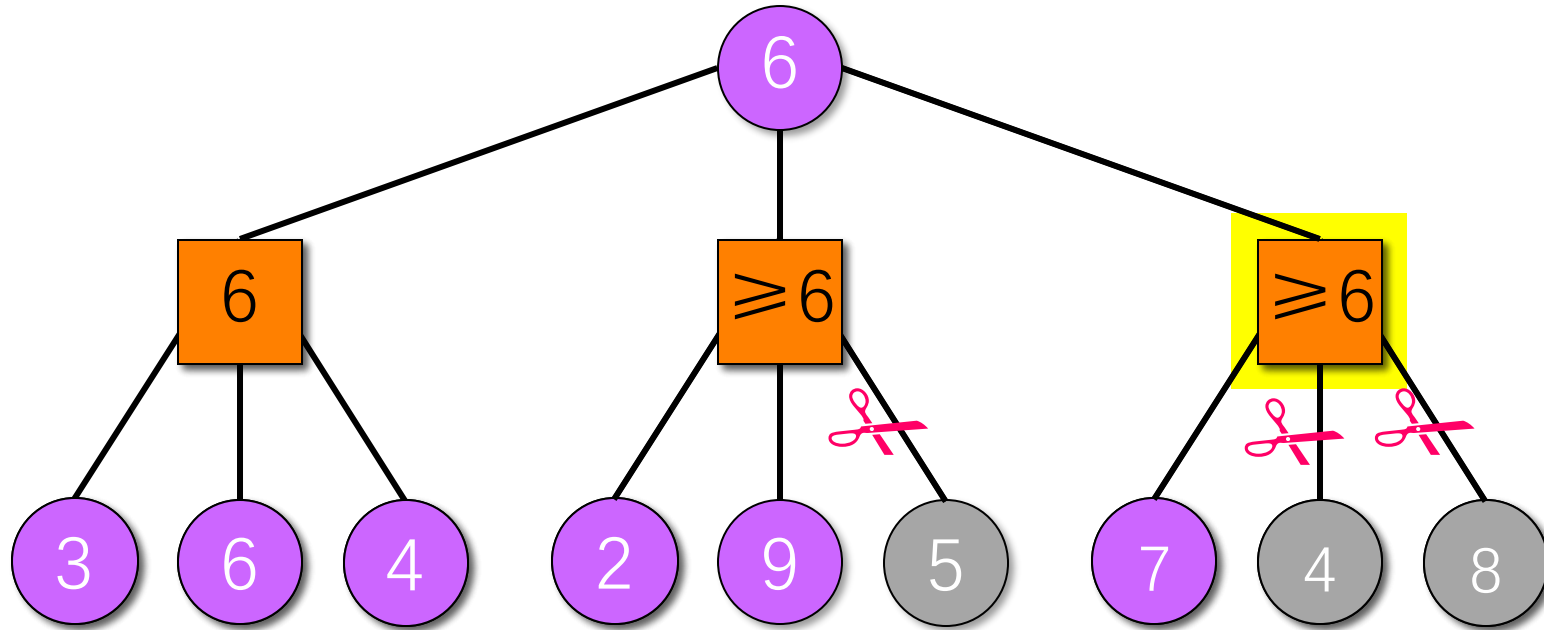
IDEA: If MAX discovers a move so good that MIN would never allow that position, MAX's other children need not be searched — beta cutoff.

Alpha-Beta Pruning



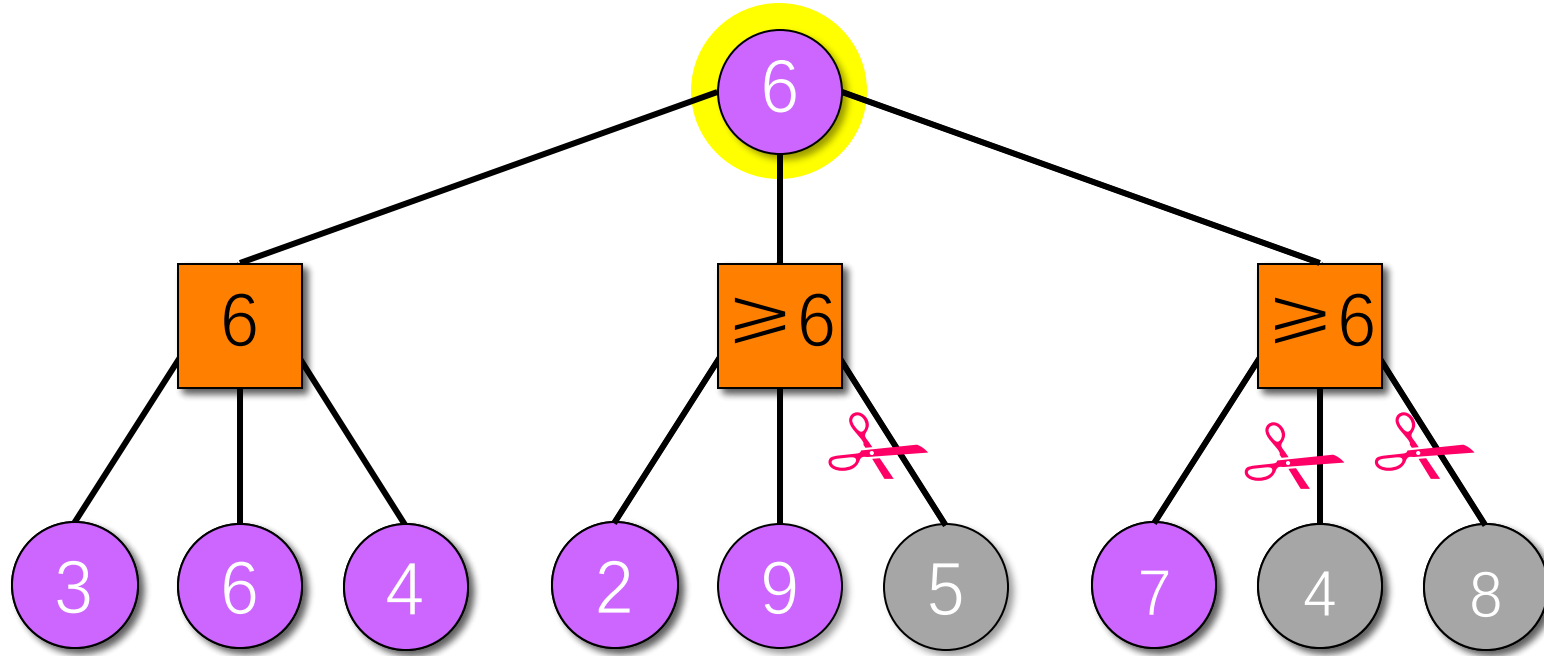
IDEA: If MAX discovers a move so good that MIN would never allow that position, MAX's other children need not be searched — beta cutoff.

Alpha-Beta Pruning



IDEA: If MAX discovers a move so good that MIN would never allow that position, MAX's other children need not be searched — beta cutoff.

Alpha-Beta Pruning



IDEA: If MAX discovers a move so good that MIN would never allow that position, MAX's other children need not be searched — beta cutoff.

ALPHA-BETA SEARCH

THE CODE

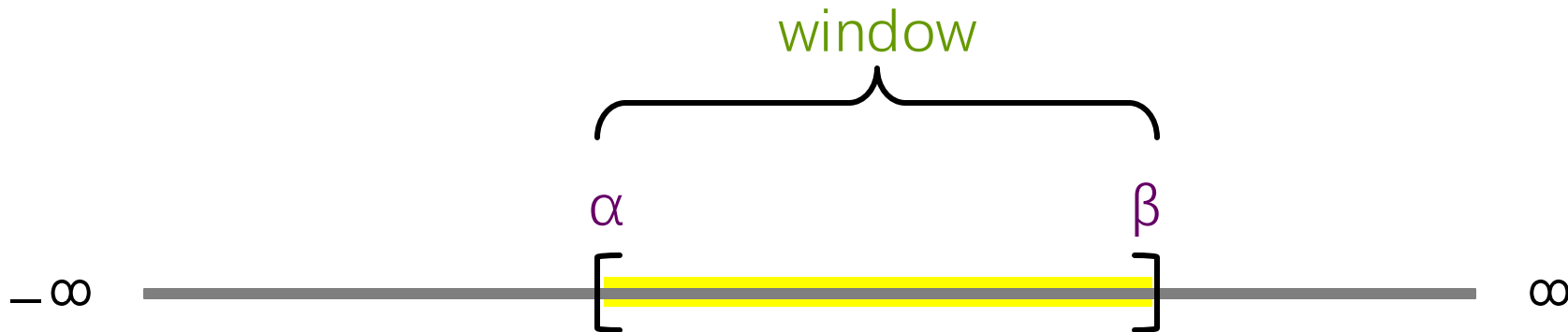


Alpha-Beta

```
int search(pos x, int alpha, int beta, int d) {
    if (d == 0 || is_leaf(x)) return static_eval(x);
    pos c[MAX_CHILDREN];
    int nc;
    gen_moves(x, c, &nc);                // generate children
    for (int i = 0; i < nc; ++i) {
        int s = -search(c[i], -beta, -alpha, d-1); // negamax
        if (s > alpha) alpha = s;
        if (alpha >= beta) return alpha;          // beta cutoff
    }
    return alpha;
}
```

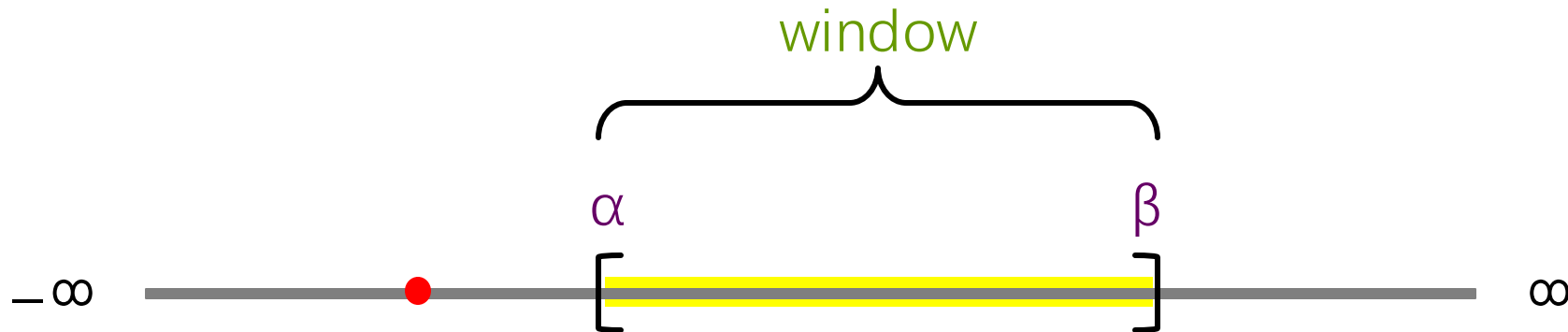
Alpha-Beta

```
int search(pos x, int alpha, int beta, int d) {  
    if (d == 0 || is_leaf(x)) return static_eval(x);  
    pos c[MAX_CHILDREN];  
    int nc;  
    gen_moves(x, c, &nc);           // generate children  
    for (int i = 0; i < nc; ++i) {  
        int s = -search(c[i], -beta, -alpha, d-1); // negamax  
        if (s > alpha) alpha = s;  
        if (alpha >= beta) return alpha;           // beta cutoff  
    }  
    return alpha;  
}
```



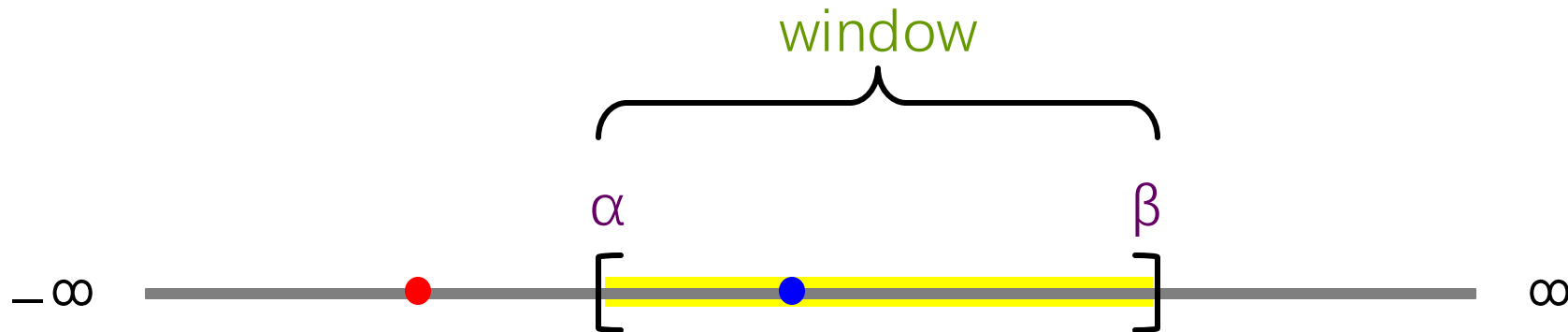
Alpha-Beta

```
int search(pos x, int alpha, int beta, int d) {  
    if (d == 0 || is_leaf(x)) return static_eval(x);  
    pos c[MAX_CHILDREN];  
    int nc;  
    gen_moves(x, c, &nc); // generate children  
    for (int i = 0; i < nc; ++i) {  
        int s = -search(c[i], -beta, -alpha, d-1); // negamax  
        if (s > alpha) alpha = s;  
        if (alpha >= beta) return alpha; // beta cutoff  
    }  
    return alpha;  
}
```



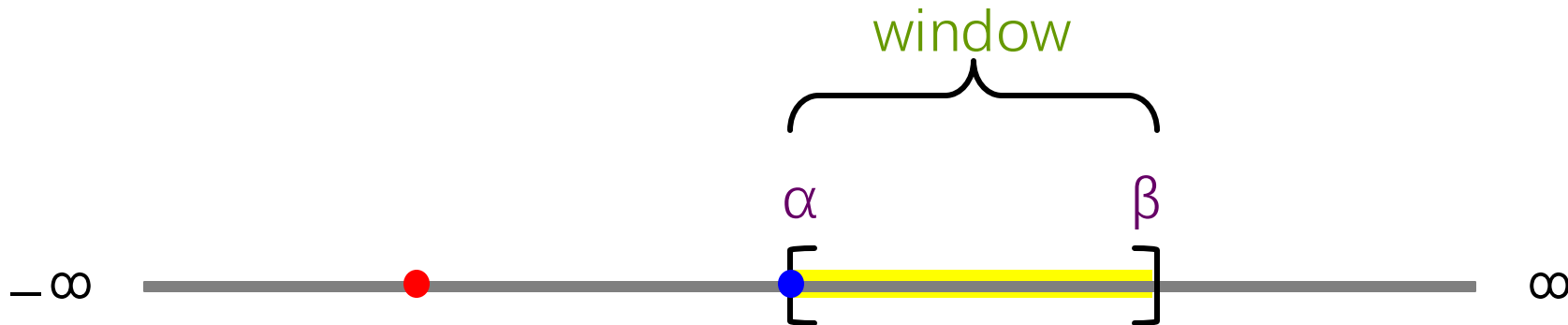
Alpha-Beta

```
int search(pos x, int alpha, int beta, int d) {  
    if (d == 0 || is_leaf(x)) return static_eval(x);  
    pos c[MAX_CHILDREN];  
    int nc;  
    gen_moves(x, c, &nc); // generate children  
    for (int i = 0; i < nc; ++i) {  
        int s = -search(c[i], -beta, -alpha, d-1); // negamax  
        if (s > alpha) alpha = s;  
        if (alpha >= beta) return alpha; // beta cutoff  
    }  
    return alpha;  
}
```



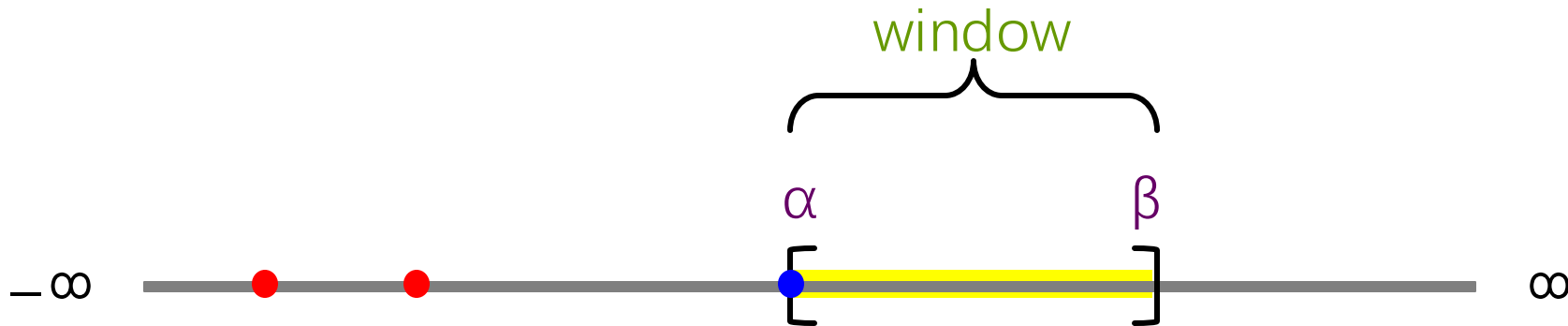
Alpha-Beta

```
int search(pos x, int alpha, int beta, int d) {  
    if (d == 0 || is_leaf(x)) return static_eval(x);  
    pos c[MAX_CHILDREN];  
    int nc;  
    gen_moves(x, c, &nc); // generate children  
    for (int i = 0; i < nc; ++i) {  
        int s = -search(c[i], -beta, -alpha, d-1); // negamax  
        if (s > alpha) alpha = s;  
        if (alpha >= beta) return alpha; // beta cutoff  
    }  
    return alpha;  
}
```



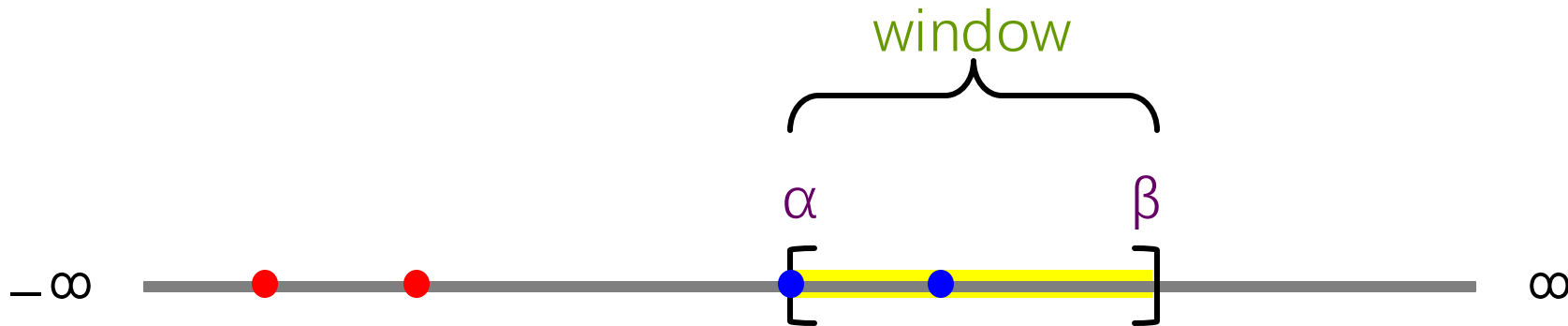
Alpha-Beta

```
int search(pos x, int alpha, int beta, int d) {  
    if (d == 0 || is_leaf(x)) return static_eval(x);  
    pos c[MAX_CHILDREN];  
    int nc;  
    gen_moves(x, c, &nc); // generate children  
    for (int i = 0; i < nc; ++i) {  
        int s = -search(c[i], -beta, -alpha, d-1); // negamax  
        if (s > alpha) alpha = s;  
        if (alpha >= beta) return alpha; // beta cutoff  
    }  
    return alpha;  
}
```



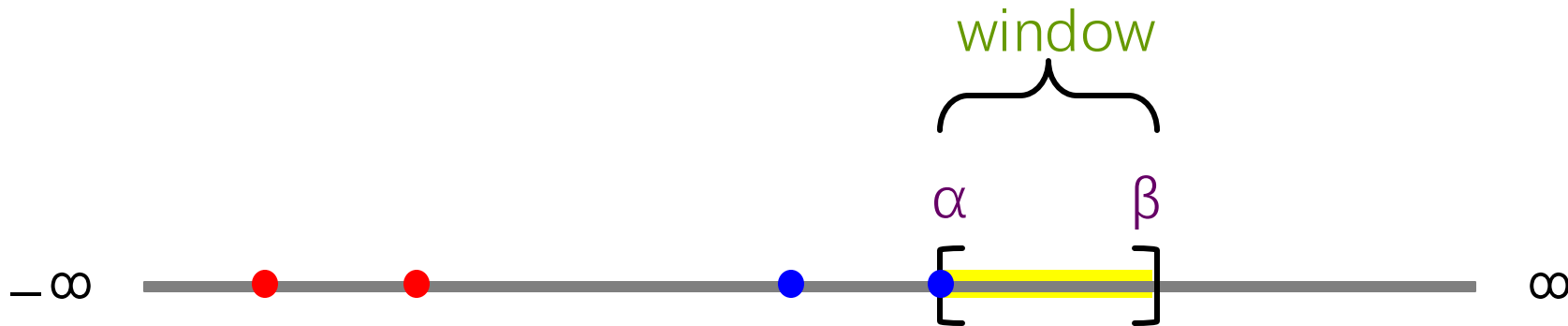
Alpha-Beta

```
int search(pos x, int alpha, int beta, int d) {  
    if (d == 0 || is_leaf(x)) return static_eval(x);  
    pos c[MAX_CHILDREN];  
    int nc;  
    gen_moves(x, c, &nc); // generate children  
    for (int i = 0; i < nc; ++i) {  
        int s = -search(c[i], -beta, -alpha, d-1); // negamax  
        if (s > alpha) alpha = s;  
        if (alpha >= beta) return alpha; // beta cutoff  
    }  
    return alpha;  
}
```



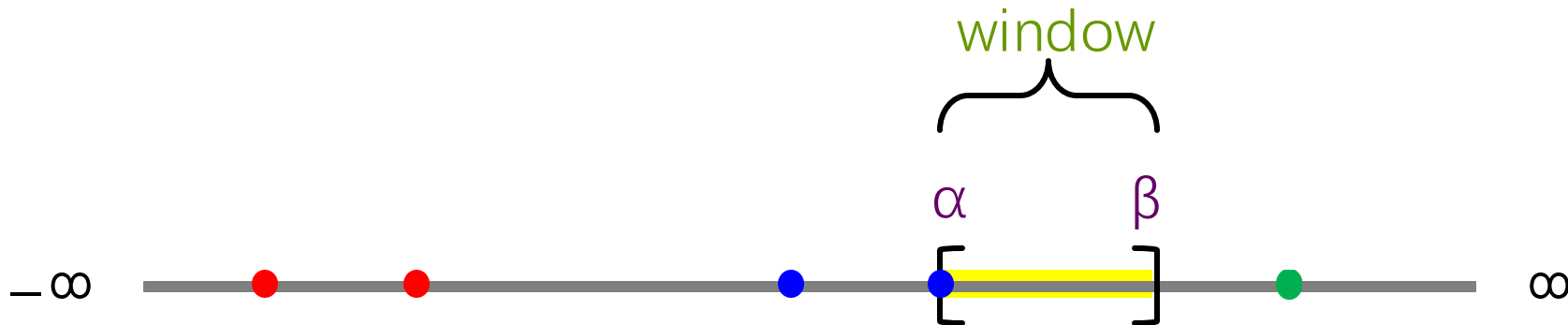
Alpha-Beta

```
int search(pos x, int alpha, int beta, int d) {
    if (d == 0 || is_leaf(x)) return static_eval(x);
    pos c[MAX_CHILDREN];
    int nc;
    gen_moves(x, c, &nc);                // generate children
    for (int i = 0; i < nc; ++i) {
        int s = -search(c[i], -beta, -alpha, d-1); // negamax
        if (s > alpha) alpha = s;
        if (alpha >= beta) return alpha;          // beta cutoff
    }
    return alpha;
}
```



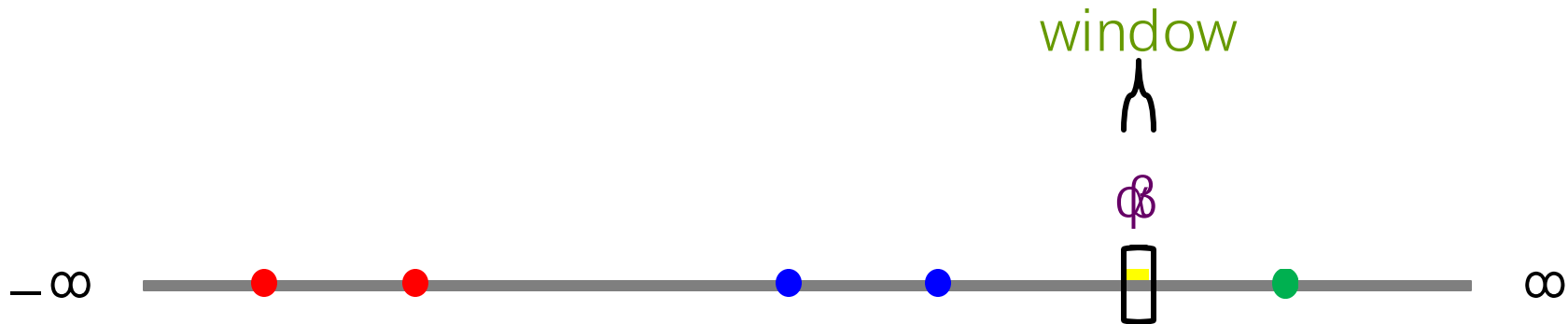
Alpha-Beta

```
int search(pos x, int alpha, int beta, int d) {  
    if (d == 0 || is_leaf(x)) return static_eval(x);  
    pos c[MAX_CHILDREN];  
    int nc;  
    gen_moves(x, c, &nc); // generate children  
    for (int i = 0; i < nc; ++i) {  
        int s = -search(c[i], -beta, -alpha, d-1); // negamax  
        if (s > alpha) alpha = s;  
        if (alpha >= beta) return alpha; // beta cutoff  
    }  
    return alpha;  
}
```



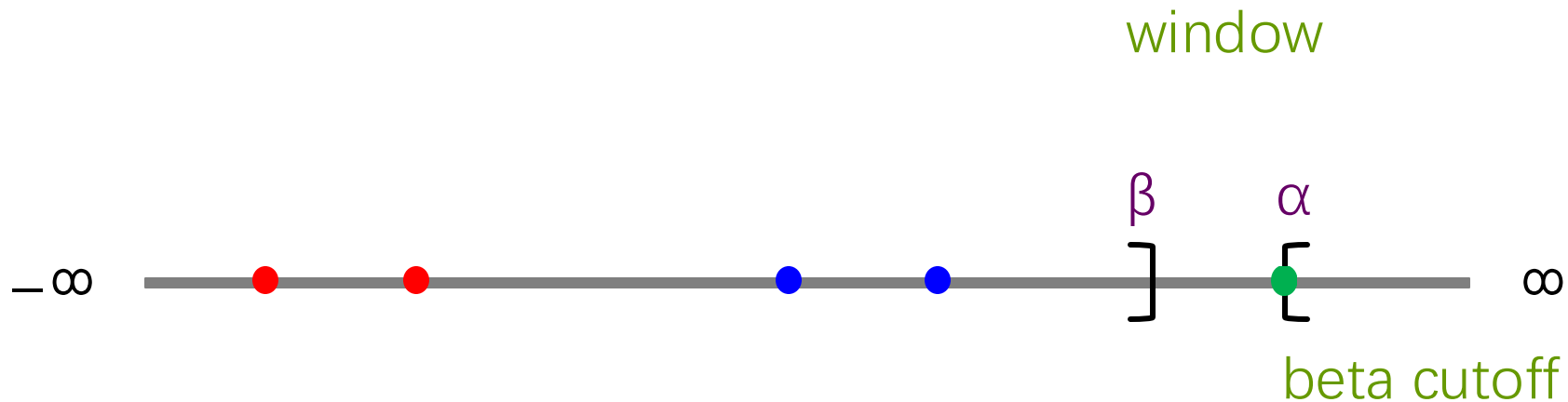
Alpha-Beta

```
int search(pos x, int alpha, int beta, int d) {  
    if (d == 0 || is_leaf(x)) return static_eval(x);  
    pos c[MAX_CHILDREN];  
    int nc;  
    gen_moves(x, c, &nc); // generate children  
    for (int i = 0; i < nc; ++i) {  
        int s = -search(c[i], -beta, -alpha, d-1); // negamax  
        if (s > alpha) alpha = s;  
        if (alpha >= beta) return alpha; // beta cutoff  
    }  
    return alpha;  
}
```



Alpha-Beta

```
int search(pos x, int alpha, int beta, int d) {  
    if (d == 0 || is_leaf(x)) return static_eval(x);  
    pos c[MAX_CHILDREN];  
    int nc;  
    gen_moves(x, c, &nc); // generate children  
    for (int i = 0; i < nc; ++i) {  
        int s = -search(c[i], -beta, -alpha, d-1); // negamax  
        if (s > alpha) alpha = s;  
        if (alpha >= beta) return alpha; // beta cutoff  
    }  
    return alpha;  
}
```

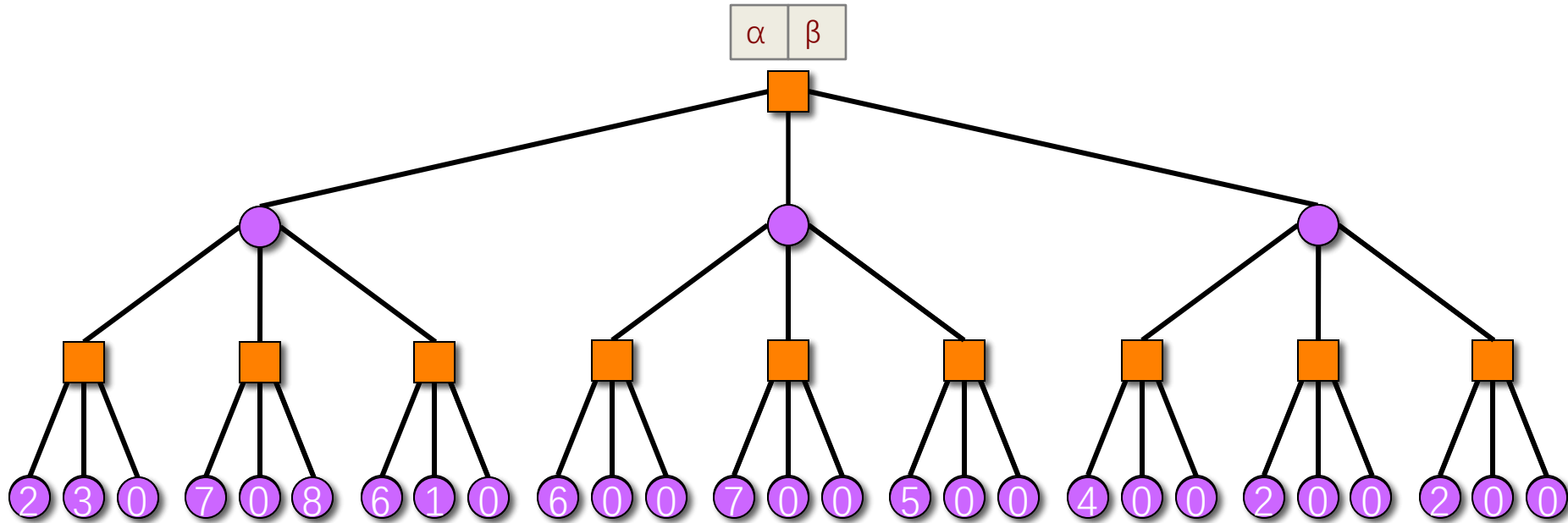


ALPHA-BETA SEARCH

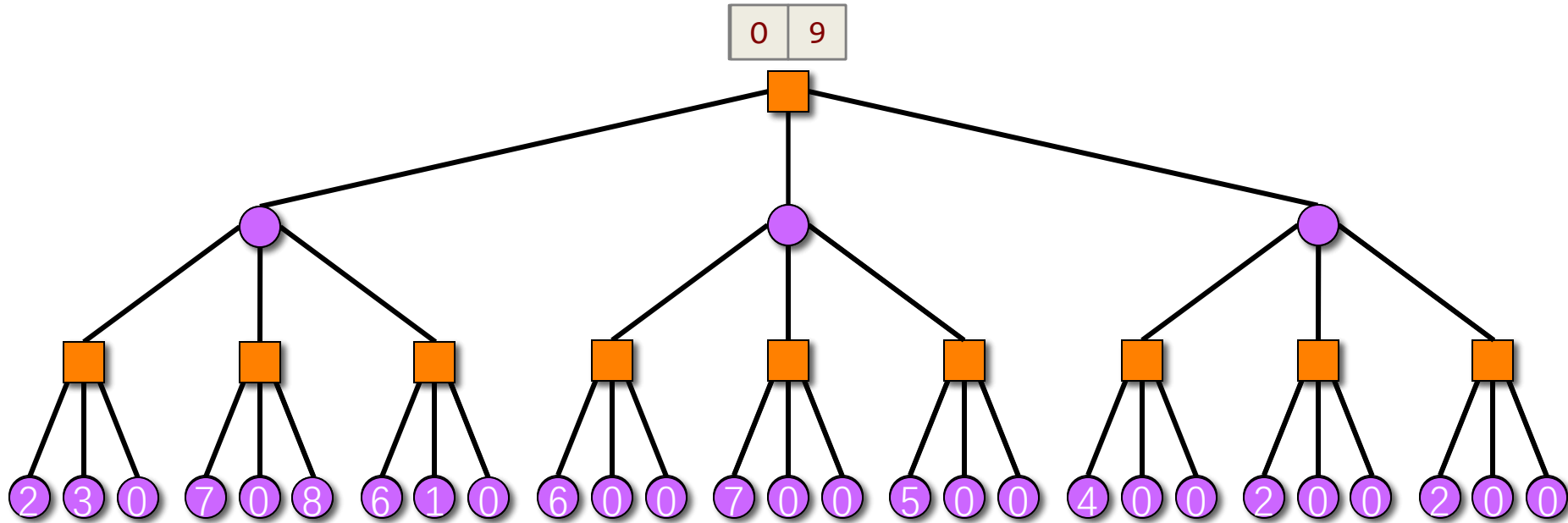
A WORKED EXAMPLE



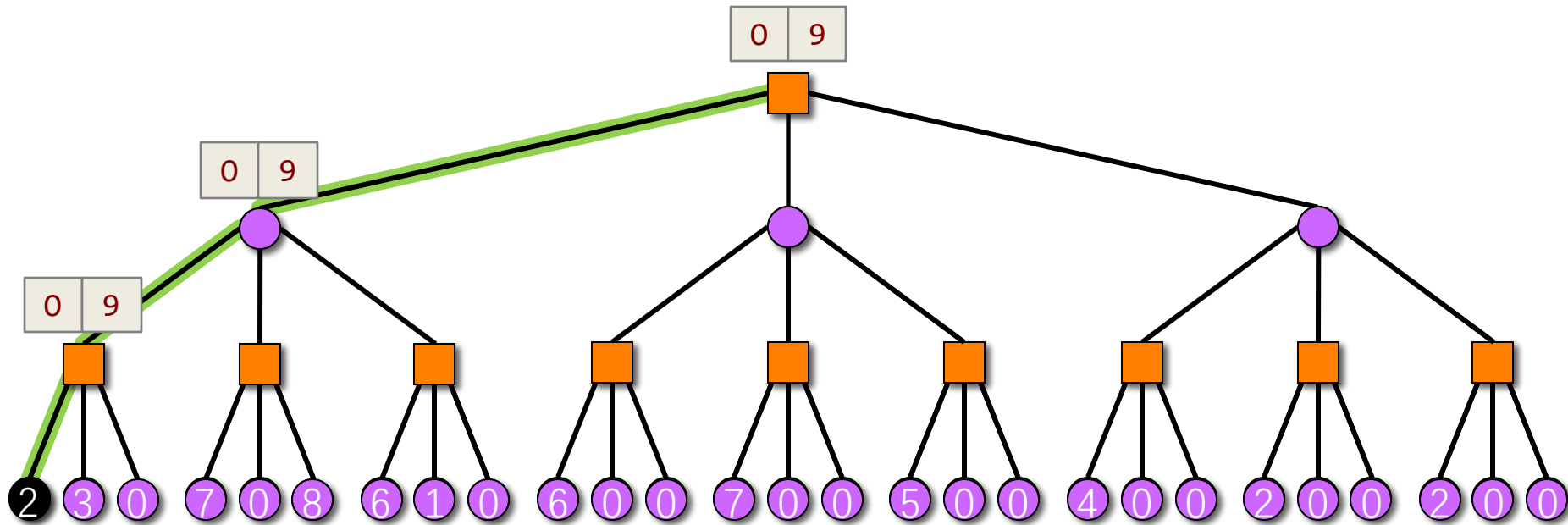
Alpha-Beta Search: Example



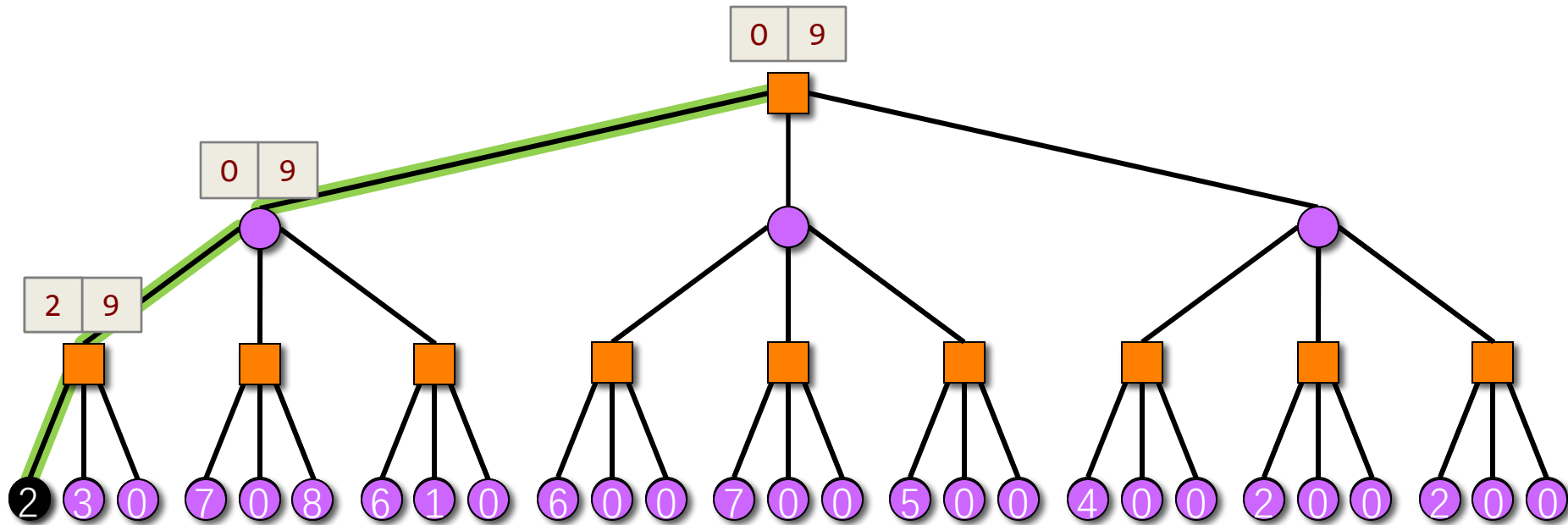
Alpha-Beta Search: Example



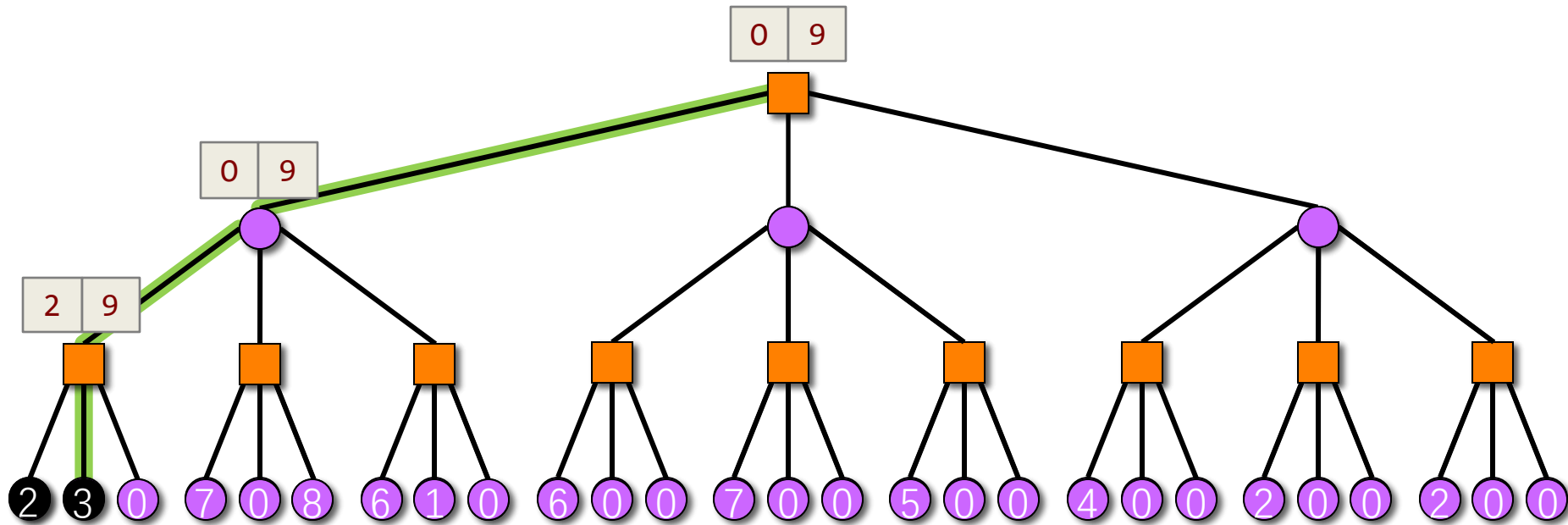
Alpha-Beta Search: Example



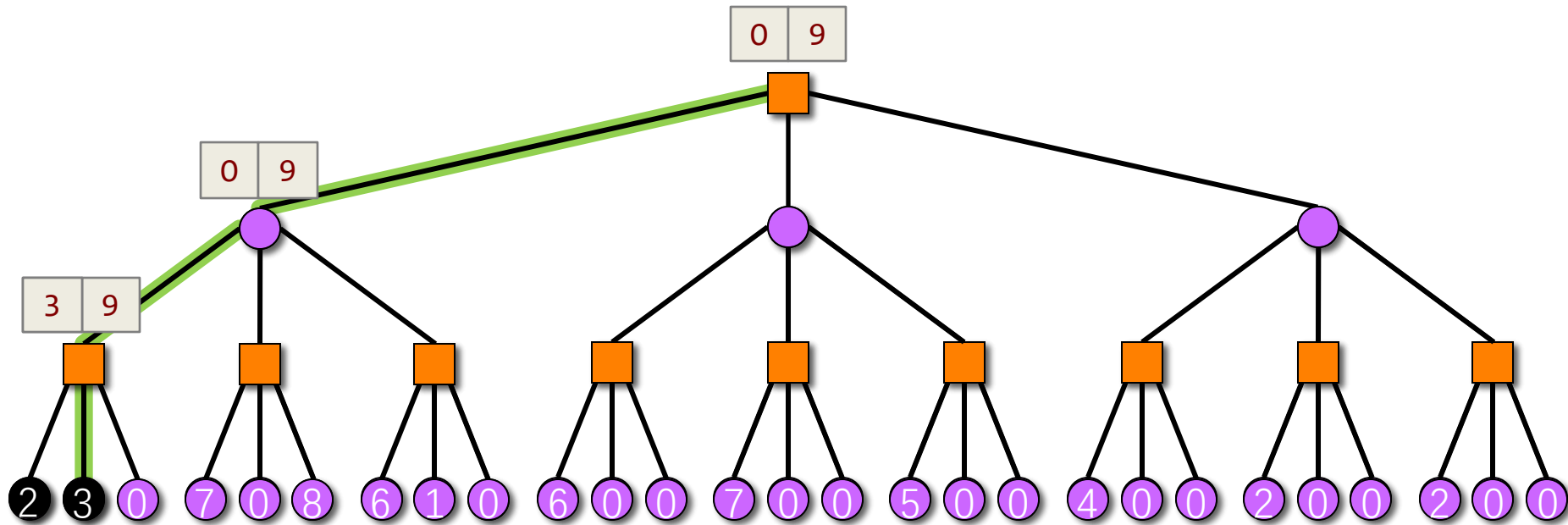
Alpha-Beta Search: Example



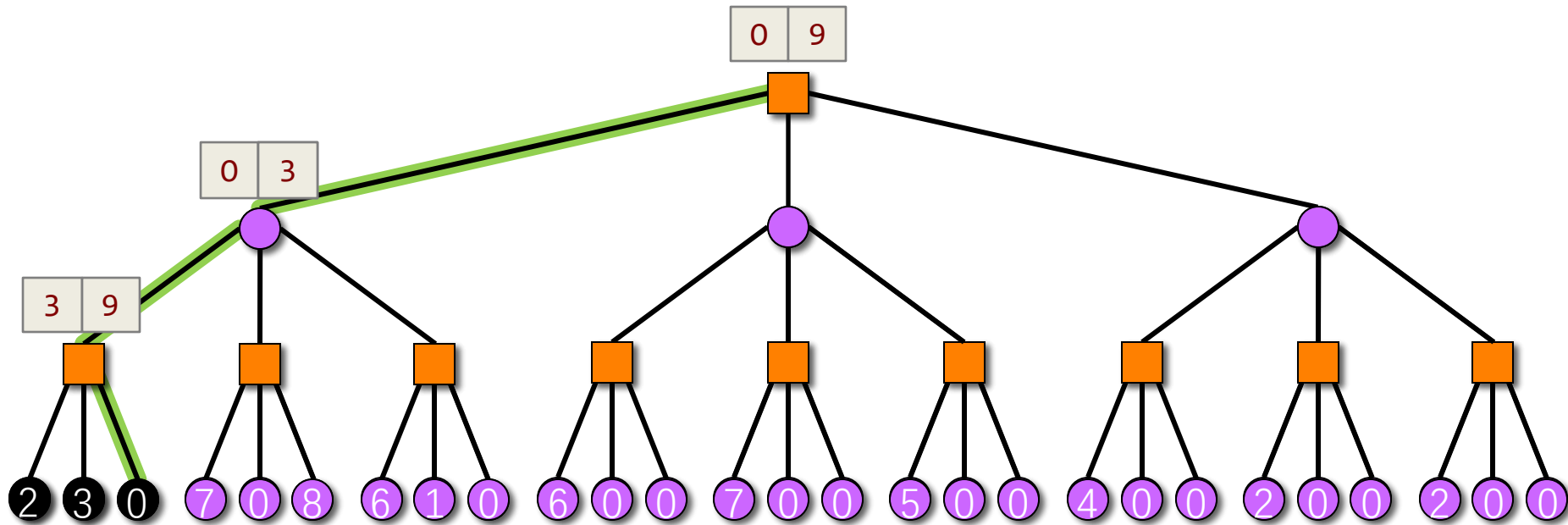
Alpha-Beta Search: Example



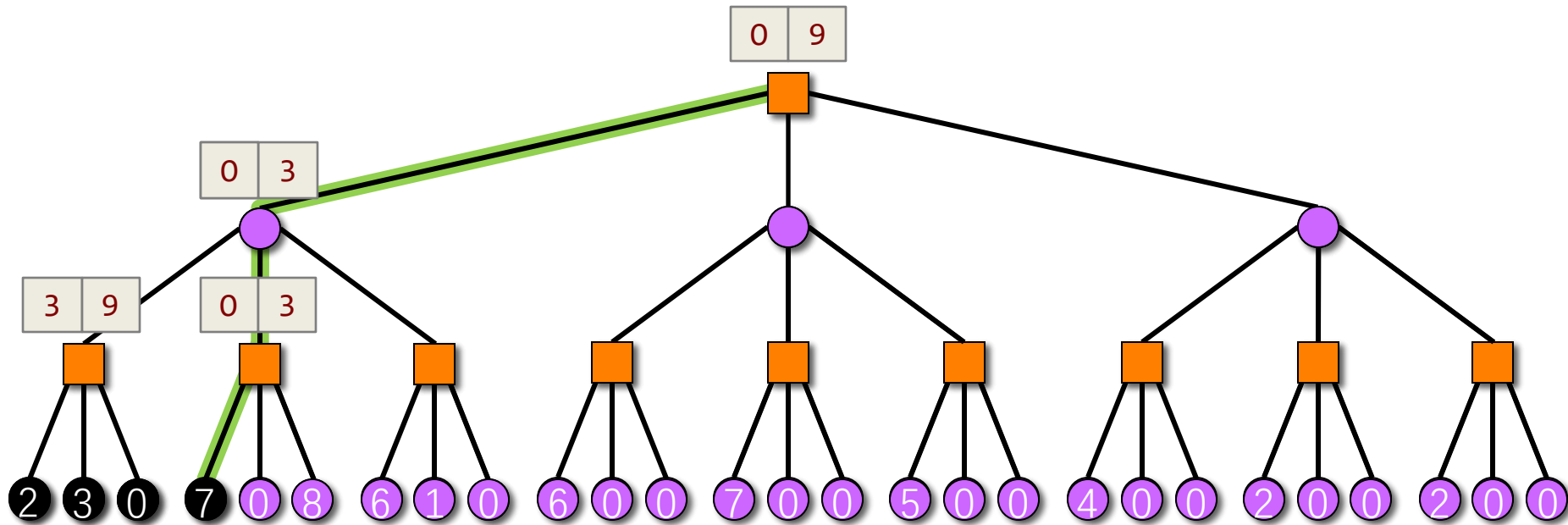
Alpha-Beta Search: Example



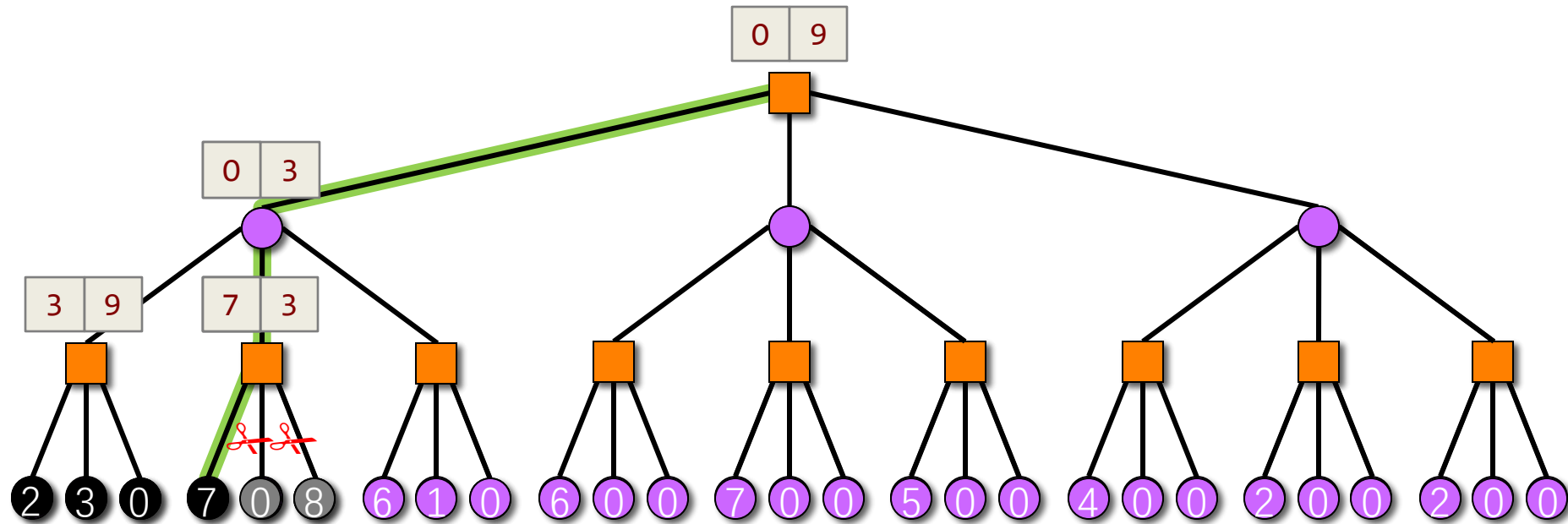
Alpha-Beta Search: Example



Alpha-Beta Search: Example

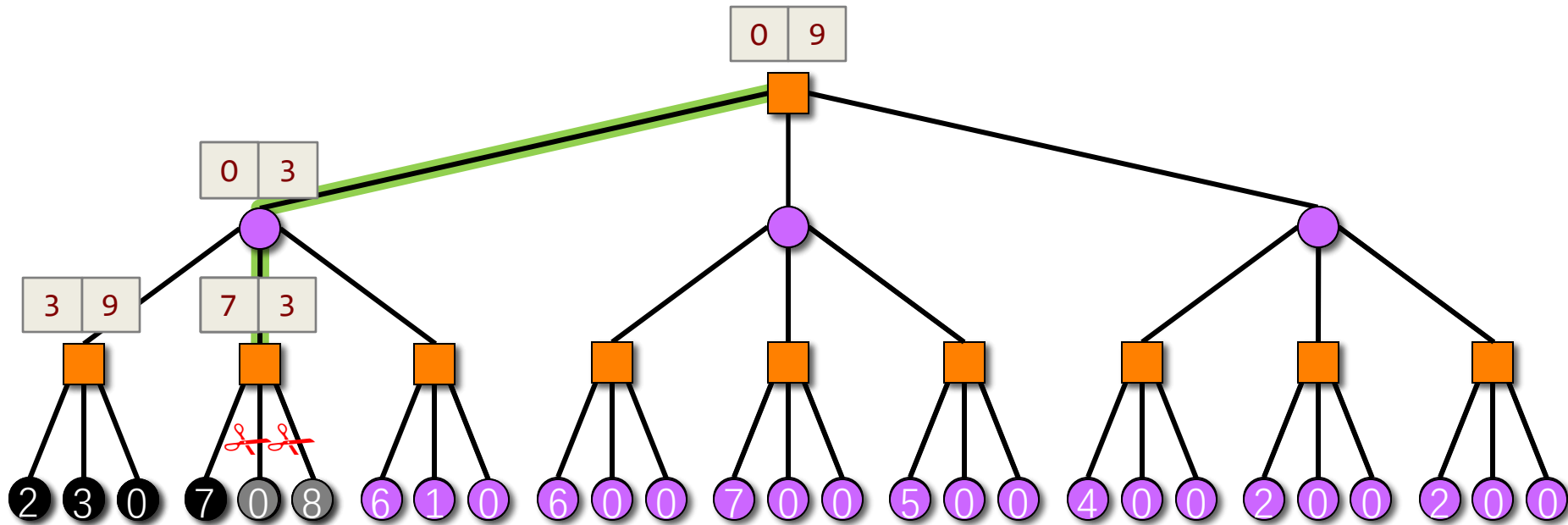


Alpha-Beta Search: Example

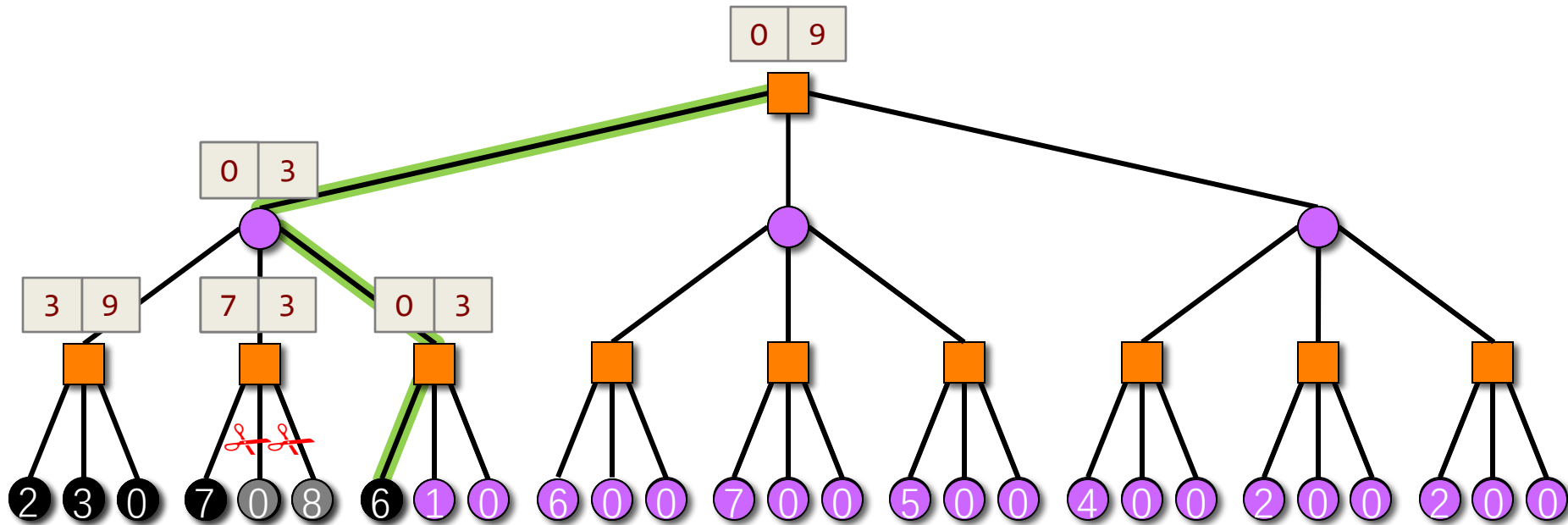


beta cutoff

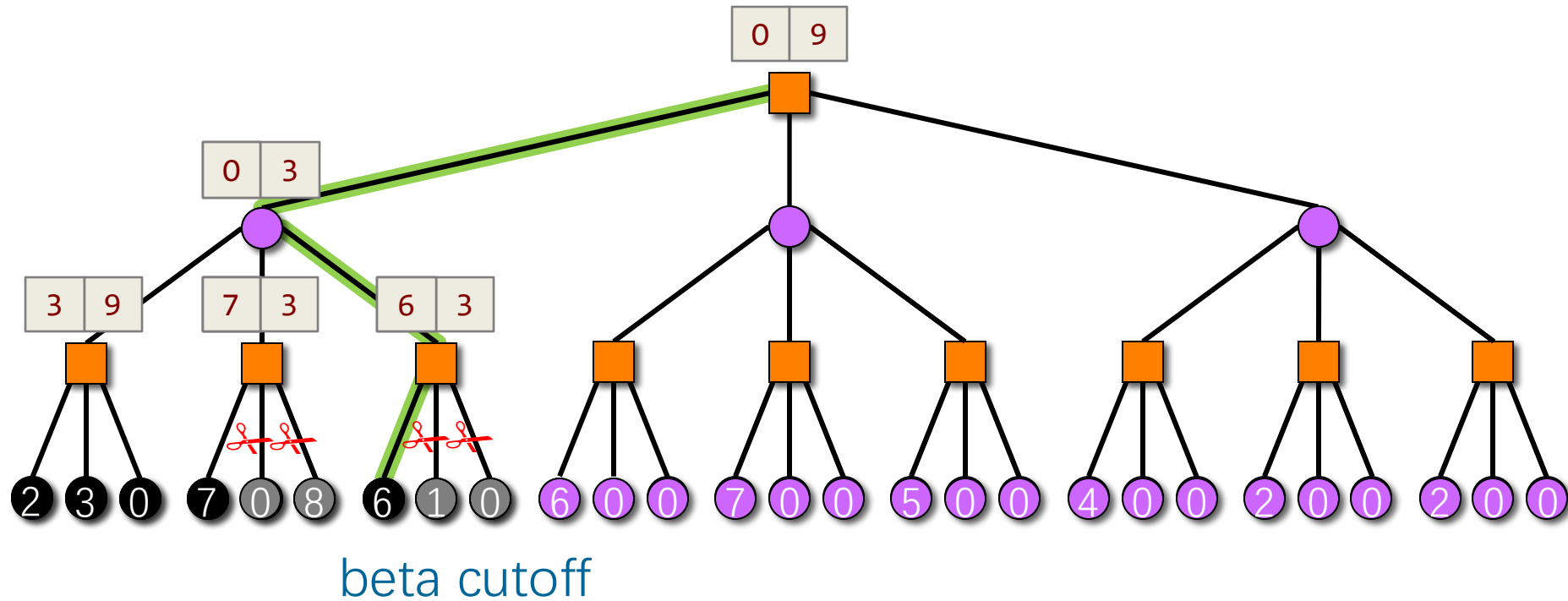
Alpha-Beta Search: Example



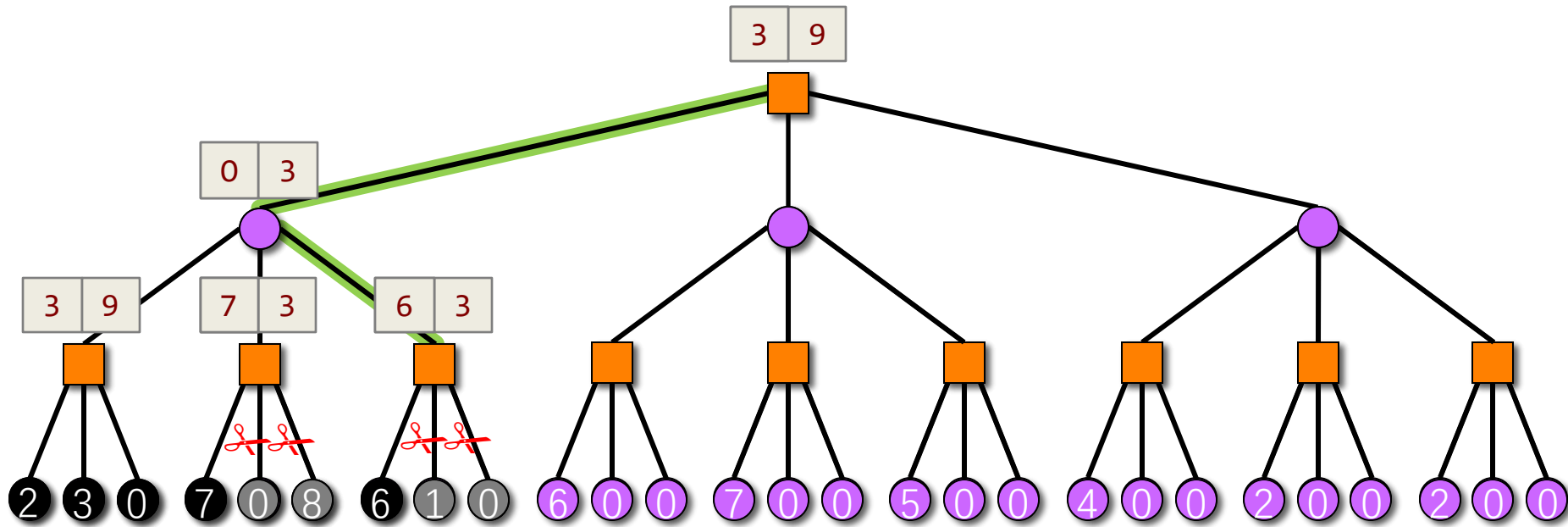
Alpha-Beta Search: Example



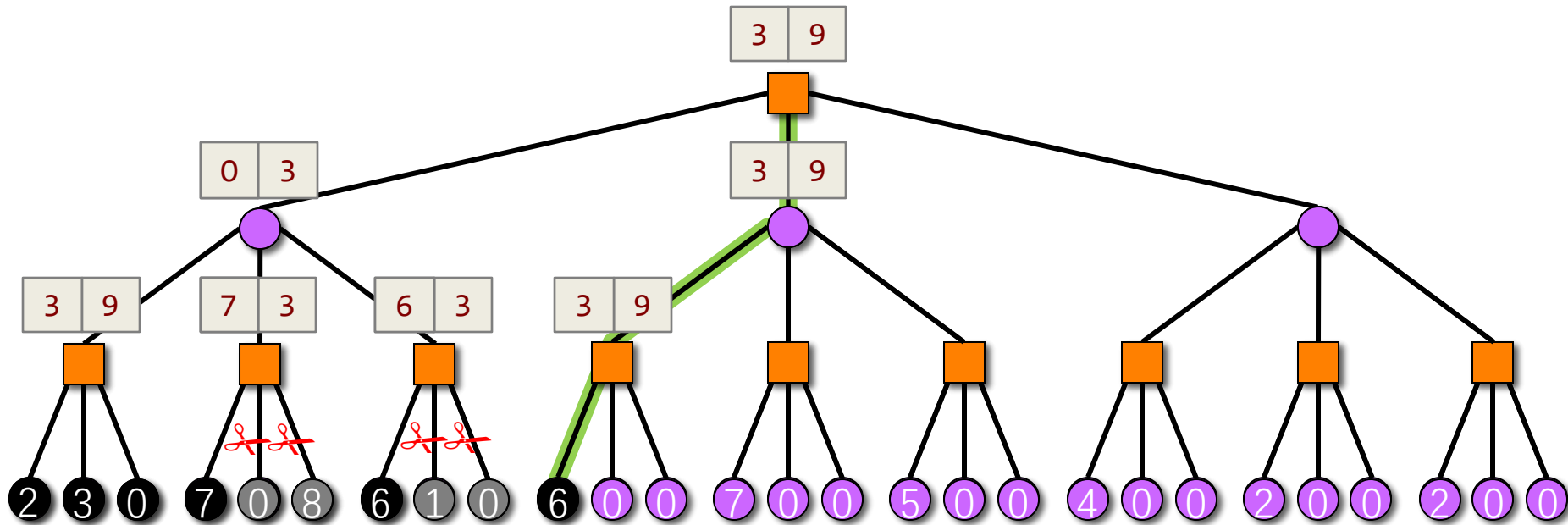
Alpha-Beta Search: Example



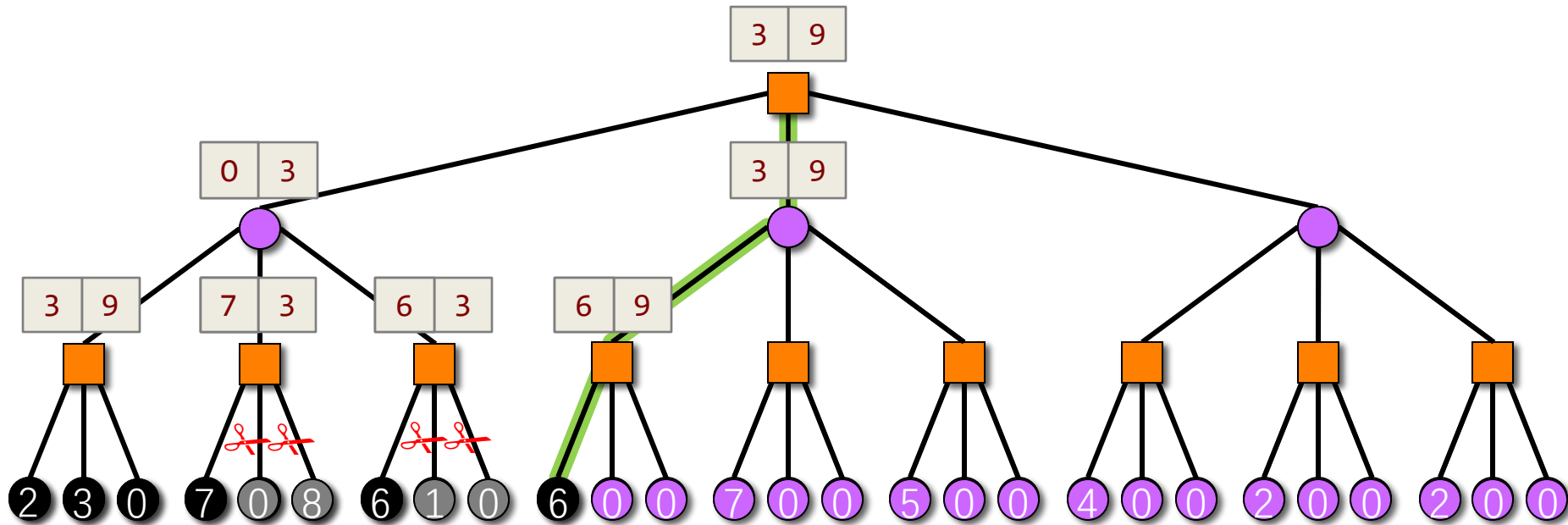
Alpha-Beta Search: Example



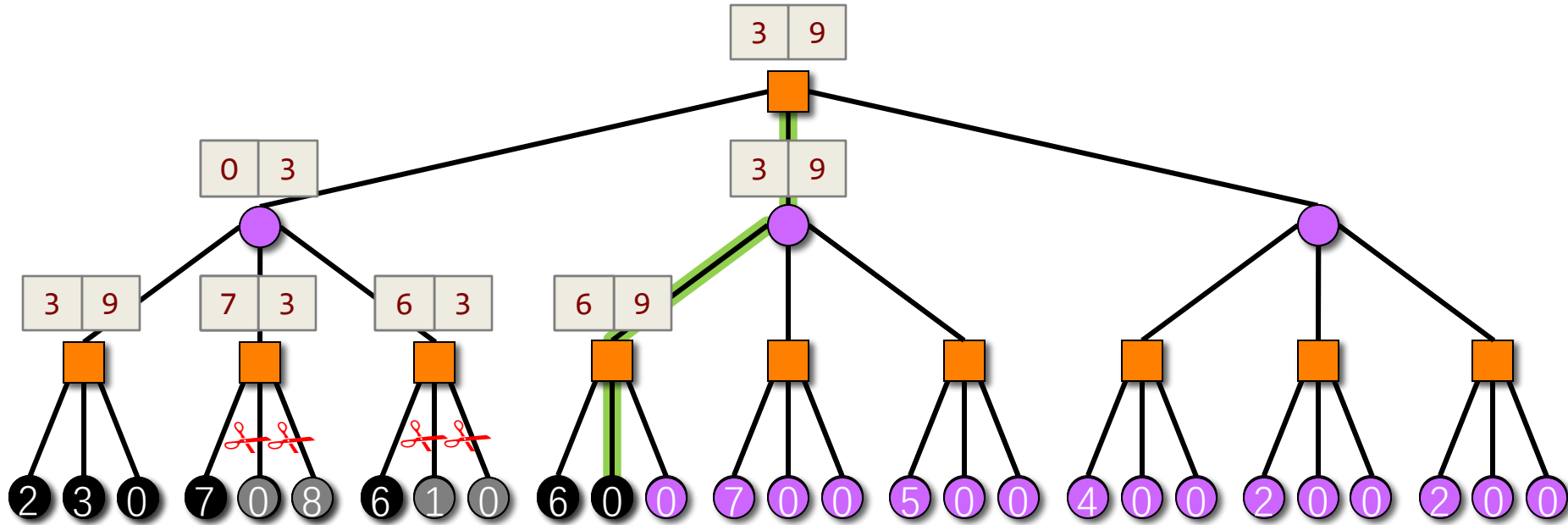
Alpha-Beta Search: Example



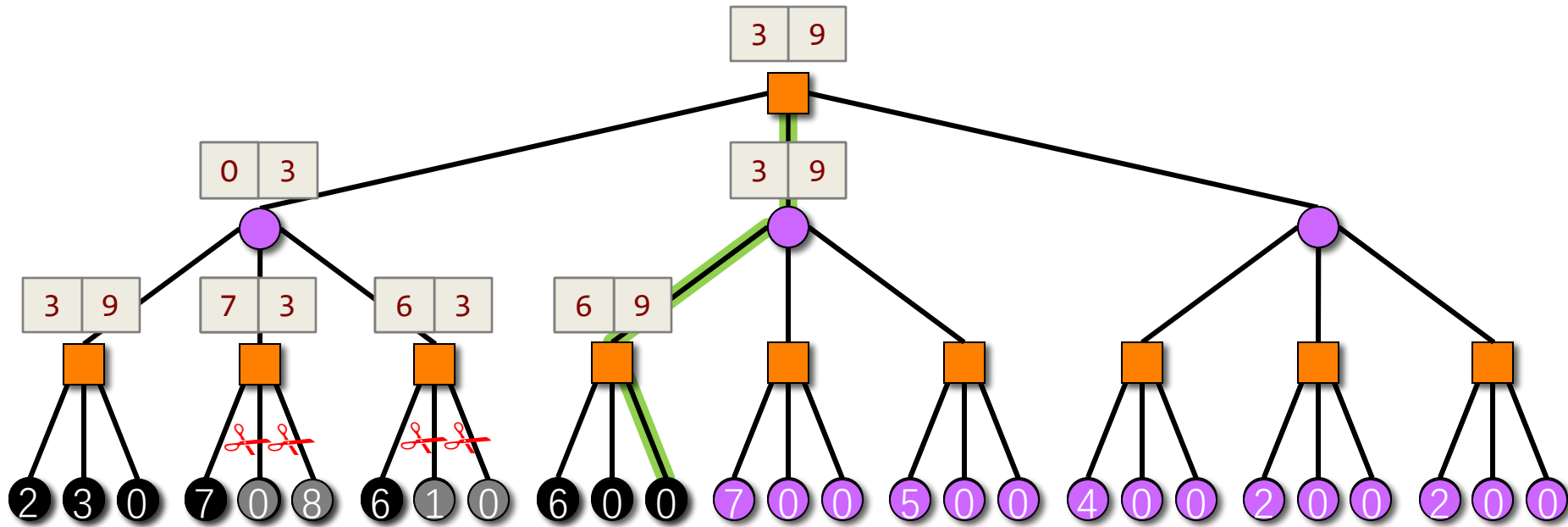
Alpha-Beta Search: Example



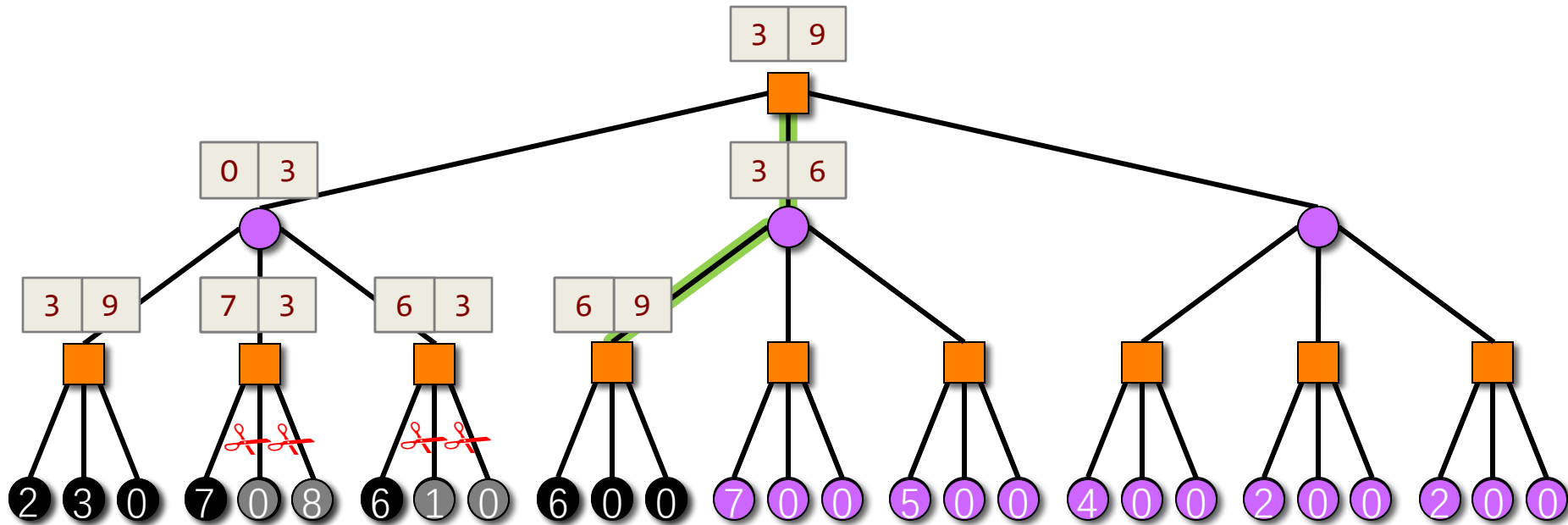
Alpha-Beta Search: Example



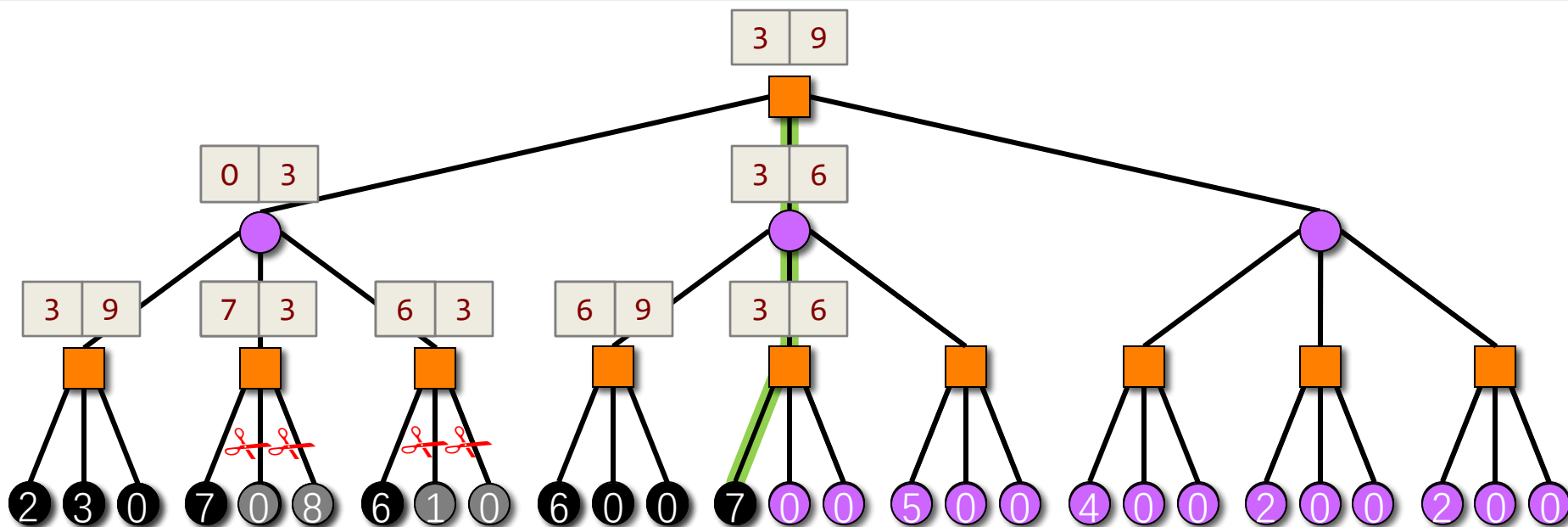
Alpha-Beta Search: Example



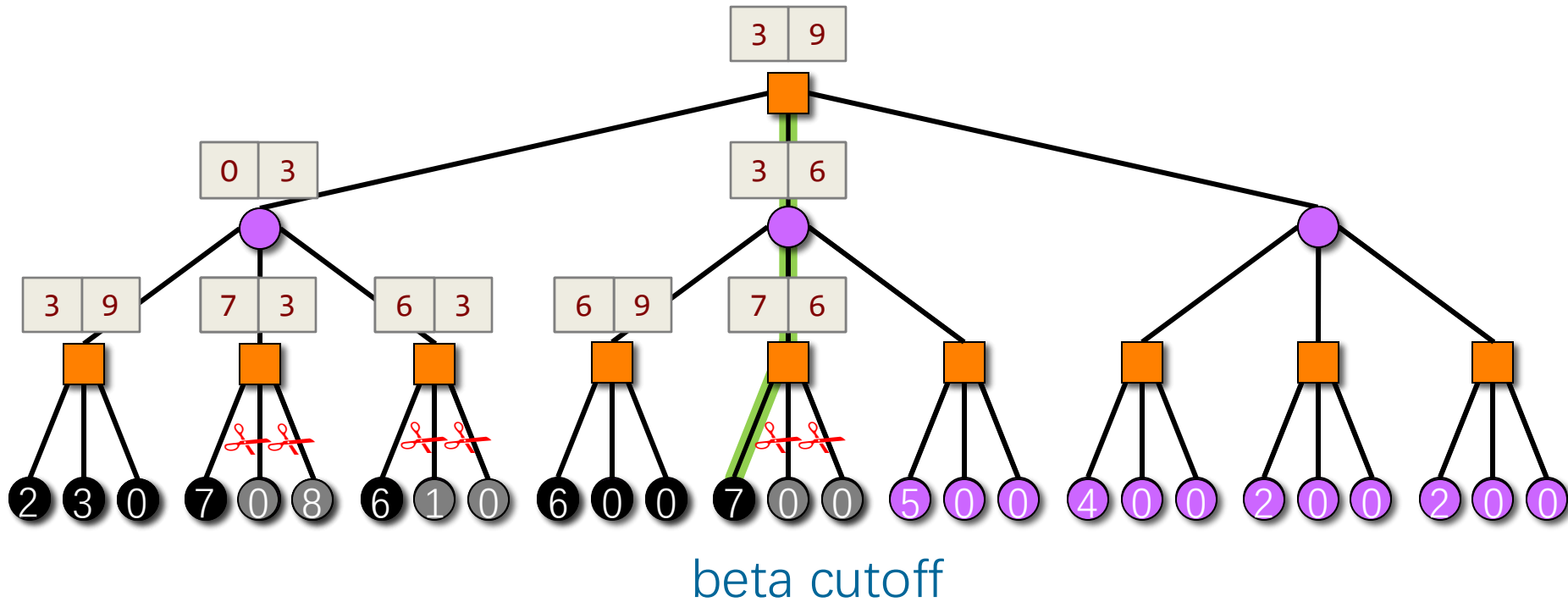
Alpha-Beta Search: Example



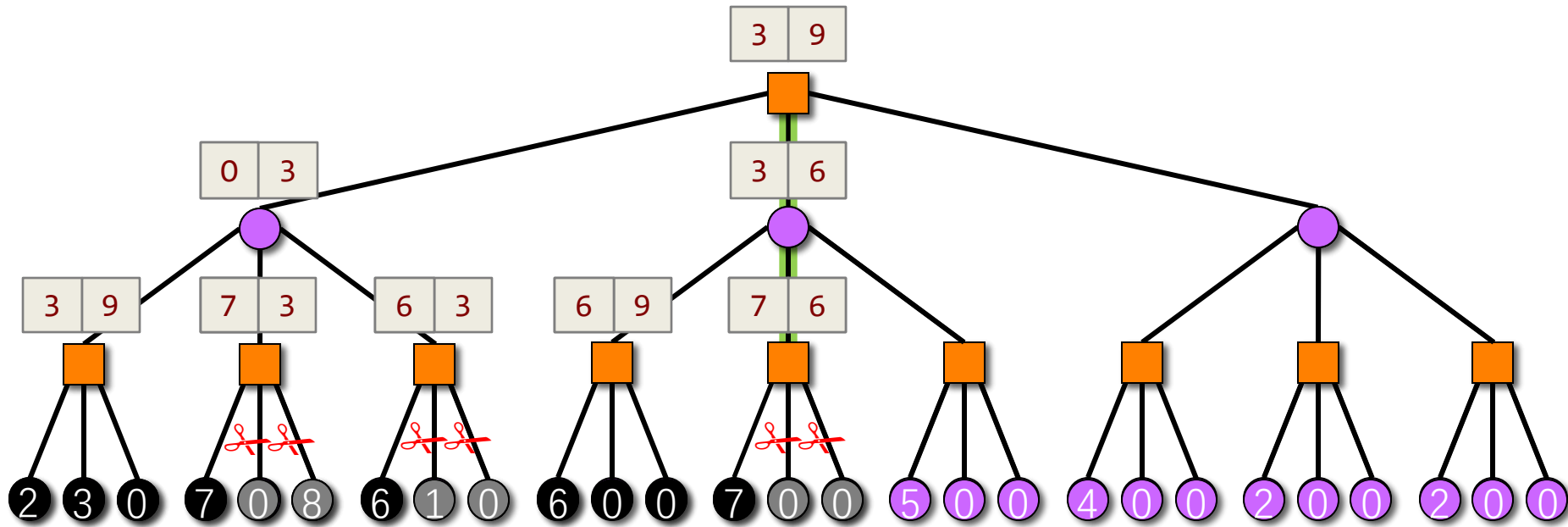
Alpha-Beta Search: Example



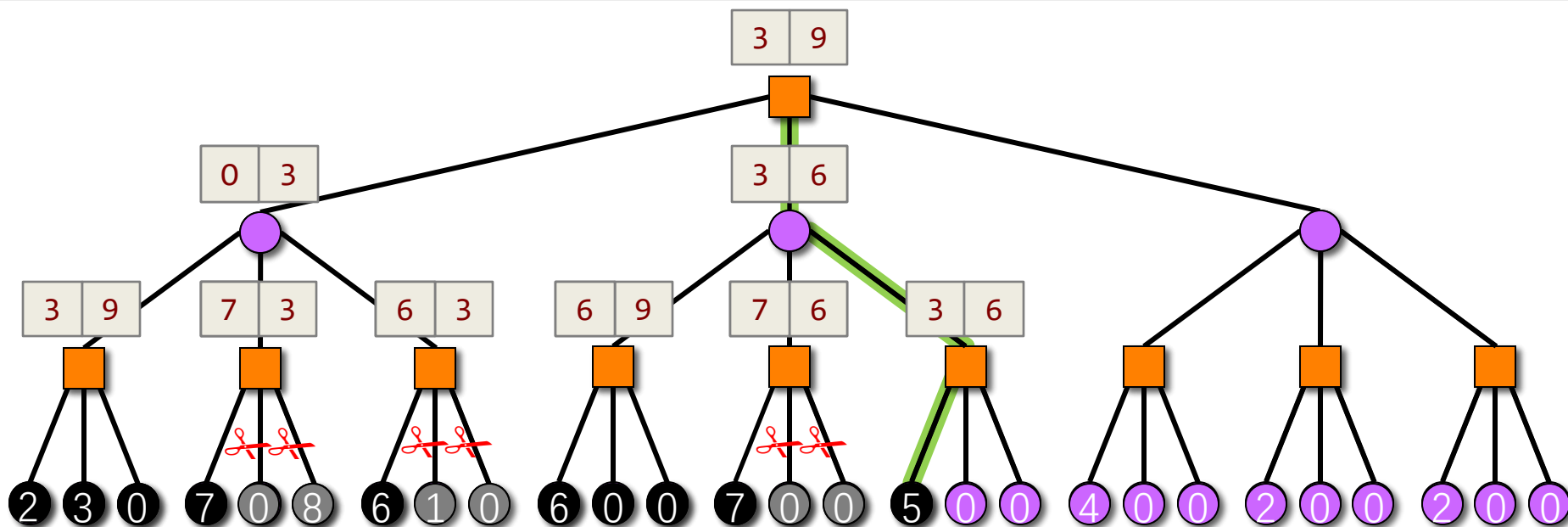
Alpha-Beta Search: Example



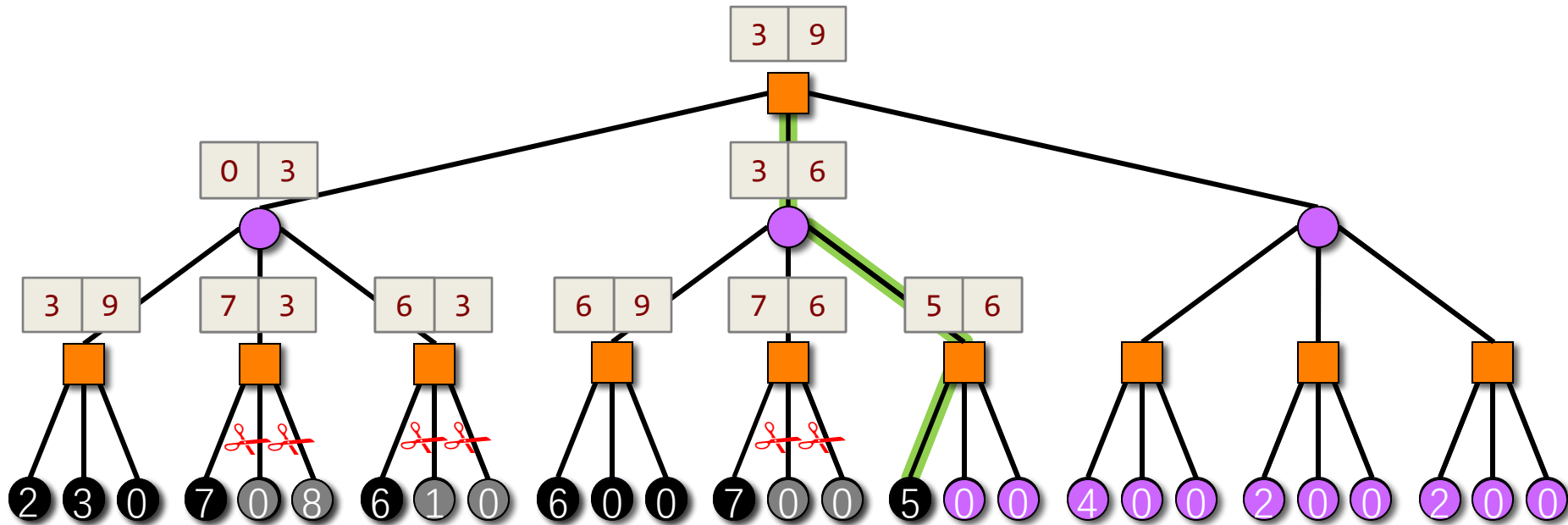
Alpha-Beta Search: Example



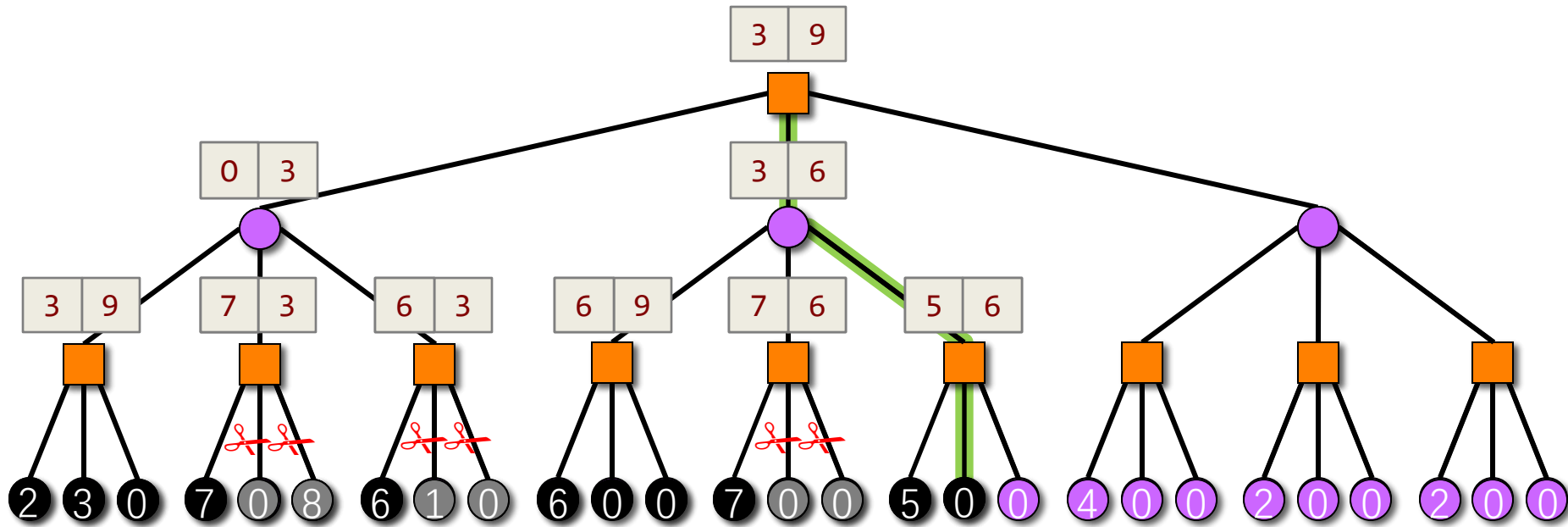
Alpha-Beta Search: Example



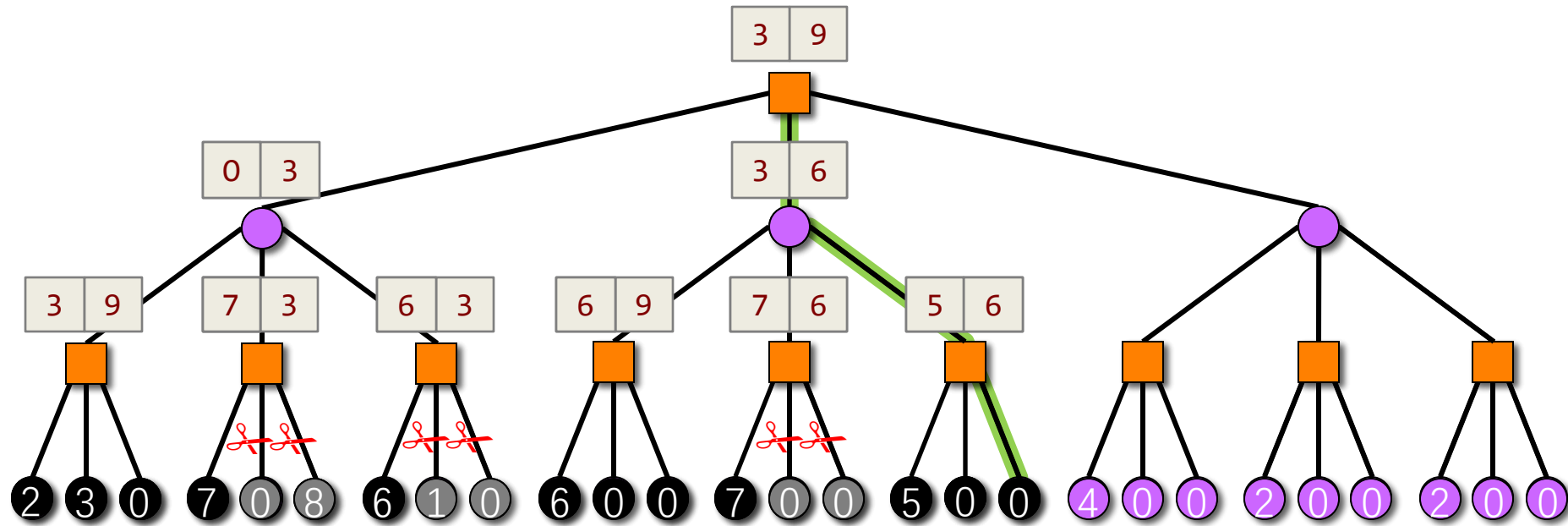
Alpha-Beta Search: Example



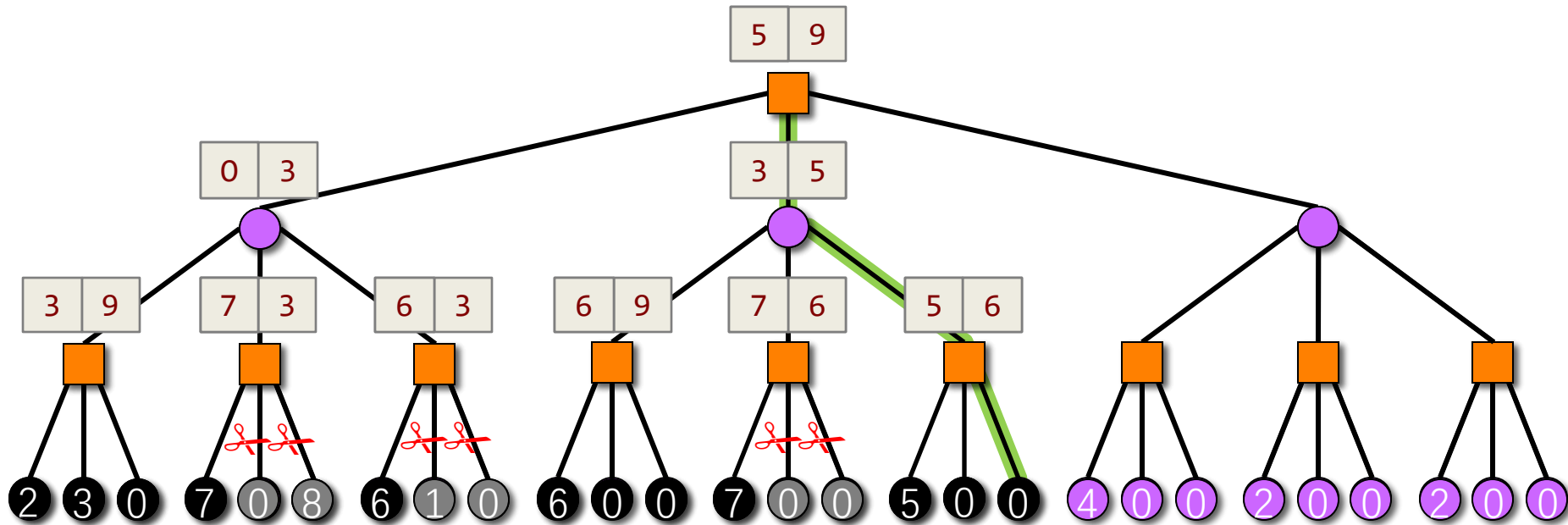
Alpha-Beta Search: Example



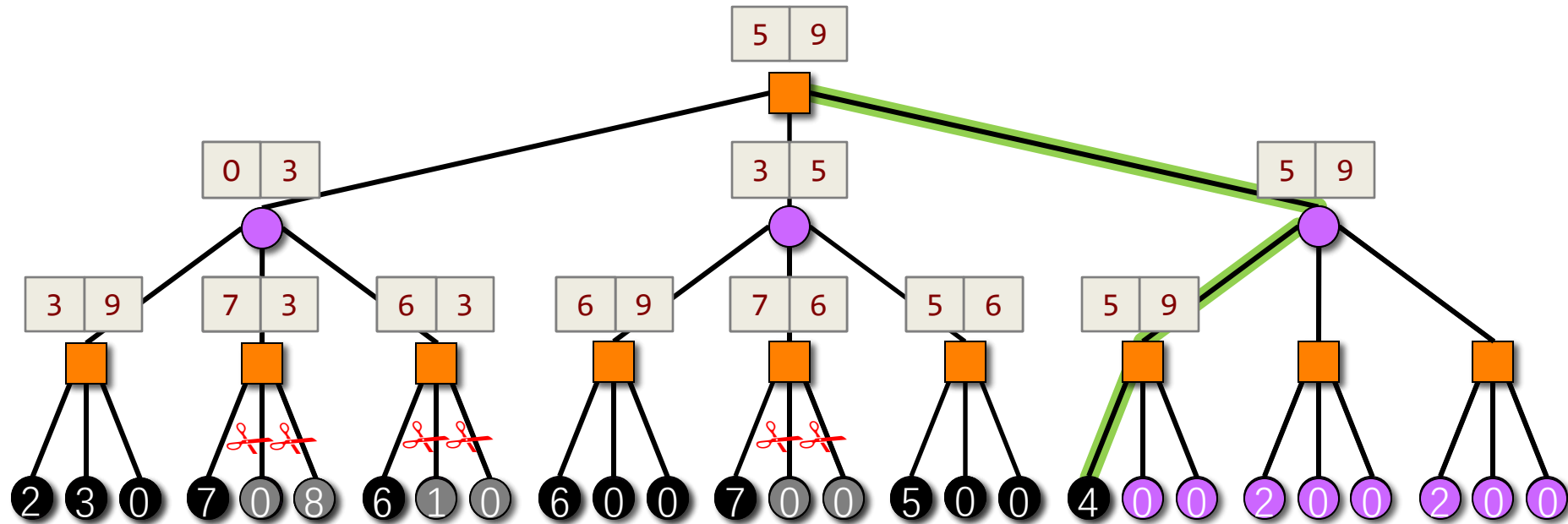
Alpha-Beta Search: Example



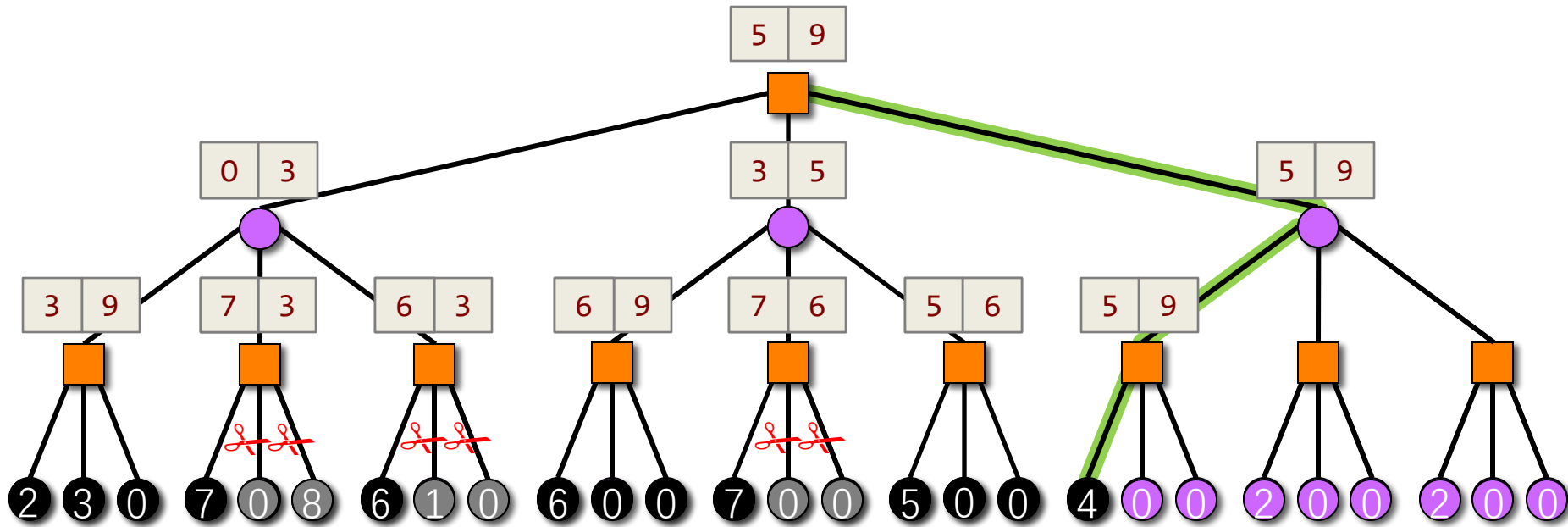
Alpha-Beta Search: Example



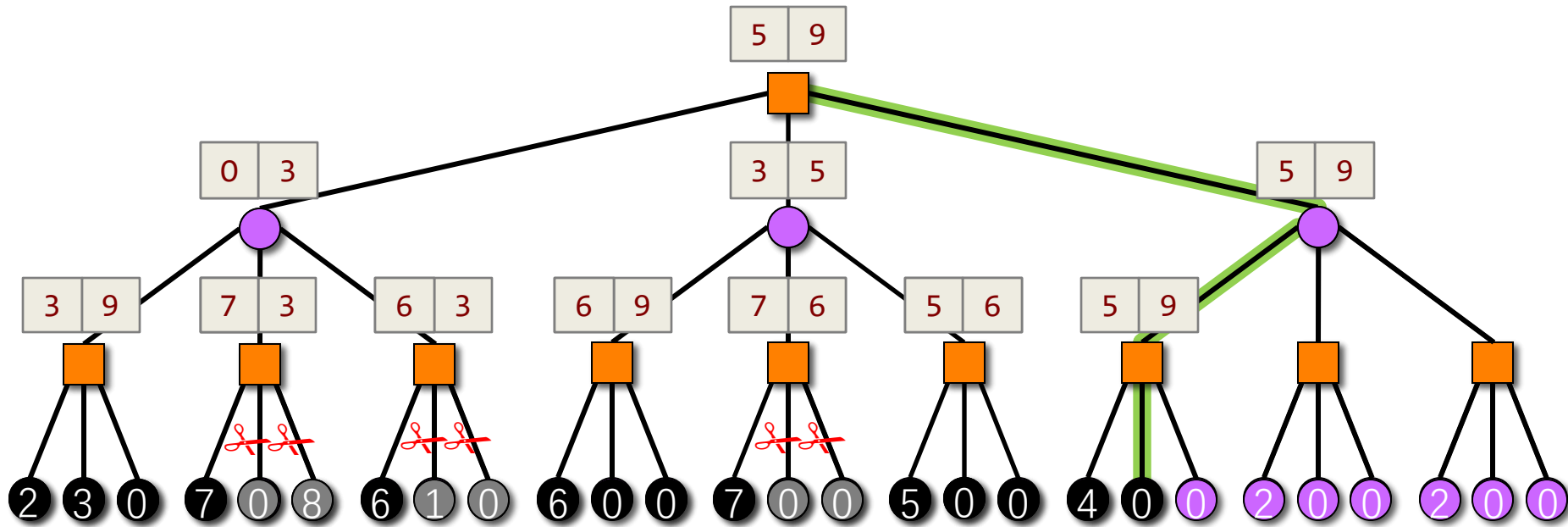
Alpha-Beta Search: Example



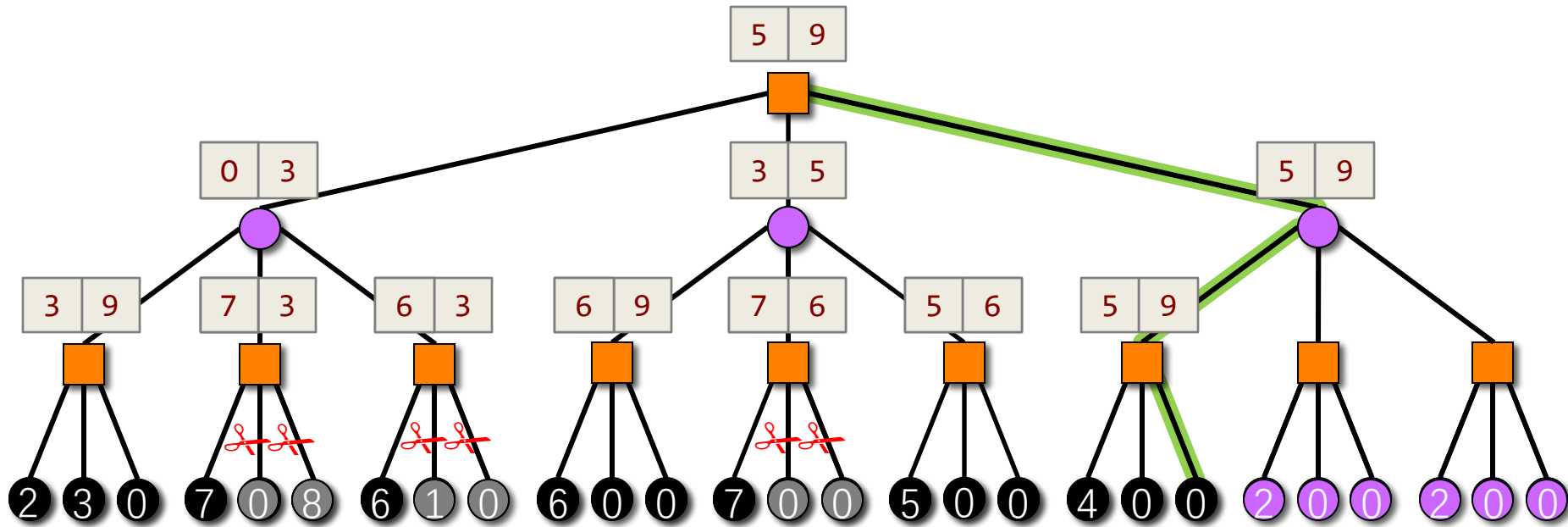
Alpha-Beta Search: Example



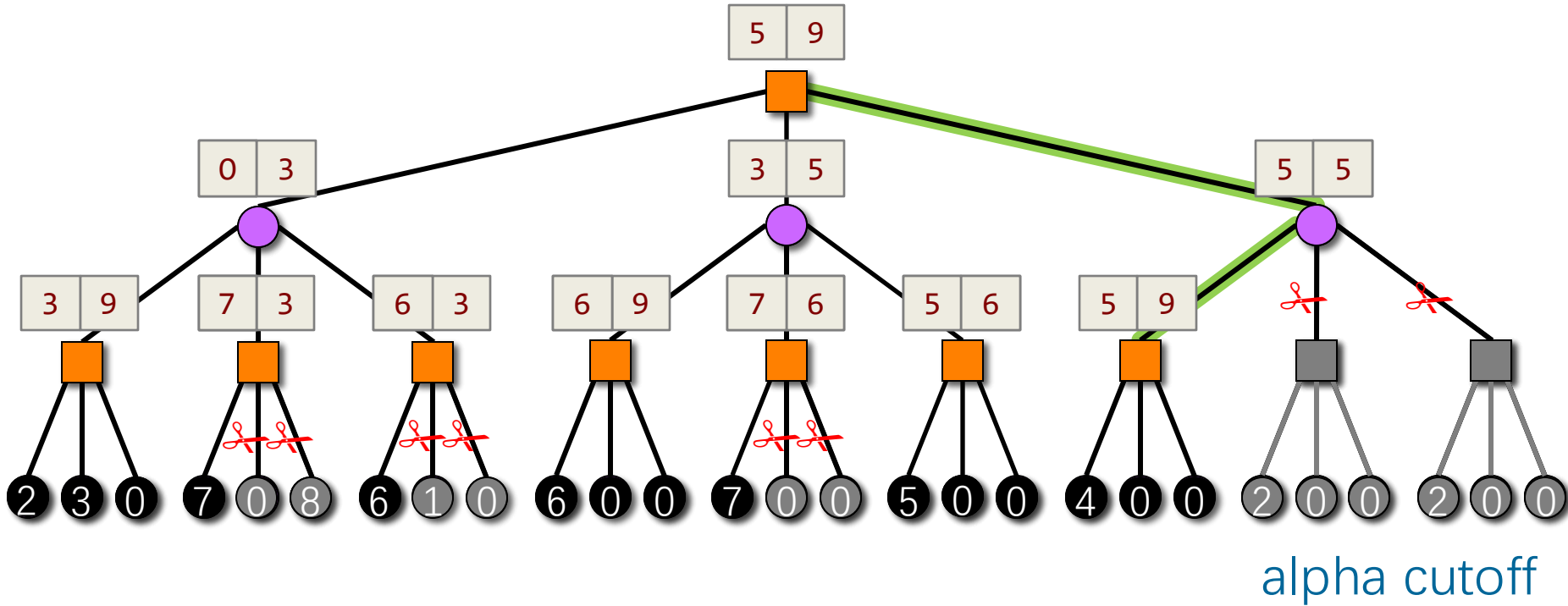
Alpha-Beta Search: Example



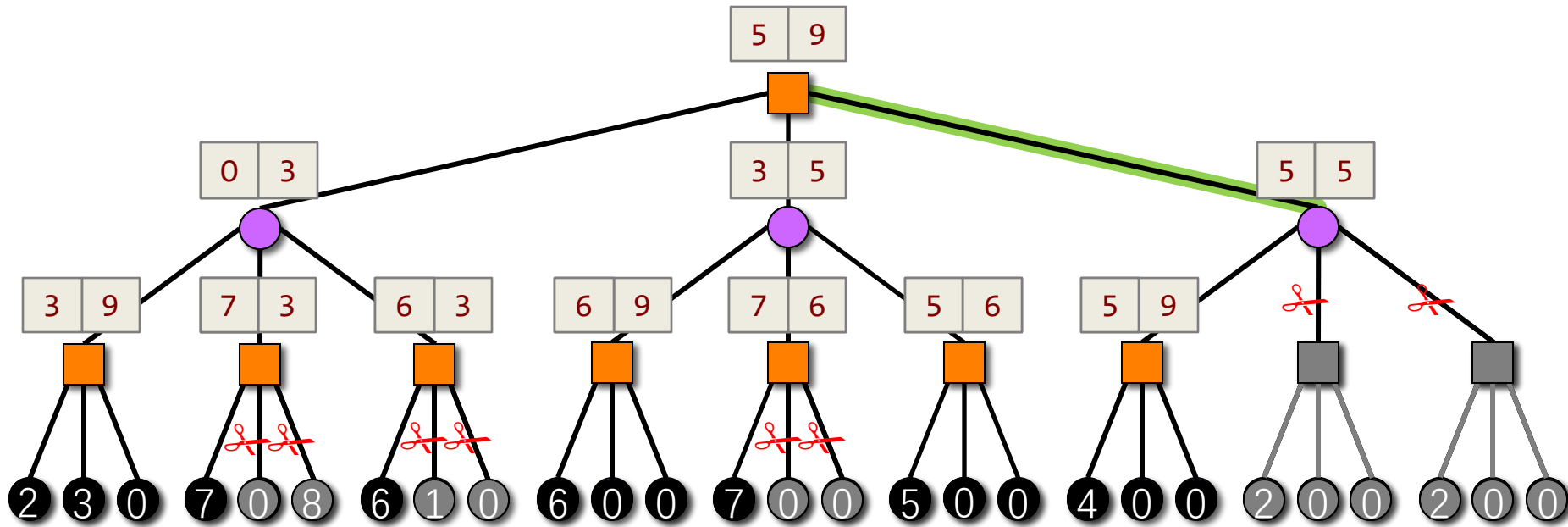
Alpha-Beta Search: Example



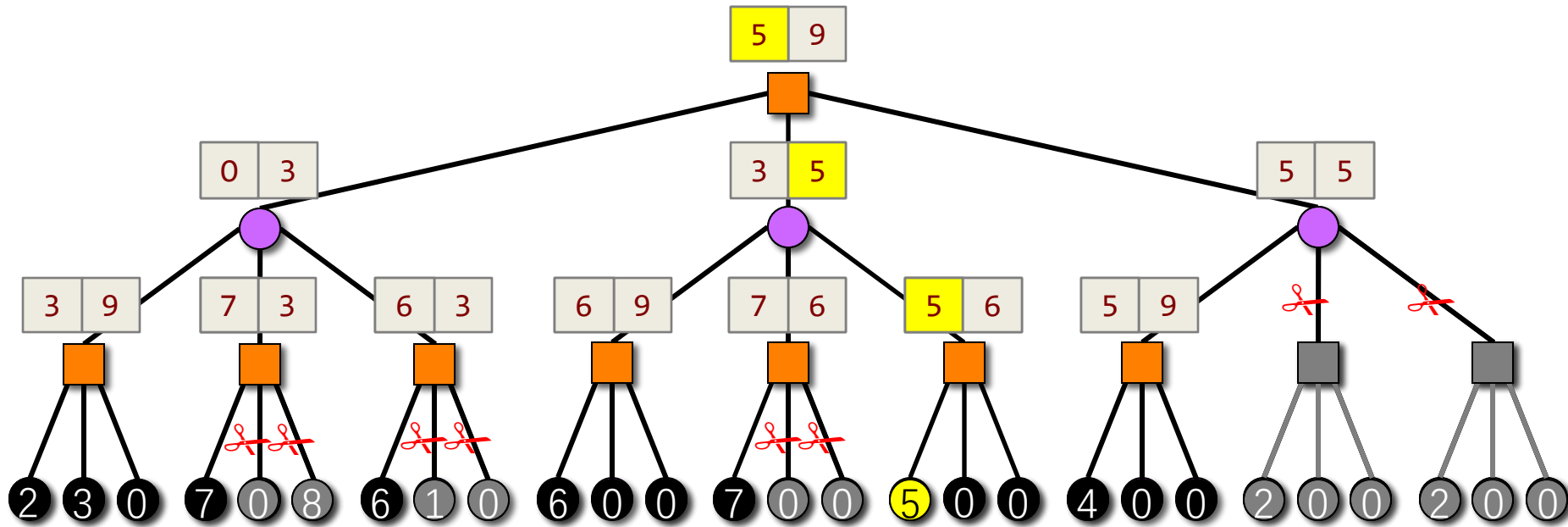
Alpha-Beta Search: Example



Alpha-Beta Search: Example



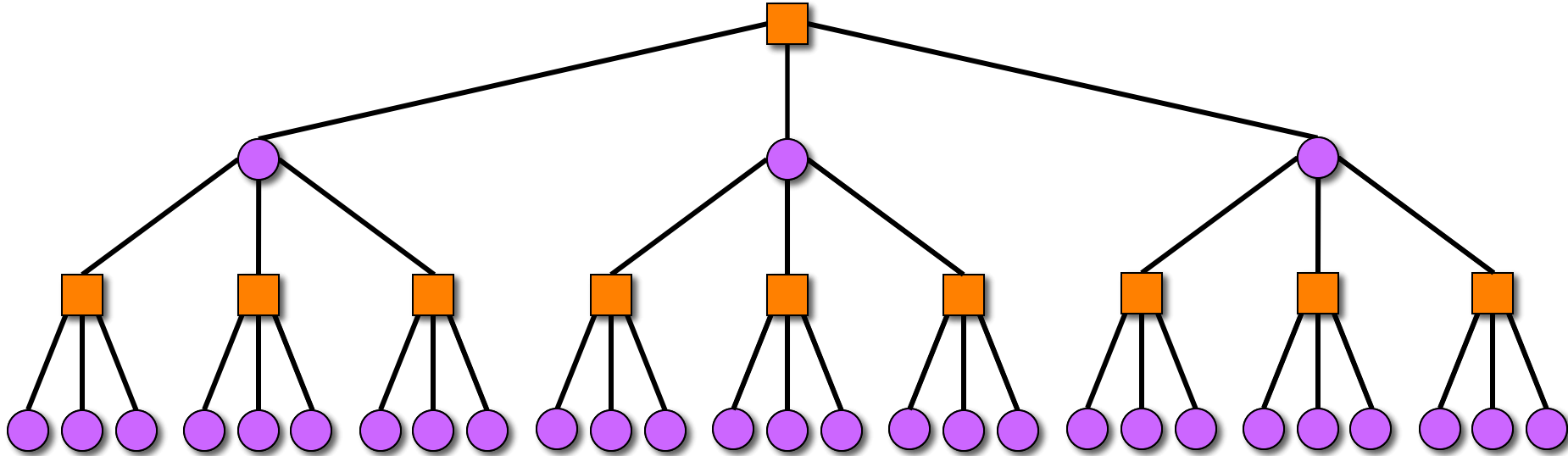
Alpha-Beta Search: Example



ALPHA-BETA SEARCH ANALYSIS



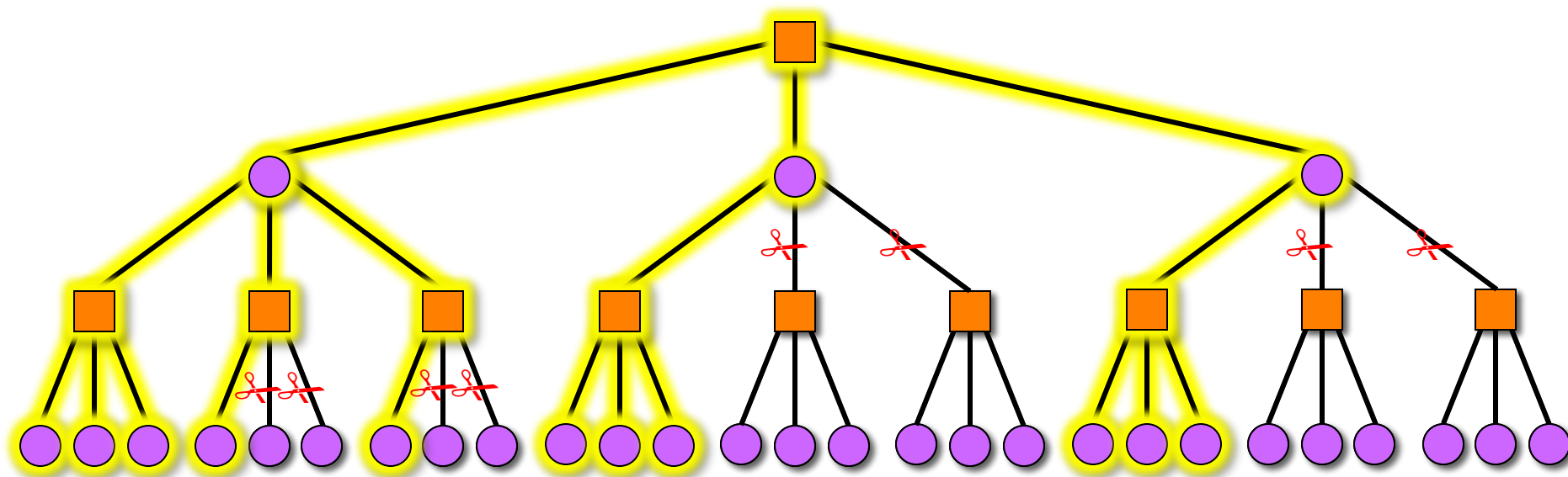
Min-Max Game Tree



The number of nodes in a game tree with branching factor $b > 1$ and depth $d \geq 0$ is

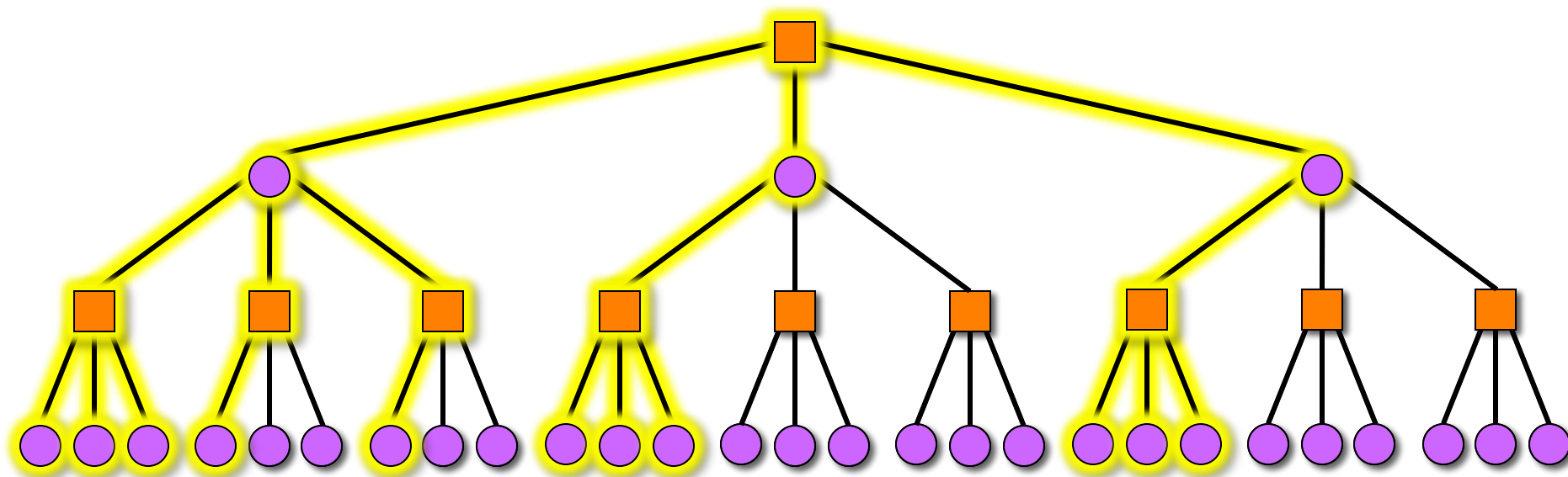
$$1 + b + b^2 + \dots + b^d = \frac{b^{d+1} - 1}{b - 1} \approx b^d.$$

Best-Case Alpha-Beta Analysis



Theorem [KM75]. For a game tree with branching factor $b > 1$ and depth $d \geq 0$, an alpha-beta search with moves searched in **best-first order** examines exactly $b^{\lceil k/2 \rceil} + b^{\lfloor k/2 \rfloor} - 1$ nodes at ply k , which is approximately $2b^{d/2}$ nodes overall. ■

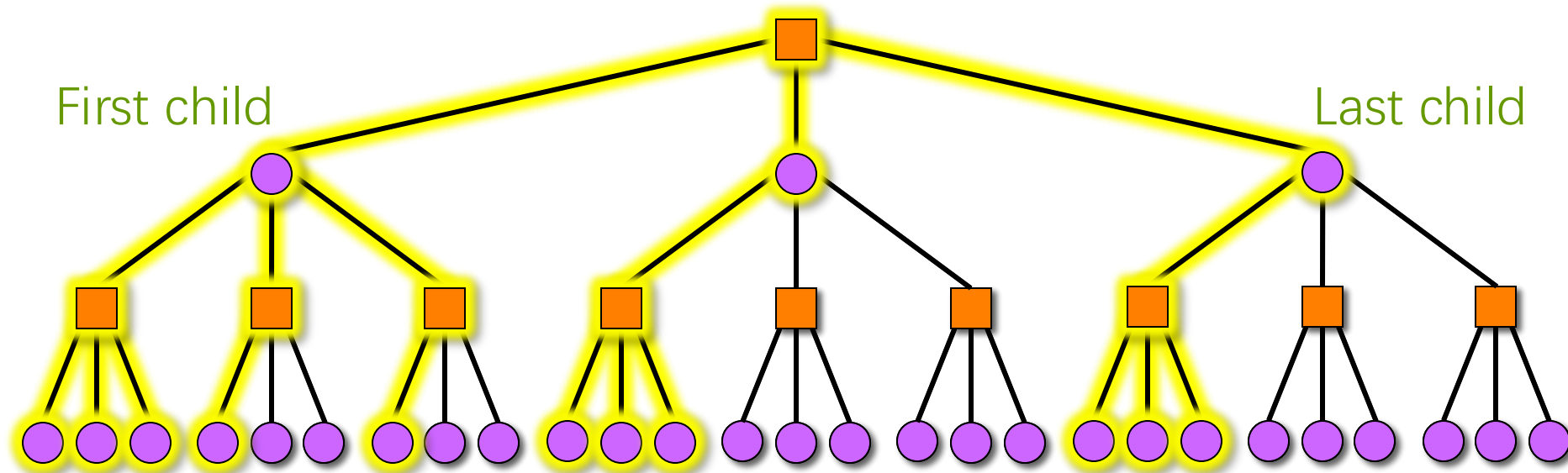
Best-Case Alpha-Beta Analysis



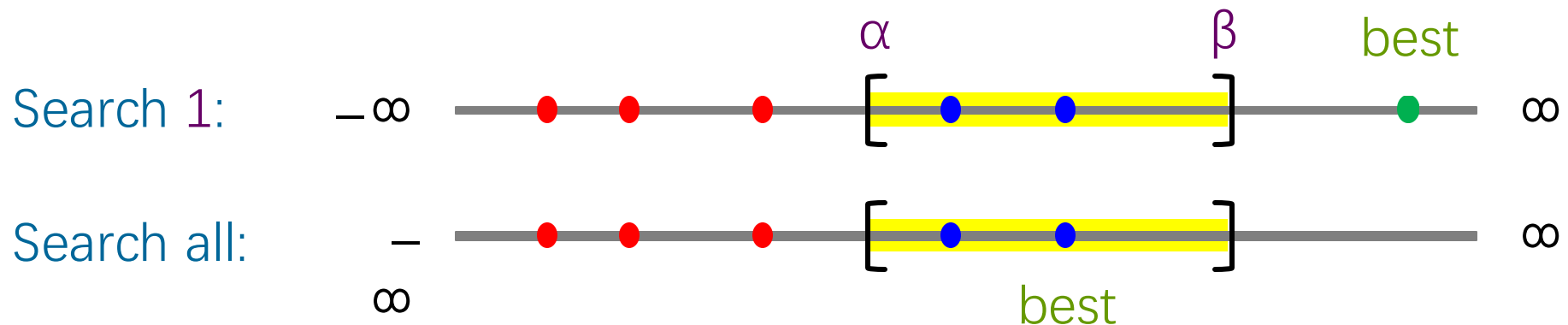
What that means: Since the naive algorithm examines about b^d nodes and alpha-beta examines only about $2b^{d/2} = 2\sqrt{b^d} = 2(b^{d/2})$ nodes, alpha-beta effectively

- doubles the search depth,
- square-roots the work,
- square-roots the branching factor.

Best-Ordered Trees



Observation: In a **best-ordered tree** (first child is best), the degree of every node is either 1 or b (maximal).



SPECULATIVE PARALLELISM



Searching an Unsorted Set

```
bool find(int *S, int n, int key) {  
    // S[] is an unsorted set of ints  
    assert(n > 0);  
  
    bool found = false;  
    for (int i = 0; i < n; ++i) {  
        if (S[i] == key) found = true;  
    }  
    return found;  
}
```

Short-Circuiting

Short-circuit optimization (Bentley rule)

- Quit the loop early if you find the element.

```
bool find(int *S, int n, int key) {  
    // S[] is an unsorted set of ints  
    assert(n > 0);  
  
    for (int i = 0; i < n; ++i) {  
        if (S[i] == key) return true;  
    }  
    return false;  
}
```

Question

- How can we parallelize this loop?

Short-Circuit Using D&C

```
void p_find_helper(int *S, int n, int key, bool* found) {
    if (*found) return;
    if (n == 1) {
        if (S[n] == key) *found = true;
    } else cilk_scope {
        cilk_spawn p_find_helper(S, n/2, key, found);
        if (*found) return;
        p_find_helper(S + n/2, n - n/2, key, found);
    }
    return;
}

bool p_find(int *S, int n, int key) {
    bool found = false;
    p_find_helper(S, n, key, &found);
    return found;
}
```

Short-Circuit Using D&C

```
void p_find_helper(int *S, int n, int key, bool* found) {
    if (*found) return;
    if (n == 1) {
        if (S[n] == key) *found = true;
    } else cilk_scope {
        cilk_spawn p_find_helper(S, n/2, key, found);
        if (*found) return;
        p_find_helper(S + n/2, n/2, key, found);
    }
    return;
}

bool p_find(int *S, int n, int key) {
    bool found = false;
    p_find_helper(S, n, key, &found);
    return found;
}
```

Notes

-  Nondeterministic code! 
- The benign race on ***found** is dangerous and technically undefined behavior in C.
 - Should use **int* found** (atomic).
- Is a memory fence necessary?
- Cilksan:
 - `__cilksan_disable_checking()`
 - `__cilksan_enable_checking()`

Speculative Parallelism

Definition

Speculative parallelism occurs when a program spawns a parallel task that would not be performed by the serial projection.

Rule of Thumb

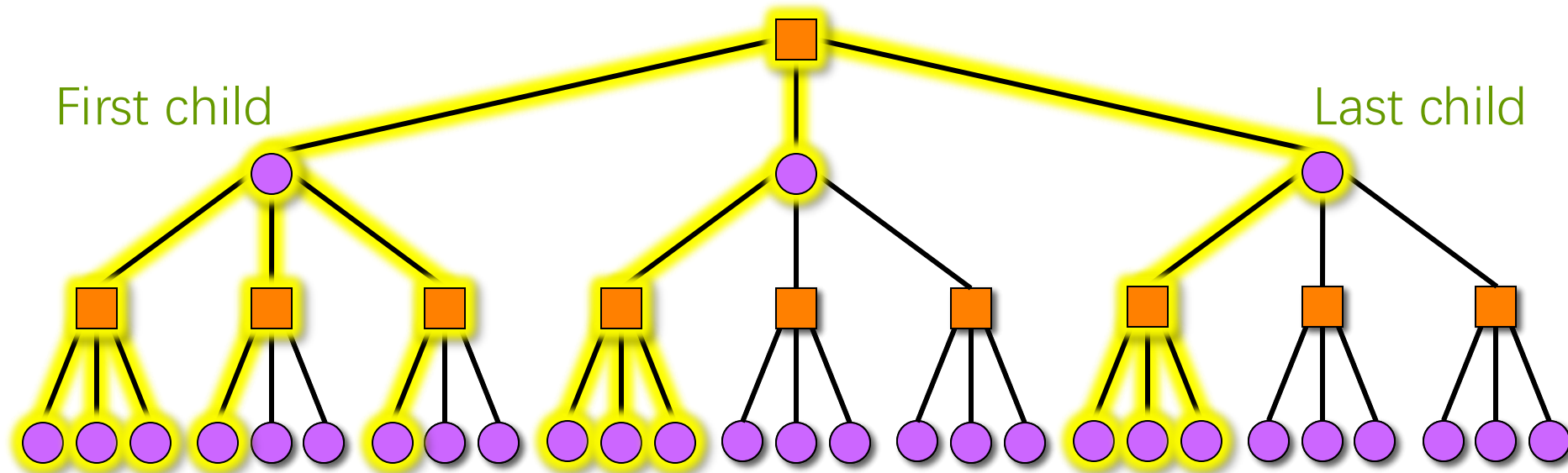
Don't spawn speculative work unless there is little other opportunity for parallelism and there's a good chance you need it.



PARALLEL ALPHA-BETA SEARCH



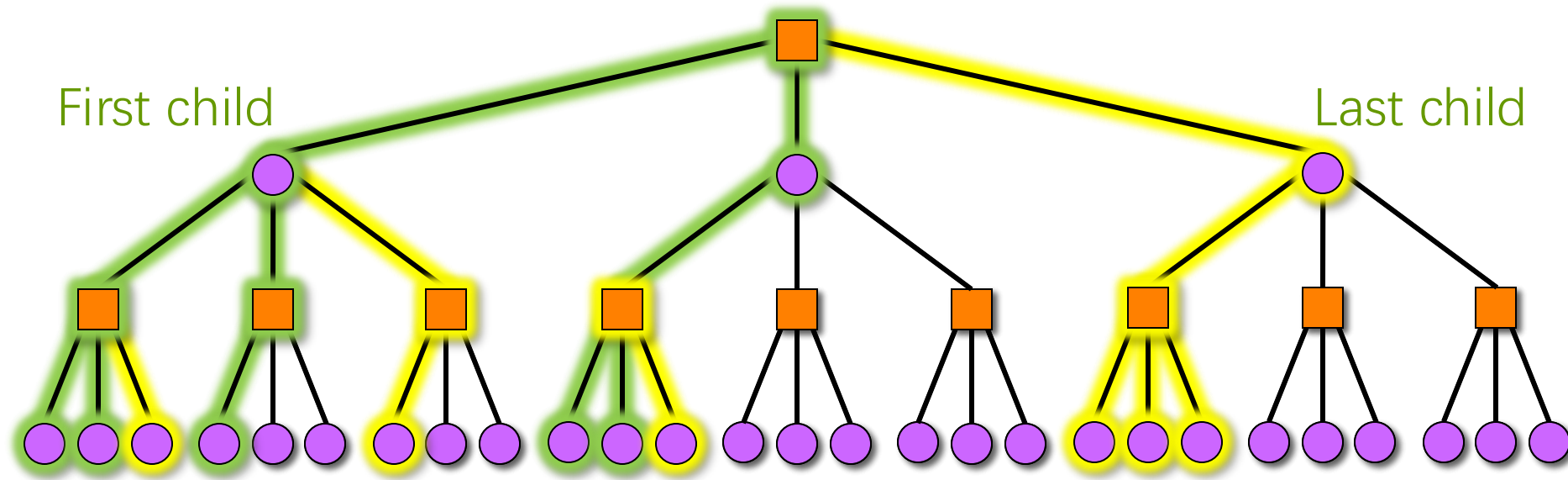
Parallel Alpha-Beta



Observation: In a **best-ordered tree** (first child is best), the degree of every node is either 1 or b (maximal).

Idea [FMMV89]: **Young siblings wait** — If the first (hope-fully best) child fails to generate a beta-cutoff, speculate that the node is maximal, and search the remaining children in parallel. Bet that you won't waste any work.

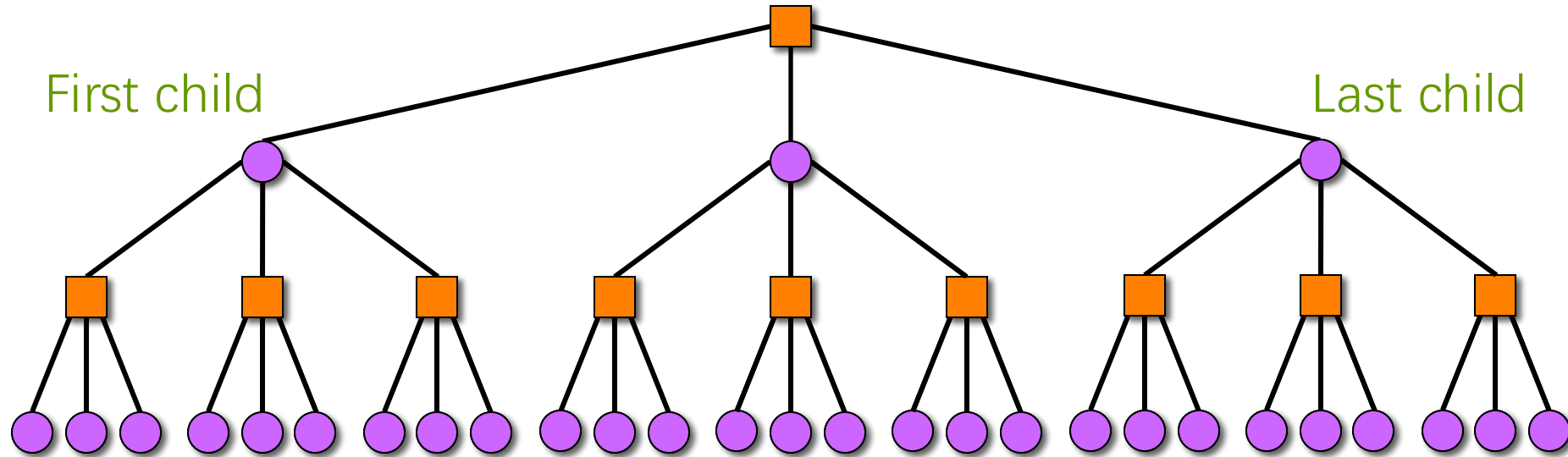
Parallelism of Young Siblings Wait



Analysis. If we search all maximal nodes' children 2 through b in parallel, the span is just like the work of searching a tree with branching factor 2 . Thus, $T_\infty = 2 \cdot 2^{d/2}$, and parallelism $= 2 \cdot b^{d/2} / (2 \cdot 2^{d/2}) = (b/2)^{d/2}$.

Example. If we search 1 billion nodes in a depth- 10 search, we might hope to achieve a parallelism of $10^9/64 \approx 16M$, which is more than ample.

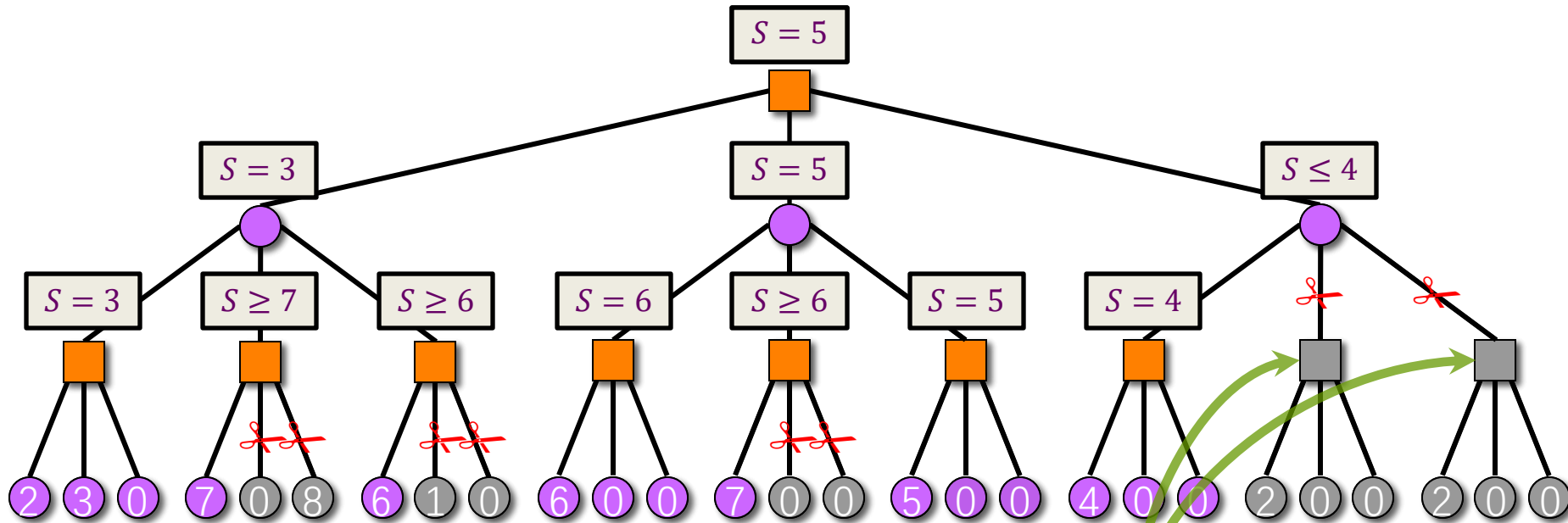
Back to Reality



But what if our speculation is wrong?

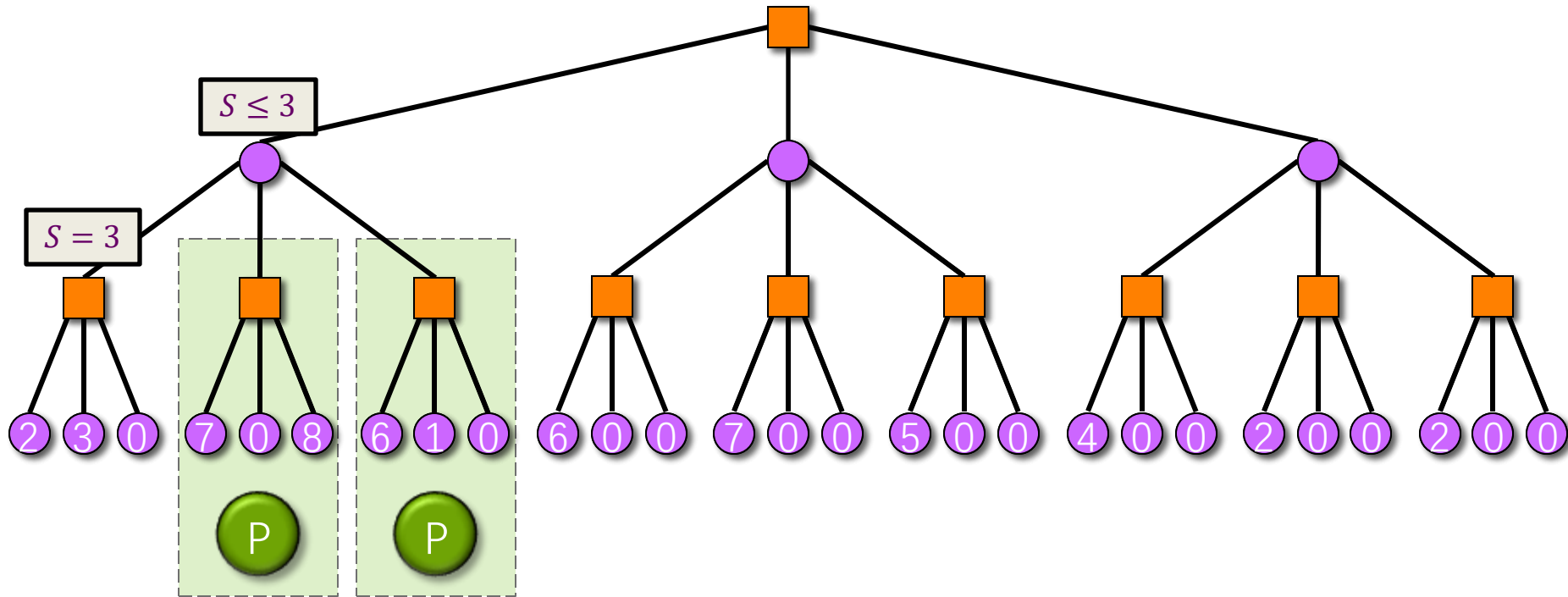
- In general, game trees are not best-ordered.
- Parallel alpha-beta search with young-siblings-wait wastes work if the best child is not first.

Alpha-Beta Search: Example



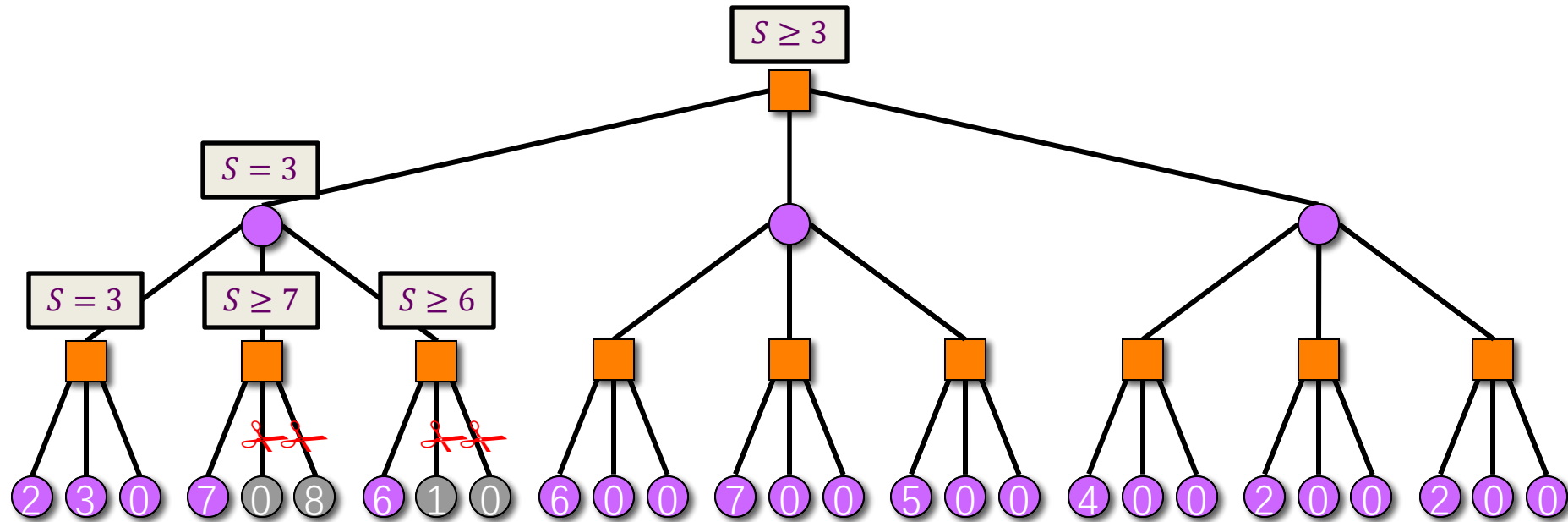
The second sibling of the root provides the cutoff.

Young Siblings Wait: Example

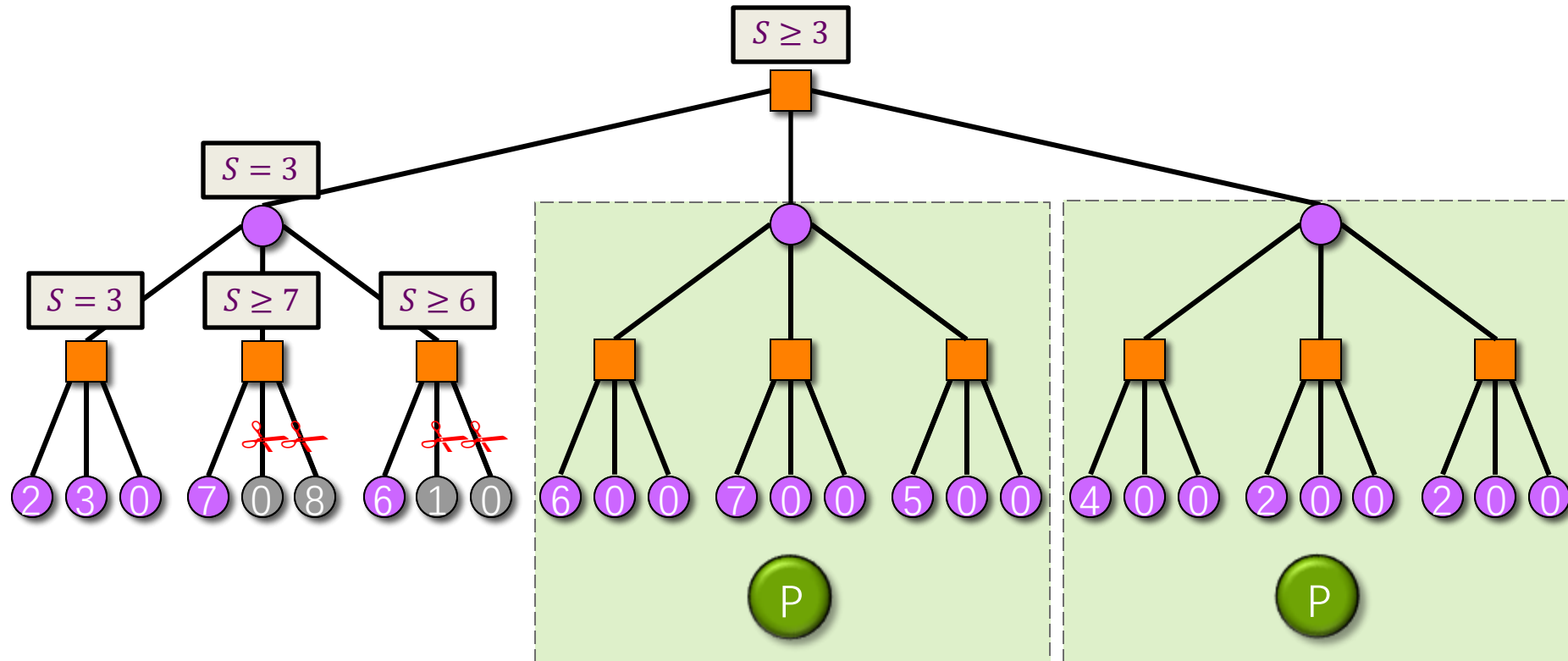


Parallel recursive
searches

Young Siblings Wait: Example

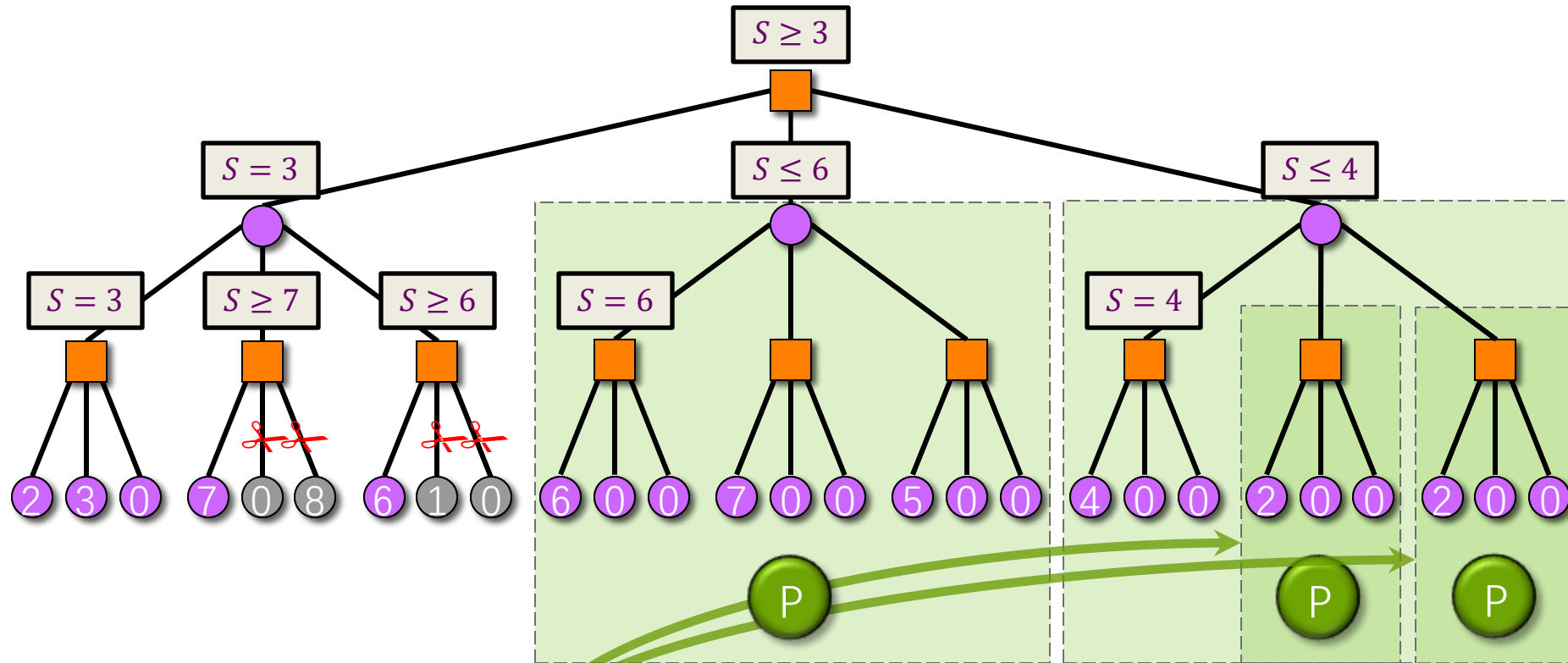


Young Siblings Wait: Example



Parallel recursive
searches

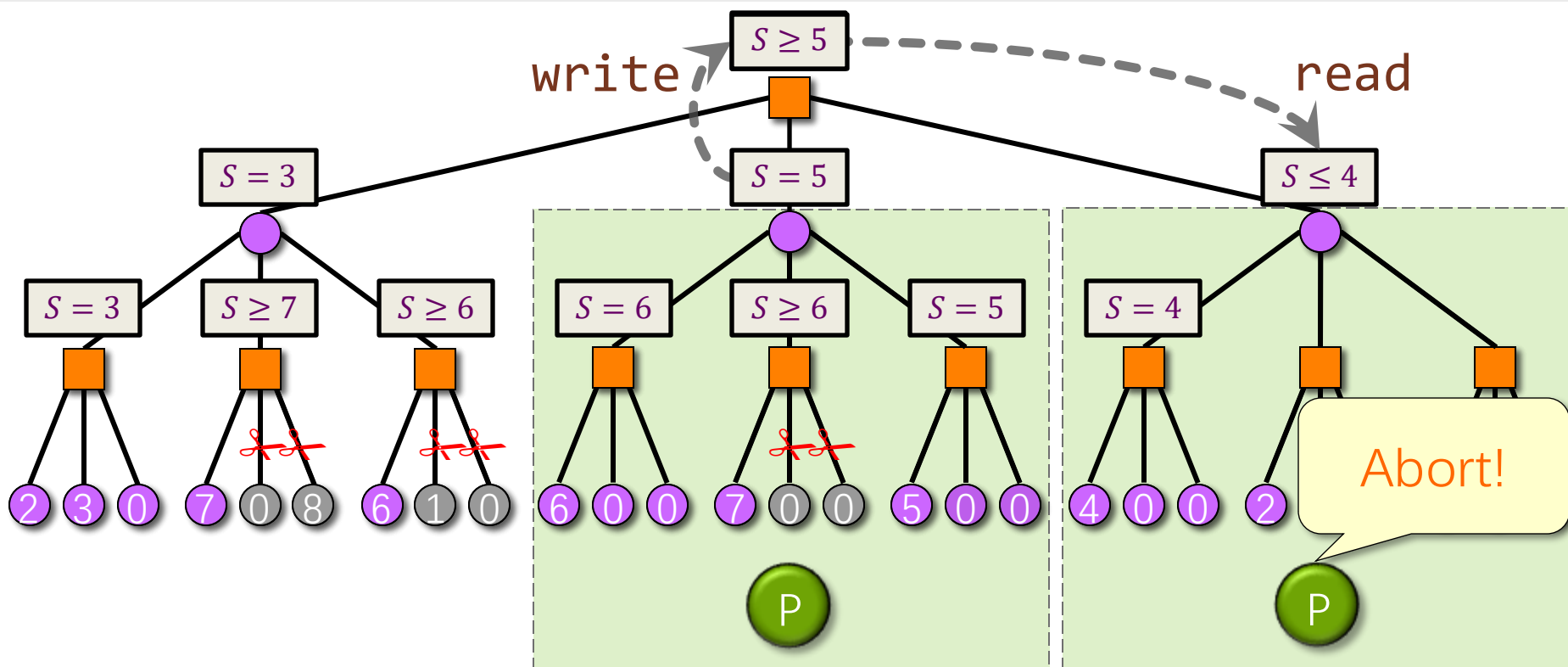
Young Siblings Wait: Example



A value from the second child of the root is not yet available to prune these searches.

Parallel recursive searches

Aborting Unnecessary Work



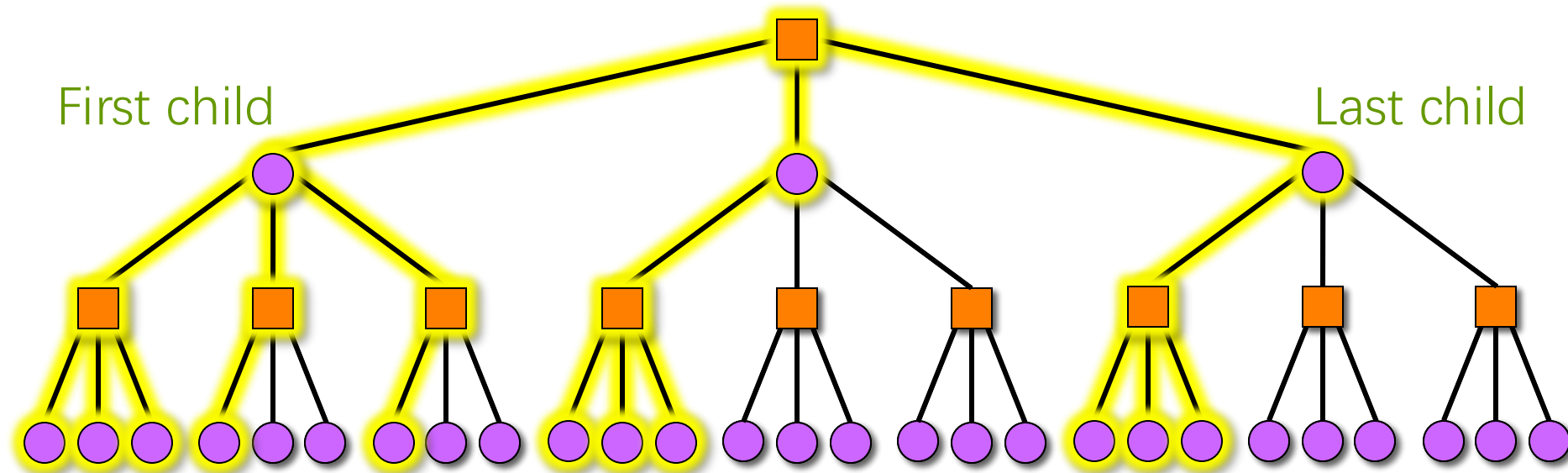
Possible idea: Nodes propagate their scores up the tree to keep alpha/beta values current.

- Poll up the tree and abort if possible.

Problem: Difficult to implement correctly.

Problem: Efficiency relies on lucky scheduling!

Wasted Work in Parallel Alpha-Beta



In practice, speculation in parallel alpha-beta search always wastes some work. Balance two conflicting goals:

- Avoid unnecessary work — **work-first principle**.
- Generate sufficient parallelism to guarantee a good parallel speedup — **small span**.